Kubernetes
==========
Source: /Users/malapalli_aravind@optum.com/Desktop/maravin4/SD/Sources/Kubernete
s

==========
FILE: K8s-App-Lifecycle-Story-Mode.md
==========

KUBERNETES APP LIFECYCLE THROUGH STORY MODE

> You have a React frontend and a Java Spring Boot backend running fine on your laptop. Then the app grows, traffic increases, deployments become risky, configs differ by environment, and scaling manually becomes painful. This guide explains the full Kubernetes app lifecycle in story mode, focused on the topics that matter most in interviews.

TABLE OF CONTENTS

- The App on Your Laptop (#1-the-app-on-your-laptop)
- Why Kubernetes Enters the Story (#2-why-kubernetes-enters-the-story)
- The Big Mental Model (#3-the-big-mental-model)
- Stage 1: Your First Pod (#4-stage-1-your-first-pod)
- Stage 2: ReplicaSet - Keep It Running (#5-stage-2-replicaset---keep-it-running)
- Stage 3: Deployment - Real App Management (#6-stage-3-deployment---real-app-management)
- Stage 4: Service - Stable Networking (#7-stage-4-service---stable-networking)
- Stage 5: Ingress - One Entry Point for the App (#8-stage-5-ingress---one-entry-point-for-the-app)
- Stage 6: ConfigMaps and Secrets - Separate Code from Configuration (#9-stage-6-configmaps-and-secrets---separate-code-from-configuration)
- Stage 7: Probes - Know When the App Is Ready (#10-stage-7-probes---know-when-the-app-is-ready)
- Stage 8: HPA - Scale When Traffic Grows (#11-stage-8-hpa---scale-when-traffic-grows)
- Stage 9: Rolling Update and Rollback (#12-stage-9-rolling-update-and-rollback)
- Stage 10: Namespaces and Production Structure (#13-stage-10-namespaces-and-production-structure)
- What Actually Happens on kubectl apply (#14-what-actually-happens-on-kubectl-apply)
- Putting It All Together: Full App Lifecycle (#15-putting-it-all-together-full-app-lifecycle)
- Common Mistakes and Troubleshooting (#16-common-mistakes-and-troubleshooting)
- Interview-Ready Answers (#17-interview-ready-answers)
- Quick Revision Sheet (#18-quick-revision-sheet)

---

1. THE APP ON YOUR LAPTOP

On your laptop, life is simple:

[Code Block (text)]
```
React frontend      → http://localhost:3000
Spring Boot backend → http://localhost:8080
```

```
PostgreSQL          → localhost:5432
```
[End Code Block]

You start things manually:

[Code Block (text)]
```
1. start postgres
2. start Spring Boot
3. start React
4. test the app in the browser
```
[End Code Block]

This works because:

- only one machine exists
- you know where every process is running
- if something crashes, you restart it manually
- scaling means opening another terminal, which is not real scaling

This is fine for development. It is not enough for production.

---

2. WHY KUBERNETES ENTERS THE STORY

Once the app goes to production, new problems appear:

[Code Block (text)]
```
Problem 1: The backend crashes at 2 AM.
           You need it to restart automatically.

Problem 2: Traffic grows.
           One backend instance is not enough anymore.

Problem 3: Pods are recreated frequently.
           Their IP addresses keep changing.
           How will the frontend or other services find them?

Problem 4: Production config is different from local config.
           DB URLs, API URLs, secrets, feature flags all differ.

Problem 5: You need zero-downtime deployments.
           Users should not see errors while a new version rolls out.

Problem 6: During traffic spikes, the app should scale out.
           During quiet periods, it should scale back in.
```
[End Code Block]

Kubernetes solves these problems by taking your application and managing its ful
l runtime lifecycle.

---

3. THE BIG MENTAL MODEL

If you remember only one thing, remember this:

[Code Block (text)]
```
Pod         = one running instance of your app
ReplicaSet  = keeps the desired number of Pods alive
```

```
Deployment  = manages ReplicaSets and handles updates/rollbacks
Service     = stable network identity in front of Pods
Ingress     = HTTP entry point into the cluster
ConfigMap   = non-sensitive config
Secret      = sensitive config
Probe       = health checks for startup/readiness/liveness
HPA         = scales number of Pods based on load
Namespace   = logical environment boundary
```
[End Code Block]

The core hierarchy is:

[Code Block (text)]
```
Deployment
   ↓
ReplicaSet
   ↓
Pods
   ↓
Containers
```
[End Code Block]

For networking:

[Code Block (text)]
```
Ingress → Service → Pods
```
[End Code Block]

For configuration:

[Code Block (text)]
```
ConfigMap / Secret → mounted into Pods
```
[End Code Block]

For scaling:

[Code Block (text)]
```
HPA → changes Deployment replica count → ReplicaSet creates more Pods
```
[End Code Block]

---

4. STAGE 1: YOUR FIRST POD

4.1 THE STORY

You containerize your Spring Boot app and say:

[Code Block (text)]
```
"I just want this app to run inside Kubernetes."
```
[End Code Block]

The smallest thing Kubernetes can run is a Pod.

Real-life analogy:

[Code Block (text)]
```
A Pod is like one apartment.
Inside the apartment, one or more people live together.
They share the same address, same utilities, and same lifecycle.
```

If the apartment is destroyed, everyone inside goes with it.
[End Code Block]

For most Java apps:

[Code Block (text)]
1 Pod = 1 Spring Boot container
[End Code Block]

4.2 WHY A POD ALONE IS NOT ENOUGH

If you create a Pod directly:

- it runs once
- if the Pod dies, Kubernetes does not recreate it automatically in a controlled higher-level way
- if you want 3 copies, you manually create 3 Pods
- if you want to update the image, you are doing it manually

That is why interviews often say:

[Code Block (text)]
"You almost never create Pods directly for application workloads."
[End Code Block]

4.3 MINIMAL POD EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: order-service
  labels:
    app: order-service
spec:
  containers:
    - name: order-service
      image: myrepo/order-service:v1
      ports:
        - containerPort: 8080
```
[End Code Block]

Good for learning. Not good for production lifecycle management.

---

5. STAGE 2: REPLICASET - KEEP IT RUNNING

5.1 THE STORY

Now you say:

[Code Block (text)]
"If one Pod dies, I want another one automatically."
"Actually I want 3 identical Pods always running."
[End Code Block]

That is the job of a ReplicaSet.

Real-life analogy:

[Code Block (text)]
ReplicaSet is like a shift manager.
The manager says: "I need exactly 3 cashiers on duty."

If one cashier leaves, the manager replaces them.
If there are 4 cashiers by mistake, the manager removes one.
[End Code Block]

5.2 WHAT REPLICASET REALLY DOES

[Code Block (text)]
Desired state: 3 Pods
Actual state: 2 Pods
Action: create 1 more Pod
[End Code Block]

This is Kubernetes' reconciliation model.

5.3 THE IMPORTANT INTERVIEW POINT

You should know ReplicaSet well, but you usually do not create it directly.

Why?

- ReplicaSet only keeps count correct
- it does not manage application version changes elegantly
- it does not give the clean rollout/rollback experience that Deployment gives

So the interview sentence is:

[Code Block (text)]
ReplicaSet maintains the number of Pods.
Deployment manages ReplicaSets and gives rolling updates and rollback.
[End Code Block]

---

6. STAGE 3: DEPLOYMENT - REAL APP MANAGEMENT

6.1 THE STORY

Now the app is real.
You need:

- 3 replicas
- safe upgrades
- rollback to previous version
- controlled rollout

That is where Deployment enters.

Real-life analogy:

[Code Block (text)]

Deployment is like the operations manager.
It does not directly serve customers.
It supervises the shift manager (ReplicaSet), who supervises workers (Pods).
[End Code Block]

6.2 WHY DEPLOYMENT IS THE DEFAULT FOR STATELESS APPS

For your Spring Boot backend or React frontend container:

- you usually use a Deployment
- it creates and manages ReplicaSets
- it keeps revision history
- it supports rolling updates

6.3 THE MOST IMPORTANT HIERARCHY

[Code Block (text)]
```
Deployment: order-service
   ├─ ReplicaSet: order-service-v1
   |    ├─ Pod A
   |    ├─ Pod B
   |    └─ Pod C
   └─ ReplicaSet: old-revision kept for rollback
```
[End Code Block]

When image changes from v1 to v2:

[Code Block (text)]
```
Old ReplicaSet scales down
New ReplicaSet scales up
```
[End Code Block]

That is the heart of rolling update.

6.4 DEPLOYMENT EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: myrepo/order-service:v1
          ports:
            - containerPort: 8080
```
[End Code Block]

6.5 WHAT TO SAY IN INTERVIEWS

==========

[Code Block (text)]
Use Deployment for stateless applications.
It manages ReplicaSets and gives rolling updates, rollback, and declarative scaling.
[End Code Block]

---

7. STAGE 4: SERVICE - STABLE NETWORKING

7.1 THE STORY

You now have 3 backend Pods.
Each Pod gets its own IP.
But Pods are replaceable. Their IPs change.

Problem:

[Code Block (text)]
How will the frontend reach the backend reliably?
How will one service call another if Pod IPs keep changing?
[End Code Block]

Answer: Service.

Real-life analogy:

[Code Block (text)]
Pods are people moving between hotel rooms.
Service is the hotel's front desk number.

You do not call room 312 directly because guests move.
You call the front desk, and it routes you to whoever is currently assigned.
[End Code Block]

7.2 WHAT SERVICE DOES

Service gives:

- a stable virtual IP
- a stable DNS name
- load balancing across matching Pods

Example:

[Code Block (text)]
order-service.default.svc.cluster.local
[End Code Block]

This always points to the current healthy Pods selected by labels.

7.3 THE FOUR TYPES YOU MUST KNOW

ClusterIP

[Code Block (text)]
Internal only.
Used for service-to-service communication inside the cluster.
This is the default and the most common one.
[End Code Block]

NodePort

[Code Block (text)]
Exposes the service on a port on every node.
Mostly used for quick testing, not ideal for production.
[End Code Block]

LoadBalancer

[Code Block (text)]
Creates a cloud load balancer in AWS/GCP/Azure.
Used for external production access.
[End Code Block]

Headless Service

[Code Block (text)]
No ClusterIP.
Returns Pod IPs directly.
Used mainly for StatefulSets.
[End Code Block]

7.4 SERVICE EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```
[End Code Block]

7.5 THE CRITICAL INTERVIEW SENTENCE

[Code Block (text)]
Pods are ephemeral. Services provide stable discovery and load balancing in front of Pods.
[End Code Block]

---

8. STAGE 5: INGRESS - ONE ENTRY POINT FOR THE APP

8.1 THE STORY

==========

Now you have:

- frontend service
- backend service
- maybe auth service

If every service gets its own external load balancer, cost and complexity grow.

So you introduce Ingress.

Real-life analogy:

[Code Block (text)]
```
Ingress is the receptionist at the building entrance.

Visitors say:
  "/api/orders"   → send to backend
  "/"             → send to frontend

The receptionist decides where HTTP traffic should go.
```
[End Code Block]

8.2 WHAT INGRESS DOES

Ingress provides:

- host-based routing
- path-based routing
- TLS termination
- one public entry for multiple internal services

Flow:

[Code Block (text)]
```
User → Ingress → Service → Pod
```
[End Code Block]

8.3 IMPORTANT CLARIFICATION

Ingress resource by itself does nothing.
You need an Ingress Controller like NGINX or AWS ALB Controller.

8.4 EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
```

```
              service:
                name: frontend-service
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 80
```
[End Code Block]

---

9. STAGE 6: CONFIGMAPS AND SECRETS - SEPARATE CODE FROM CONFIGURATION

9.1 THE STORY

Local app config is easy:

[Code Block (text)]
```
application.properties
.env file
hardcoded localhost URLs
```
[End Code Block]

Production is different:

- prod database URL
- prod Redis host
- JWT secret
- API keys
- feature flags

You do not want to rebuild the image for every environment.

That is why you use ConfigMaps and Secrets.

9.2 CONFIGMAP

ConfigMap stores non-sensitive configuration.

Examples:

- log level
- API base URL
- feature flags
- Spring profile settings

Real-life analogy:

[Code Block (text)]
```
ConfigMap is the instruction sheet pinned on the office wall.
Everyone can read it. It is operational information, not confidential.
```
[End Code Block]

Example:

[Code Block (yaml)]
```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: order-config
data:
  DB_HOST: postgres-service
  LOG_LEVEL: INFO
```
[End Code Block]

9.3 SECRET

Secret stores sensitive values.

Examples:

- DB password
- JWT signing key
- API tokens
- TLS keys

Real-life analogy:

[Code Block (text)]
```text
Secret is the office locker.
Only authorized people should access it.
```
[End Code Block]

Example:

[Code Block (yaml)]
```yaml
apiVersion: v1
kind: Secret
metadata:
  name: order-secrets
type: Opaque
stringData:
  DB_PASSWORD: super-secret-password
```
[End Code Block]

9.4 THE IMPORTANT SECURITY POINT

This matters a lot in interviews:

[Code Block (text)]
```text
Kubernetes Secret is base64 encoded, not encrypted by default.
For stronger security, enable etcd encryption at rest and preferably use an exte
rnal secret manager.
```
[End Code Block]

9.5 INJECTING INTO THE POD

[Code Block (yaml)]
```yaml
env:
  - name: DB_HOST
    valueFrom:
      configMapKeyRef:
        name: order-config
        key: DB_HOST
```

```
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: order-secrets
        key: DB_PASSWORD
```
[End Code Block]

9.6 ONE MORE IMPORTANT INTERVIEW DETAIL

[Code Block (text)]
```
ConfigMap/Secret mounted as files can refresh.
ConfigMap/Secret injected as env vars do not hot reload automatically.
Usually the Pod must restart to pick up env var changes.
```
[End Code Block]

---

10. STAGE 7: PROBES - KNOW WHEN THE APP IS READY

10.1 THE STORY

Your Spring Boot app takes time to start.
If Kubernetes sends traffic too early, users get failures.
If the app hangs, Kubernetes should restart it.

That is why probes exist.

10.2 THE THREE PROBES

Readiness Probe

[Code Block (text)]
```
Question: Can this Pod receive traffic right now?
If it fails: Pod stays running, but is removed from Service endpoints.
```
[End Code Block]

This is the most important one for zero-downtime deployments.

Liveness Probe

[Code Block (text)]
```
Question: Is the process alive and healthy?
If it fails: kubelet restarts the container.
```
[End Code Block]

Startup Probe

[Code Block (text)]
```
Question: Has the app finished booting?
If it fails repeatedly: container is restarted.
Used for slow-starting apps like Spring Boot.
```
[End Code Block]

10.3 REAL-LIFE ANALOGY

[Code Block (text)]
```
Startup probe   = "Has the shop opened for the day?"
Readiness probe = "Can customers enter right now?"
Liveness probe  = "Is the shop still functioning or totally stuck?"
```
[End Code Block]

10.4 SPRING BOOT-FRIENDLY EXAMPLE

[Code Block (yaml)]
```yaml
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 20

startupProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  failureThreshold: 30
  periodSeconds: 2
```
[End Code Block]

10.5 INTERVIEW GOLD POINT

[Code Block (text)]
```
Readiness protects traffic.
Liveness protects process health.
Startup protects slow application boot.
```
[End Code Block]

---

11. STAGE 8: HPA - SCALE WHEN TRAFFIC GROWS

11.1 THE STORY

Now traffic spikes.
Three Pods are not enough.

You do not want manual scaling like this:

[Code Block (text)]
```
kubectl scale deployment/order-service --replicas=10
```
[End Code Block]

every time load changes.

So you add HPA.

Real-life analogy:

[Code Block (text)]
```text
HPA is like a restaurant manager watching customer volume.
If the place gets crowded, the manager adds more staff.
If it becomes quiet, the manager reduces staff.
```
[End Code Block]

11.2 HOW HPA WORKS

[Code Block (text)]
```text
Metrics Server collects CPU/memory usage
      ↓
HPA checks metrics periodically
      ↓
If usage is above target, HPA increases replicas
      ↓
Deployment updates replica count
      ↓
ReplicaSet creates more Pods
```
[End Code Block]

11.3 EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```
[End Code Block]

11.4 THE BIG INTERVIEW WARNING

[Code Block (text)]
```text
HPA needs resource requests.
Without CPU/memory requests on the container, HPA cannot calculate utilization c
orrectly.
```
[End Code Block]

11.5 ANOTHER IMPORTANT REALITY

HPA is not magic.

- it reacts after load rises
- new Pods need startup time
- burst traffic is not handled instantly

So the mature answer is:

[Code Block (text)]
HPA improves elasticity, but it is reactive, not instant.
[End Code Block]

---

12. STAGE 9: ROLLING UPDATE AND ROLLBACK

12.1 THE STORY

You deploy version v2.
If it breaks, users should still be served by v1.

That is the purpose of Deployment rolling updates.

12.2 WHAT HAPPENS DURING ROLLING UPDATE

[Code Block (text)]
Current state:
  ReplicaSet v1 → 3 Pods

Deploy v2:
  Deployment creates new ReplicaSet v2
  v2 Pods come up gradually
  once ready, old v1 Pods are scaled down gradually
[End Code Block]

Key settings:

[Code Block (text)]
maxUnavailable = how many old Pods may be unavailable during rollout
maxSurge   = how many extra new Pods may be created temporarily
[End Code Block]

Best practical setting for many APIs:

[Code Block (text)]
maxUnavailable: 0
maxSurge: 1
[End Code Block]

12.3 WHY READINESS PROBE IS CRITICAL HERE

Without readiness:

- Kubernetes may think the new Pod is ready too soon
- old Pod gets removed
- users get failed requests

So the interview-quality answer is:

[Code Block (text)]

Zero-downtime rollout depends on Deployment strategy plus readiness probe.
[End Code Block]

12.4 ROLLBACK

If v2 is broken:

[Code Block (bash)]
```bash
kubectl rollout undo deployment/order-service
```
[End Code Block]

Why rollback is possible:

- Deployment keeps previous ReplicaSet revisions
- rollback means scaling the healthy old ReplicaSet back up

12.5 GOOD PRODUCTION EXTRAS

- terminationGracePeriodSeconds
- graceful shutdown in Spring Boot
- preStop hook
- PodDisruptionBudget

These are not the first concepts to memorize, but they make your answer stronger in interviews.

---

13. STAGE 10: NAMESPACES AND PRODUCTION STRUCTURE

13.1 THE STORY

Now the company has:

- dev environment
- staging environment
- prod environment

You do not want all workloads mixed together.

So you use Namespaces.

Real-life analogy:

[Code Block (text)]
```text
Namespaces are like separate office floors in the same building.
Each floor has its own teams, configs, and access control.
```
[End Code Block]

13.2 WHY NAMESPACES MATTER

They help with:

- logical isolation
- environment separation
- RBAC separation
- quotas and limits

- cleaner operations

Typical model:

[Code Block (text)]
dev
staging
production
[End Code Block]

13.3 INTERVIEW POINT

[Code Block (text)]
Namespaces are not strong security boundaries by themselves.
They are logical isolation boundaries and work best with RBAC, quotas, and polic
ies.
[End Code Block]

---

14. WHAT ACTUALLY HAPPENS ON KUBECTL APPLY

This is one of the strongest senior-level Kubernetes explanations.

14.1 THE LIFECYCLE

You run:

[Code Block (bash)]
kubectl apply -f deployment.yaml
[End Code Block]

Then:

Step 1: kubectl sends the request to the API server

[Code Block (text)]
kubectl authenticates using kubeconfig and sends the Deployment spec.
[End Code Block]

Step 2: API server validates and stores it

[Code Block (text)]
API server authenticates the caller
API server authorizes via RBAC
Admission controllers may mutate/validate the object
Object is stored in etcd
[End Code Block]

Step 3: Deployment controller reacts

[Code Block (text)]
It sees a new Deployment and creates a ReplicaSet.
[End Code Block]

Step 4: ReplicaSet controller reacts

[Code Block (text)]
It sees desired replicas and creates Pod objects.
[End Code Block]

Step 5: Scheduler assigns Pods to nodes

[Code Block (text)]
Pending Pods are matched to suitable worker nodes.
[End Code Block]

Step 6: Kubelet runs the containers

[Code Block (text)]
The node's kubelet pulls the image, mounts volumes,
injects Secrets/ConfigMaps, starts the container, and reports status.
[End Code Block]

Step 7: Service starts routing traffic

[Code Block (text)]
If a Service selects the Pod labels and readiness passes,
the Pod becomes an endpoint behind the Service.
[End Code Block]

14.2 THE SENIOR-LEVEL MENTAL MODEL

[Code Block (text)]
Kubernetes is declarative and controller-driven.
You submit desired state.
Controllers continuously work to make actual state match it.
[End Code Block]

---

15. PUTTING IT ALL TOGETHER: FULL APP LIFECYCLE

Let us walk the whole application story end to end.

15.1 STAGE-BY-STAGE EVOLUTION

Stage A: Laptop

[Code Block (text)]
React on localhost:3000
Spring Boot on localhost:8080
Everything manual
[End Code Block]

Stage B: Containerized App

[Code Block (text)]
Docker image created for backend and frontend

Now the app is portable
[End Code Block]

Stage C: Single Pod in Kubernetes

[Code Block (text)]
The app can run in the cluster, but there is no real lifecycle management.
[End Code Block]

Stage D: Deployment with 3 Replicas

[Code Block (text)]
Now Kubernetes maintains 3 running copies of your backend.
If one dies, another is created.
Rolling update becomes possible.
[End Code Block]

Stage E: Service in Front

[Code Block (text)]
The backend gets a stable DNS identity.
Frontend and other services no longer care about Pod IP changes.
[End Code Block]

Stage F: Ingress for External Traffic

[Code Block (text)]
Users enter through one HTTP entry point.
Ingress routes to frontend and backend services.
[End Code Block]

Stage G: ConfigMaps and Secrets

[Code Block (text)]
Same Docker image runs in dev/staging/prod.
Only config and secrets change per environment.
[End Code Block]

Stage H: Probes Added

[Code Block (text)]
Traffic is sent only to healthy Pods.
Hung containers restart automatically.
Slow Java startup stops causing false failures.
[End Code Block]

Stage I: HPA Added

[Code Block (text)]
Traffic spike → CPU rises → HPA increases replicas.
Traffic drops → HPA reduces replicas.
[End Code Block]

Stage J: Safe Release Process

[Code Block (text)]
```
New version rolls out gradually.
If broken, rollback to previous ReplicaSet.
```
[End Code Block]

15.2 FINAL PRODUCTION FLOW

[Code Block (text)]
```
User
  ↓
Ingress
  ↓
frontend-service → frontend Pods
  ↓
order-service → backend Pods managed by Deployment
  ↓
Pods read ConfigMap and Secret
  ↓
Readiness/Liveness/Startup probes protect lifecycle
  ↓
HPA scales replicas based on load
  ↓
Deployment handles rollout and rollback
```
[End Code Block]

That is the lifecycle story most interviewers want to hear.

---

16. COMMON MISTAKES AND TROUBLESHOOTING

16.1 "WE CREATED PODS DIRECTLY"

[Code Block (text)]
```
Problem: no proper rollout, rollback, or higher-level management.
Fix: use Deployment for stateless apps.
```
[End Code Block]

16.2 "SERVICE IS NOT ROUTING TRAFFIC"

Check:

[Code Block (text)]
```
1. Does Service selector match Pod labels?
2. Are Pods passing readiness probe?
3. Does kubectl get endpoints show Pod IPs?
```
[End Code Block]

16.3 "APP IS IN CRASHLOOPBACKOFF"

Check:

[Code Block (text)]
```
kubectl describe pod <pod>
kubectl logs <pod> --previous
```
[End Code Block]

Typical causes:

- wrong config
- bad secret
- DB unreachable
- app startup failure
- liveness probe too aggressive
- OOMKilled

16.4 "HPA IS NOT SCALING"

Check:

- resource requests exist
- Metrics Server is installed
- current load actually exceeds target
- new Pods are able to start

16.5 "NEW ROLLOUT CAUSED ERRORS"

Most common reasons:

- readiness probe missing or wrong
- image broken
- config mismatch
- incompatible DB change

16.6 STRONG DEBUG SEQUENCE

[Code Block (text)]
```
Pod issue?        → describe pod, logs, events
Service issue?    → get svc, get endpoints, check labels/readiness
Scheduling issue? → check Pending events, node capacity, affinity/taints
Scaling issue?    → check HPA, metrics, resource requests
Rollout issue?    → rollout status, rollout history, undo if needed
```
[End Code Block]

---

17. INTERVIEW-READY ANSWERS

17.1 "WHAT IS THE DIFFERENCE BETWEEN POD, REPLICASET, AND DEPLOYMENT?"

[Code Block (text)]
```
Pod is the smallest deployable unit and runs the container.
ReplicaSet ensures the desired number of Pods are running.
Deployment manages ReplicaSets and adds rolling updates, rollback, and declarati
ve app management.
```
[End Code Block]

17.2 "WHY DO WE NEED A SERVICE IF PODS ALREADY HAVE IPS?"

[Code Block (text)]
```
Pods are ephemeral and their IPs change on recreation.
Service gives a stable virtual IP and DNS name, and load balances across healthy
```

Pods.
[End Code Block]

17.3 "WHAT IS THE DIFFERENCE BETWEEN CONFIGMAP AND SECRET?"

[Code Block (text)]
ConfigMap stores non-sensitive configuration.
Secret stores sensitive data like passwords or tokens.
Kubernetes Secrets are base64 encoded, not strongly encrypted by default, so production setups should also use encryption at rest and often external secret managers.
[End Code Block]

17.4 "WHAT IS THE DIFFERENCE BETWEEN READINESS AND LIVENESS PROBE?"

[Code Block (text)]
Readiness controls whether the Pod should receive traffic.
Liveness controls whether the container should be restarted.
Startup probe protects slow-booting apps before liveness kicks in.
[End Code Block]

17.5 "HOW DOES HPA WORK?"

[Code Block (text)]
HPA reads metrics like CPU or memory, compares them to target utilization, and changes the Deployment replica count. The Deployment then creates or removes Pods through its ReplicaSet. It needs resource requests to work correctly.
[End Code Block]

17.6 "HOW DO ZERO-DOWNTIME DEPLOYMENTS WORK IN KUBERNETES?"

[Code Block (text)]
Deployment performs a rolling update by creating a new ReplicaSet and scaling it up while scaling the old one down. Readiness probe is critical because traffic should go only to Pods that are truly ready. Rollback is possible because previous ReplicaSets are kept as revision history.
[End Code Block]

17.7 "WALK ME THROUGH KUBECTL APPLY INTERNALLY"

[Code Block (text)]
kubectl sends the resource to the API server, the API server validates and stores it in etcd, controllers observe the new desired state and create ReplicaSets and Pods, the scheduler assigns Pods to nodes, kubelet runs the containers, and Services start routing traffic once Pods are ready.
[End Code Block]

---

18. QUICK REVISION SHEET

ONE-LINE MAPPING

[Code Block (text)]
Pod   = one running instance of the app

```
ReplicaSet  = maintains Pod count
Deployment  = rollout, rollback, replica management
Service     = stable identity and load balancing for Pods
Ingress     = HTTP entry point and routing
ConfigMap   = non-sensitive config
Secret      = sensitive config
Readiness   = should this Pod get traffic?
Liveness    = should this container be restarted?
Startup     = has app finished booting?
HPA         = auto-scale replica count
Namespace   = logical isolation boundary
```
[End Code Block]

THE LIFECYCLE IN ONE FLOW
==========

[Code Block (text)]
```
Developer writes YAML
  ↓
kubectl apply
  ↓
API server stores desired state
  ↓
Deployment creates ReplicaSet
  ↓
ReplicaSet creates Pods
  ↓
Scheduler picks nodes
  ↓
Kubelet starts containers
  ↓
Service routes traffic to ready Pods
  ↓
Ingress exposes app externally
  ↓
HPA scales replicas when load changes
  ↓
Deployment rolls forward or back during releases
```
[End Code Block]

GOLD STANDARD ANSWER
==========

[Code Block (text)]
```
In Kubernetes, I typically run stateless applications using Deployments. The Dep
loyment manages ReplicaSets, and ReplicaSets maintain the desired number of Pods
. Services give the Pods a stable network identity, and Ingress exposes the app
externally with host or path-based routing. ConfigMaps and Secrets separate conf
iguration from the image, probes protect startup and traffic safety, and HPA sca
les the number of replicas based on load. During releases, rolling updates and r
ollback are handled by the Deployment using ReplicaSet revision history.
```
[End Code Block]

==========
FILE: K8s-Complete-Refresher.md
==========

KUBERNETES — COMPLETE KNOWLEDGE REFRESHER (JAVA FULL STACK ENGINEER)
==========

> Everything you need to know about K8s — architecture, objects, networking, sto

rage, security, scaling, Java-specific concerns, and production operations. Interview-calibrated for 7+ YOE.

TABLE OF CONTENTS
==========

- What is Kubernetes & Why It Exists (#1-what-is-kubernetes--why-it-exists)
- Architecture — The Full Picture (#2-architecture--the-full-picture)
- Core Objects — Workloads (#3-core-objects--workloads)
- Networking — How Traffic Flows (#4-networking--how-traffic-flows)
- Storage — Persistence in K8s (#5-storage--persistence-in-k8s)
- Configuration — ConfigMaps, Secrets, Env (#6-configuration--configmaps-secrets-env)
- Scheduling — Where Pods Land (#7-scheduling--where-pods-land)
- Scaling — HPA, VPA, Cluster Autoscaler (#8-scaling--hpa-vpa-cluster-autoscaler)
- Security — RBAC, Pod Security, Secrets (#9-security--rbac-pod-security-secrets)
- Observability — Probes, Logs, Monitoring (#10-observability--probes-logs-monitoring)
- Deployment Strategies — Zero Downtime (#11-deployment-strategies--zero-downtime)
- Helm — Package Management (#12-helm--package-management)
- Java on Kubernetes — JVM-Specific Concerns (#13-java-on-kubernetes--jvm-specific-concerns)
- Namespaces, Resource Quotas & Limit Ranges (#14-namespaces-resource-quotas--limit-ranges)
- CRDs & Operators (#15-crds--operators)
- Service Mesh (Istio Basics) (#16-service-mesh-istio-basics)
- CI/CD with Kubernetes (#17-cicd-with-kubernetes)
- Troubleshooting Playbook (#18-troubleshooting-playbook)
- Essential Commands Cheat Sheet (#19-essential-commands-cheat-sheet)

---

1. WHAT IS KUBERNETES & WHY IT EXISTS
==========

Kubernetes (K8s) is a container orchestration platform. It automates deployment, scaling, networking, and management of containerized applications.

Before K8s:

[Code Block]
```
Problem: You have 50 microservices, each in Docker containers.
  - Which server does each container run on?
  - What if a server dies? Who restarts the containers?
  - How do containers find each other (service discovery)?
  - How do you scale from 3 to 30 replicas under load?
  - How do you roll out a new version without downtime?
  - How do you manage secrets, configs, storage?

Answer: You do it all manually. Or you build K8s.
```
[End Code Block]

What K8s solves:

Concern	K8s Solution
Where to run containers	Scheduler places Pods on nodes

Self-healing	Restarts crashed containers, replaces dead nodes
Service discovery	DNS + Services
Load balancing	kube-proxy + Services
Scaling	HPA, VPA, Cluster Autoscaler
Rolling updates	Deployment controller
Config management	ConfigMaps + Secrets
Storage	PersistentVolumes
Access control	RBAC

As a Java dev, you care because: Your Spring Boot / Quarkus / Micronaut apps run in containers, deployed to K8s clusters in production. You write Dockerfiles, Helm charts, K8s manifests, and configure probes, resources, and environment. K8s is your runtime.

---

2. ARCHITECTURE — THE FULL PICTURE

A K8s cluster has two planes: Control Plane (brain) and Worker Nodes (muscle).

[Code Block]

```
┌──────────────────────────── CONTROL PLANE ────────────────────────────┐
│                                                                        │
│   ┌──────────────┐   ┌──────────────────┐   ┌────────────────┐       │
│   │ kube-apiserver│  │ kube-controller  │  │ kube-scheduler│        │
│   │              │   │    manager       │   │                │       │
│   └──────────────┘   └──────────────────┘   └────────────────┘       │
│          │                                                             │
│          │           ┌────────────────────┐                            │
│          └──────────│       etcd         │                            │
│                      │ (key-value store) │                            │
│                      └────────────────────┘                            │
│                                                                        │
│   ┌────────────────────┐ (optional)                                   │
│   │ cloud-controller  │                                               │
│   │    manager        │                                               │
│   └────────────────────┘                                               │
└────────────────────────────────────────────────────────────────────────┘

┌──────────────────── WORKER NODE 1 ────────────────────┐
│   ┌──────────┐   ┌──────────────┐   ┌──────────────┐   │
│   │ kubelet  │   │ kube-proxy  │   │ container   │   │
│   │          │   │              │   │  runtime    │   │
│   └──────────┘   └──────────────┘   │(containerd)│   │
│                                      └──────────────┘   │
│   ┌────────┐ ┌────────┐ ┌────────┐                     │
│   │ Pod A  │ │ Pod B  │ │ Pod C  │                     │
│   └────────┘ └────────┘ └────────┘                     │
└────────────────────────────────────────────────────────┘

┌──────────────────── WORKER NODE 2 ────────────────────┐
│  (same structure: kubelet, kube-proxy, runtime)│
│   ┌────────┐ ┌────────┐                                │
│   │ Pod D  │ │ Pod E  │                                │
│   └────────┘ └────────┘                                │
└────────────────────────────────────────────────────────┘
```

[End Code Block]

2.1 CONTROL PLANE COMPONENTS

kube-apiserver

[Code Block]
```
Role: The FRONT DOOR of the cluster. Every operation goes through it.
  - kubectl commands → API server
  - Controllers → API server
  - Kubelet → API server
  - Dashboard/CI-CD → API server

What it does:
  1. Receives all REST API requests
  2. Authenticates the caller (certs, tokens, OIDC)
  3. Authorizes the request (RBAC)
  4. Runs Admission Controllers (mutate / validate)
  5. Persists the object to etcd
  6. Returns response to caller

Key fact: It's the ONLY component that talks to etcd directly.
         All other components talk to the API server.
```
[End Code Block]

etcd

[Code Block]
```
Role: Distributed key-value store. THE source of truth for all cluster state.

What's stored:
  /registry/pods/default/my-pod         → Pod spec and status
  /registry/deployments/default/my-app  → Deployment definition
  /registry/services/default/my-svc     → Service definition
  /registry/secrets/default/db-pass     → Secret data

Key facts:
  - Uses Raft consensus (needs quorum: run 3 or 5 nodes in production)
  - Latency-sensitive — needs fast SSD disks
  - All data persisted here — lose etcd, lose the cluster
  - Take regular snapshots: etcdctl snapshot save backup.db
  - Only the API server reads/writes etcd

If etcd goes down:
  - Existing running containers KEEP running (kubelet is independent)
  - But no new deployments, no scaling, no healing, no scheduling
  - Cluster is effectively frozen until etcd recovers
```
[End Code Block]

kube-controller-manager

[Code Block]
```
Role: Runs all the controllers — the reconciliation loops that make
      the ACTUAL state match the DESIRED state.

Controllers inside it (each runs as a goroutine):
  - Deployment Controller: watches Deployments → manages ReplicaSets
  - ReplicaSet Controller: watches ReplicaSets → creates/deletes Pods
  - Node Controller: monitors node health, marks NotReady, evicts Pods
  - Job Controller: watches Jobs → creates Pods for batch work
  - EndpointSlice Controller: watches Services → updates endpoint lists
  - ServiceAccount Controller: creates default SA for new namespaces
  - Namespace Controller: cleans up resources when namespace is deleted
```

```
How it works (reconciliation loop):
  while true:
    desired_state = read from etcd (via API server)
    actual_state  = observe current reality
    if actual != desired:
      take action to converge (create Pod, delete Pod, etc.)
    sleep(interval)

This is WHY K8s is declarative — you declare desired state, controllers converge
.
```
[End Code Block]

kube-scheduler

[Code Block]

```
Role: Decides WHICH node a new Pod runs on.

Only acts on Pods that have no nodeName assigned (Pending Pods).

Two phases:
  Phase 1: FILTERING (eliminate unsuitable nodes)
    - Enough CPU/memory?
    - Pod tolerates node's taints?
    - Node affinity matches?
    - PVC zone affinity?
    - Not cordoned/unschedulable?

  Phase 2: SCORING (rank surviving nodes)
    - LeastRequestedPriority (prefer emptier nodes)
    - BalancedResourceAllocation (even CPU/memory ratio)
    - InterPodAffinity (prefer co-location if specified)
    - TopologySpreadConstraints (spread across zones)

  Result: Pod.spec.nodeName = <best-scoring-node>

Does NOT run containers — that's kubelet's job.
```
[End Code Block]

cloud-controller-manager

[Code Block]

```
Role: Integrates with cloud provider APIs (AWS/GCP/Azure).

Handles:
  - LoadBalancer Services → provisions cloud LB (ELB, ALB, etc.)
  - Node lifecycle → detects when a cloud VM is terminated
  - Route management → sets up cloud network routes for Pod CIDRs
  - Volume provisioning → creates cloud disks (EBS, GCE PD)

Only exists in cloud-managed clusters (EKS, GKE, AKS).
Not present in bare-metal or on-prem clusters.
```
[End Code Block]

2.2 WORKER NODE COMPONENTS
==========

kubelet

[Code Block]
```
Role: The agent on every worker node. Makes sure containers are running.

What it does:
  1. Watches API server for Pods assigned to its node
  2. Pulls container images (via containerd / CRI-O)
  3. Creates containers via CRI (Container Runtime Interface)
  4. Sets up Pod networking (via CNI plugin)
  5. Mounts volumes (PVCs, ConfigMaps, Secrets)
  6. Runs health probes (liveness, readiness, startup)
  7. Reports Pod status back to API server
  8. Sends node heartbeats (every 10s via Lease objects)

Key fact: kubelet operates INDEPENDENTLY of the control plane.
  If the API server goes down, kubelet keeps running existing Pods.
  It just can't receive new work or report status.
```
[End Code Block]

kube-proxy

[Code Block]
```
Role: Network proxy on every node. Implements Service networking.

When a Service is created:
  kube-proxy creates rules so that traffic to ServiceIP:Port
  gets load-balanced across the Service's backing Pods.

Three modes:
  iptables (default): writes iptables rules per Service/endpoint
    - Pros: stable, well-tested
    - Cons: slow with 5000+ Services (linear iptables scan)

  IPVS: uses Linux IPVS (IP Virtual Server) kernel module
    - Pros: faster with many Services (hash-based lookup)
    - Cons: needs IPVS kernel modules

  userspace (legacy): proxies in userspace — slow, deprecated

How it works:
  Service: my-api → ClusterIP 10.96.45.12:80
    Backend Pods: 10.244.1.5:8080, 10.244.2.8:8080

  kube-proxy creates iptables rules:
    Traffic to 10.96.45.12:80 →
      50% chance → DNAT to 10.244.1.5:8080
      50% chance → DNAT to 10.244.2.8:8080
```
[End Code Block]

Container Runtime

[Code Block]
```
Role: Actually runs containers. K8s doesn't run containers itself.

K8s talks to the runtime via CRI (Container Runtime Interface).

Common runtimes:
  containerd — most common (used by Docker, EKS, GKE)
  CRI-O — lightweight, designed specifically for K8s (used by OpenShift)

Note: Docker as a runtime was REMOVED in K8s 1.24 (dockershim deprecated).
```

```
  Docker images still work — containerd pulls them.
  You just can't use Docker daemon as the K8s runtime anymore.
[End Code Block]
```

---

3. CORE OBJECTS — WORKLOADS

3.1 POD

```
[Code Block]
The smallest deployable unit. One or more containers sharing:
  - Network namespace (same IP, same localhost)
  - Storage volumes
  - Lifecycle (created and destroyed together)

You almost NEVER create Pods directly.
You create Deployments/StatefulSets/Jobs that create Pods for you.

Multi-container Pod patterns:
  Sidecar:    helper container alongside main (Envoy proxy, log shipper)
  Init:       runs before main container (DB migration, config fetch)
  Ambassador: proxy container for external communication
  Adapter:    transforms output of main container

Pod lifecycle:
  Pending → Running → Succeeded/Failed
                ↓
          CrashLoopBackOff (container keeps crashing)
[End Code Block]
```

3.2 DEPLOYMENT

```
[Code Block]
Manages stateless applications. Most common workload object.

What it gives you:
  - Declarative updates (change image tag → rolling update happens)
  - Rolling updates with zero downtime
  - Rollback to previous versions
  - Scaling (replica count)

Hierarchy:
  Deployment → manages → ReplicaSet(s) → manages → Pod(s)

  Deployment: my-app
    └── ReplicaSet: my-app-7b4f9c (current, 3 Pods)
    └── ReplicaSet: my-app-5a2d8e (previous, 0 Pods, kept for rollback)

Update strategies:
  RollingUpdate (default):
    - maxUnavailable: 25% — at most 25% of Pods can be down
    - maxSurge: 25% — at most 25% extra Pods during rollout

  Recreate:
    - Kill all old Pods first, then create new ones
    - Has downtime — use only when you can't run two versions simultaneously
[End Code Block]
```

3.3 REPLICASET

```
[Code Block]
Ensures N identical Pods are running at all times.

You almost NEVER create ReplicaSets directly.
Deployments manage them for you and add rolling updates + rollback.

The only time to know about ReplicaSets:
  - Debugging: kubectl get rs shows current and old ReplicaSets
  - Understanding rollbacks: rollback = scale old RS up, new RS down
[End Code Block]
```

3.4 STATEFULSET

```
[Code Block]
For stateful applications where Pods need:
  - Stable network identity (db-0, db-1, db-2 — not random names)
  - Stable storage (each Pod gets its own PVC)
  - Ordered deployment and scaling

Key behaviors:
  - Pods created in order: db-0 first, then db-1, then db-2
  - Pods deleted in reverse: db-2 first, then db-1, then db-0
  - Each Pod gets a DNS name: db-0.db-headless.namespace.svc.cluster.local
  - Each Pod gets its own PVC (persists across Pod restarts)

When to use:
  - Databases (PostgreSQL, MySQL, MongoDB)
  - Message brokers (Kafka, RabbitMQ)
  - Distributed systems (ZooKeeper, etcd, Elasticsearch)
  - Anything that needs "I am node 0" identity

Requires: a Headless Service (ClusterIP: None) for DNS to work.
[End Code Block]
```

3.5 DAEMONSET

```
[Code Block]
Runs exactly ONE Pod per node (or per matching node).

When a new node joins: DaemonSet automatically schedules a Pod on it.
When a node is removed: Pod is garbage collected.

Use cases:
  - Log collection: Fluentd, Filebeat on every node
  - Monitoring: Datadog agent, Prometheus node-exporter
  - Networking: Calico, Cilium CNI on every node
  - Storage: CSI node plugins
  - kube-proxy itself runs as a DaemonSet

Targeting specific nodes:
  nodeSelector:
    gpu: "true"
  → DaemonSet only runs on nodes labeled gpu=true
[End Code Block]
```

3.6 JOB & CRONJOB

[Code Block]
```
Job: Run-to-completion workload. Pod runs, finishes, done.
  - Use for: batch processing, DB migrations, one-time tasks
  - completions: how many Pods must succeed
  - parallelism: how many run concurrently
  - backoffLimit: retry count before marking failed

CronJob: Creates Jobs on a schedule.
  - Use for: periodic backups, report generation, cleanup
  - schedule: "0 2 * * *" (cron syntax)
  - concurrencyPolicy: Allow / Forbid / Replace

Java relevance:
  - Spring Batch jobs running as K8s Jobs
  - Flyway/Liquibase DB migration as init container or Job
  - Scheduled report generation as CronJob
```
[End Code Block]

---

4. NETWORKING — HOW TRAFFIC FLOWS

4.1 THE THREE RULES

[Code Block]
```
Rule 1: Every Pod gets its own unique IP address
Rule 2: All Pods can reach all other Pods (no NAT) — even across nodes
Rule 3: Agents on a node can reach all Pods on that node
```
[End Code Block]

4.2 CNI (CONTAINER NETWORK INTERFACE)

[Code Block]
```
CNI plugins implement the actual Pod networking.
K8s delegates all networking to the CNI plugin.

Popular choices:
  Calico:   BGP-based routing, supports Network Policies, high performance
  Cilium:   eBPF-based, advanced security + observability, growing fast
  Flannel:  Simple VXLAN overlay, no Network Policy support
  Weave:    Mesh overlay, simple setup

Same-node Pod-to-Pod:
  Pod A → veth → bridge (cbr0) → veth → Pod B

Cross-node Pod-to-Pod:
  Pod A → veth → bridge → node routing → [overlay or BGP] → Node 2 → bridge → ve
th → Pod B
```
[End Code Block]

4.3 SERVICES

[Code Block]
```
Services provide stable networking for a set of Pods.
Pods come and go (IPs change), Services don't.

Four types:
```

```
┌──────────────────────────────────────────────────────────────┐
| ClusterIP (default)                                          |
|  - Internal virtual IP, only reachable within cluster        |
|  - Use for: service-to-service communication                 |
|  - my-api.default.svc.cluster.local → 10.96.45.12           |
|    → load balances to Pod IPs                                |
|                                                              |
|  90% of your Services will be ClusterIP.                     |
|  Your Java microservice calling another service:             |
|    RestTemplate("http://user-service:8080/api/users")        |
|    K8s DNS resolves "user-service" to ClusterIP              |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
| NodePort                                                     |
|  - Extends ClusterIP                                         |
|  - Opens a port (30000-32767) on EVERY node                  |
|  - External: <any-node-ip>:31080 → ClusterIP → Pods         |
|  - Use for: dev/test external access (not production)        |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
| LoadBalancer                                                 |
|  - Extends NodePort                                          |
|  - Provisions a cloud load balancer (AWS ELB, GCP LB)        |
|  - Gets a public IP                                          |
|  - Use for: production external traffic (one LB per service) |
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
| Headless (ClusterIP: None)                                   |
|  - No virtual IP assigned                                    |
|  - DNS returns individual Pod IPs directly                   |
|  - Use for: StatefulSets, client-side load balancing         |
|  - db-0.db-headless.default.svc.cluster.local → Pod 0's IP  |
└──────────────────────────────────────────────────────────────┘
```

[End Code Block]

4.4 INGRESS

[Code Block]

```
Problem: 10 Services = 10 LoadBalancers = 10 public IPs = expensive
Solution: Ingress - one entry point, route by host/path

Ingress = L7 (HTTP) routing rules
Ingress Controller = the engine that implements them (NGINX, Traefik, ALB)

  Ingress Controller (1 cloud LB)
    → api.myapp.com/users   → user-service:8080
    → api.myapp.com/orders  → order-service:8080
    → web.myapp.com/        → frontend:3000

Features (depending on controller):
  - Path-based routing
  - Host-based routing
  - TLS termination (HTTPS)
  - Rate limiting
  - Rewrites, redirects
  - Auth (basic, OAuth)
```

```
Important: Ingress resource alone does nothing.
You MUST install an Ingress Controller in the cluster.
  - NGINX Ingress Controller (most common)
  - Traefik
  - AWS ALB Ingress Controller
  - Istio Gateway (if using Istio)
[End Code Block]
```

4.5 DNS (COREDNS)

```
[Code Block]
CoreDNS runs as a Deployment in kube-system.
Every Pod gets /etc/resolv.conf pointing to CoreDNS.

Resolution:
  Same namespace:      "my-service"         → my-service.default.svc.cluster.loc
al
  Cross namespace:     "my-service.staging"  → my-service.staging.svc.cluster.lo
cal
  StatefulSet Pod:     "db-0.db-headless"    → specific Pod IP
  External:            "google.com"          → forwarded to upstream DNS

Why this matters for Java:
  In Spring app.properties:
    spring.datasource.url=jdbc:postgresql://postgres-service:5432/mydb

  "postgres-service" resolves via CoreDNS to the Service ClusterIP.
  Works because of the search domain in /etc/resolv.conf.
[End Code Block]
```

4.6 NETWORK POLICIES

```
[Code Block]
Firewall rules at Pod level.
By default: ALL Pods can talk to ALL Pods (open network).
When you apply a NetworkPolicy: denied by default, allow explicitly.

Example: Only frontend can reach backend

  apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: backend-allow-frontend
  spec:
    podSelector:
      matchLabels:
        app: backend
    ingress:
      - from:
          - podSelector:
              matchLabels:
                app: frontend
        ports:
          - port: 8080

  Result:
    frontend → backend:8080  ✅
    anything else → backend  ❌

Requires a CNI that supports it: Calico, Cilium, Weave.
```

Flannel does NOT enforce Network Policies.
[End Code Block]

---

5. STORAGE — PERSISTENCE IN K8S

5.1 THE PROBLEM

[Code Block]
Containers are ephemeral. When a container restarts, its filesystem is lost.
For databases, file uploads, logs — you need persistent storage.
[End Code Block]

5.2 VOLUMES, PV, PVC, STORAGECLASS

[Code Block]
```
Volume Types (in-Pod, tied to Pod lifecycle):
  emptyDir:    temp directory, shared between containers in a Pod
               deleted when Pod dies. Use for: scratch space, sidecar sharing.

  hostPath:    maps a directory from the host node into the Pod
               dangerous in production (ties Pod to specific node).

  configMap:   mounts ConfigMap data as files
  secret:      mounts Secret data as files

Persistent Volumes (PV):
  - Cluster-level storage resource (not namespaced)
  - Backed by real storage: EBS, NFS, GCE PD, Azure Disk
  - Lifecycle independent of Pods

Persistent Volume Claims (PVC):
  - A REQUEST for storage by a Pod/user
  - Namespaced resource
  - Specifies: size, access mode, StorageClass
  - K8s binds PVC to a matching PV

StorageClass:
  - Defines HOW to dynamically provision storage
  - Eliminates manual PV creation

Flow:

  DYNAMIC PROVISIONING (standard):
    1. Admin creates StorageClass:
         provisioner: ebs.csi.aws.com
         parameters:
           type: gp3
    2. Developer creates PVC:
         storageClassName: gp3
         resources.requests.storage: 10Gi
    3. K8s calls the CSI provisioner → creates EBS volume
    4. PV auto-created and bound to PVC
    5. Pod mounts PVC

  STATIC PROVISIONING:
    1. Admin manually creates PV (pointing to existing storage)
    2. Developer creates PVC
```

```
    3. K8s binds PVC to PV

Access Modes:
  ReadWriteOnce (RWO):  mount read-write on ONE node
  ReadOnlyMany  (ROX):  mount read-only on MANY nodes
  ReadWriteMany (RWX):  mount read-write on MANY nodes (NFS, EFS)

Reclaim Policies:
  Retain:  keep PV after PVC deletion (manual cleanup)
  Delete:  delete PV and storage when PVC is deleted
[End Code Block]

5.3 STATEFULSET STORAGE
=======================

[Code Block]
StatefulSets use volumeClaimTemplates — each Pod gets its OWN PVC.

  StatefulSet: postgres (replicas: 3)
    volumeClaimTemplates:
      - name: data
        storage: 50Gi

  Creates:
    Pod postgres-0 → PVC data-postgres-0 → PV (50Gi)
    Pod postgres-1 → PVC data-postgres-1 → PV (50Gi)
    Pod postgres-2 → PVC data-postgres-2 → PV (50Gi)

  postgres-1 crashes → new postgres-1 Pod → re-mounts data-postgres-1
  PVCs survive Pod deletion (data is preserved).
[End Code Block]

--------------------------------------------------------------------------------

6. CONFIGURATION — CONFIGMAPS, SECRETS, ENV
===========================================

6.1 CONFIGMAPS
==============

[Code Block]
Store non-sensitive configuration as key-value pairs.

Create:
  kubectl create configmap app-config \
    --from-literal=DB_HOST=postgres-service \
    --from-literal=LOG_LEVEL=INFO \
    --from-file=application.properties

Use in Pod:

  As env vars:
    env:
      - name: DB_HOST
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: DB_HOST

  As mounted files:
    volumes:
      - name: config
```

```
        configMap:
          name: app-config
    volumeMounts:
      - name: config
        mountPath: /etc/config

  Hot reload: File-mounted ConfigMaps auto-update (~60s).
              Env var ConfigMaps do NOT auto-update — Pod restart required.

Java relevance:
  Mount application.properties or application.yml as a ConfigMap volume:
    mountPath: /config/application.properties

  Spring Boot picks up external config from /config/ by default.
[End Code Block]
```

6.2 SECRETS

```
[Code Block]
Store sensitive data (passwords, tokens, TLS certs).

Important: Secrets are base64 ENCODED, not encrypted (by default).
  echo "mypassword" | base64  → bXlwYXNzd29yZA==
  Anyone with etcd access can read them unless encryption at rest is enabled.

Best practices:
  1. Enable encryption at rest for etcd
  2. Use external secret managers (Vault, AWS Secrets Manager)
     with External Secrets Operator or Vault Agent Injector
  3. Mount as files (not env vars) — env vars leak in logs and process listings
  4. RBAC: restrict who can read Secrets
  5. Audit logging for Secret access

Types:
  Opaque:                     generic key-value (default)
  kubernetes.io/tls:          TLS cert + key
  kubernetes.io/dockerconfigjson:  Docker registry credentials
  kubernetes.io/service-account-token:  (auto-generated SA token)
[End Code Block]
```

---

7. SCHEDULING — WHERE PODS LAND

7.1 NODESELECTOR

```
[Code Block]
Simplest node assignment. Pod spec includes labels that must match a node.

  nodeSelector:
    disktype: ssd

Pod only schedules on nodes labeled disktype=ssd.
No match → Pod stays Pending.
[End Code Block]
```

7.2 NODE AFFINITY

[Code Block]
More expressive than nodeSelector. Supports hard and soft rules.

```
  requiredDuringSchedulingIgnoredDuringExecution:   ← HARD: must match
  preferredDuringSchedulingIgnoredDuringExecution:  ← SOFT: prefer but not requi
red

Example: Must be in us-east-1a or 1b, prefer m5.xlarge:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["us-east-1a", "us-east-1b"]
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 80
          preference:
            matchExpressions:
              - key: node.kubernetes.io/instance-type
                operator: In
                values: ["m5.xlarge"]
```
[End Code Block]

7.3 POD AFFINITY & ANTI-AFFINITY

[Code Block]
Schedule Pods relative to OTHER Pods.

```
Pod Affinity: "schedule me NEAR Pods with label X"
  Use: co-locate web server with cache for low latency

Pod Anti-Affinity: "schedule me AWAY from Pods with label X"
  Use: spread replicas across nodes/zones for HA

Example: Don't put two replicas of my-api on the same node
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["my-api"]
          topologyKey: kubernetes.io/hostname
```
[End Code Block]

7.4 TAINTS & TOLERATIONS

[Code Block]
Taints on NODES: repel Pods
Tolerations on PODS: allow scheduling on tainted nodes

```
Taint a node:
  kubectl taint nodes gpu-node-1 gpu=true:NoSchedule

Only Pods with this toleration can schedule there:
  tolerations:
    - key: "gpu"
```

```
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

Effects:
  NoSchedule:        don't schedule new Pods (existing stay)
  PreferNoSchedule:  try to avoid, but allow if needed
  NoExecute:         evict existing Pods that don't tolerate

Use cases:
  - Dedicated GPU node pools
  - Master/control-plane nodes (system taint prevents user Pods)
  - Draining nodes for maintenance
[End Code Block]
```

7.5 TOPOLOGY SPREAD CONSTRAINTS

```
[Code Block]
Spread Pods evenly across zones, nodes, or any topology.

  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: my-api

  With 3 replicas across 3 zones:
    Zone A: 1 Pod, Zone B: 1 Pod, Zone C: 1 Pod

  Better than podAntiAffinity for multi-zone HA.
[End Code Block]
```

---

8. SCALING — HPA, VPA, CLUSTER AUTOSCALER

8.1 HORIZONTAL POD AUTOSCALER (HPA)

```
[Code Block]
Automatically adjusts Pod replica count based on metrics.

How it works:
  1. Metrics Server collects CPU/memory from kubelet (every 15s)
  2. HPA controller reads metrics every 15s
  3. Calculates: desiredReplicas = current × (currentMetric / targetMetric)
  4. Scales Deployment up or down

  Example: target CPU 50%, current CPU 80%, current 3 replicas
    desired = 3 × (80/50) = 4.8 → 5 replicas

Spec:
  apiVersion: autoscaling/v2
  kind: HorizontalPodAutoscaler
  spec:
    scaleTargetRef:
      kind: Deployment
      name: my-api
```

```
    minReplicas: 2
    maxReplicas: 20
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 50

Custom metrics (advanced):
  - Requests per second (via Prometheus Adapter)
  - Queue depth
  - Any custom metric exposed through the Metrics API

Cooldown:
  Scale-up: immediate (default)
  Scale-down: 5 minute stabilization window (prevents flapping)

Prerequisite: Pods MUST have resource requests set.
HPA uses requests as the baseline for % calculation.
[End Code Block]
```

8.2 VERTICAL POD AUTOSCALER (VPA)

```
[Code Block]
Adjusts CPU/memory REQUESTS (not replicas) based on usage.

Modes:
  Off:        only recommends (no changes)
  Initial:    sets resources on Pod creation only
  Auto:       evicts and recreates Pods with new resources

Use case: You don't know the right resource requests.
  VPA observes, recommends, and applies.

Warning: VPA and HPA on the SAME metric (CPU) will conflict.
  Use HPA for scaling replicas + VPA for memory recommendations.
[End Code Block]
```

8.3 CLUSTER AUTOSCALER

```
[Code Block]
Scales the NUMBER OF NODES (not Pods).

Works with HPA:
  Load ↑ → HPA creates more Pods → not enough node capacity →
  Pods stuck Pending → Cluster Autoscaler adds nodes

Scale-up:
  Phase 1: Detect unschedulable (Pending) Pods
  Phase 2: Simulate which node group can fit them
  Phase 3: Call cloud API → launch new node(s)
  Phase 4: Node boots, joins cluster (~1-3 min)
  Phase 5: Pending Pods scheduled on new node

Scale-down:
  Phase 1: Find underutilized nodes (<50% capacity for 10 min)
  Phase 2: Check PDBs, local storage, unevictable Pods
  Phase 3: Drain node (respecting PDBs and graceful termination)
```

```
  Phase 4: Terminate node

Key: Scale-up takes 1-3 minutes. For latency-sensitive workloads,
over-provision slightly (keep warm spare capacity).
[End Code Block]
```

---

9. SECURITY — RBAC, POD SECURITY, SECRETS

9.1 RBAC (ROLE-BASED ACCESS CONTROL)

```
[Code Block]
Controls WHO can do WHAT on WHICH resources.

4 objects:
  Role              → permissions in a NAMESPACE
  ClusterRole       → permissions CLUSTER-WIDE
  RoleBinding       → grants Role to a subject in a NAMESPACE
  ClusterRoleBinding → grants ClusterRole CLUSTER-WIDE

Subjects:
  User, Group, ServiceAccount

Example: CI/CD bot can manage Deployments in staging:

  Role (staging namespace):
    rules:
      - apiGroups: ["apps"]
        resources: ["deployments"]
        verbs: ["get", "list", "create", "update", "patch", "delete"]

  RoleBinding:
    subjects:
      - kind: ServiceAccount, name: ci-bot, namespace: staging
    roleRef:
      kind: Role, name: deployment-manager

Test: kubectl auth can-i create deployments \
        --as=system:serviceaccount:staging:ci-bot -n staging
[End Code Block]
```

9.2 SERVICEACCOUNTS

```
[Code Block]
Identity for Pods (and CI/CD bots, controllers).

Every namespace has a "default" ServiceAccount.
Pods get the default SA unless you specify one.

Best practice: Create dedicated SAs with least privilege.
  Don't let every Pod use the default SA.

  spec:
    serviceAccountName: order-service-sa
    automountServiceAccountToken: false  ← if Pod doesn't need API access
[End Code Block]
```

9.3 POD SECURITY STANDARDS

==========

[Code Block]
```
Three levels (replaced PodSecurityPolicy in K8s 1.25):

  Privileged:  unrestricted (for system Pods)
  Baseline:    prevents obvious escalations (no hostNetwork, no privileged conta
iners)
  Restricted:  hardened (must run non-root, drop caps, read-only fs)

Enforce at namespace level:
  kubectl label namespace production \
    pod-security.kubernetes.io/enforce=restricted

Pod-level security context:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    readOnlyRootFilesystem: true
    allowPrivilegeEscalation: false
    capabilities:
      drop: ["ALL"]
```
[End Code Block]

---

10. OBSERVABILITY — PROBES, LOGS, MONITORING

10.1 HEALTH PROBES

[Code Block]

Probe	Purpose	On failure
Liveness	Is container alive?	RESTART container
Readiness	Can it accept traffic?	Remove from Service
Startup	Has it finished booting?	RESTART (after timeout)

```
Probe types:
  httpGet:   GET /healthz → expect 200-399
  tcpSocket: connect to port → expect success
  exec:      run command → expect exit code 0

Java / Spring Boot integration:
  Spring Actuator provides:
    /actuator/health/liveness   → liveness probe
    /actuator/health/readiness  → readiness probe

  In deployment.yaml:
    livenessProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20

    readinessProbe:
      httpGet:
        path: /actuator/health/readiness
```

```
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 10

    startupProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
      failureThreshold: 30
      periodSeconds: 2
      # gives Spring Boot up to 60s to start
```
[End Code Block]

10.2 LOGGING

[Code Block]
```
K8s does NOT provide built-in log aggregation. You need a stack:

Common: EFK (Elasticsearch + Fluentd + Kibana)
  - Fluentd DaemonSet on every node → collects container logs
  - Ships to Elasticsearch
  - Kibana for visualization

Or: ELK, Loki + Grafana, Datadog, CloudWatch

Container logs:
  Written to stdout/stderr → captured by container runtime →
  stored at /var/log/containers/ on the node →
  collected by log shipper (Fluentd/Filebeat)

Java relevance:
  Configure your Java app to log to STDOUT (not files).
  SLF4J + Logback with ConsoleAppender.
  K8s captures stdout automatically.

  In logback-spring.xml:
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
      <encoder>
        <pattern>%d{ISO8601} [%thread] %-5level %logger - %msg%n</pattern>
      </encoder>
    </appender>
```
[End Code Block]

10.3 MONITORING

[Code Block]
```
Common stack: Prometheus + Grafana

Prometheus:
  - Scrapes /metrics endpoints from Pods
  - Time-series database
  - Alert rules (Alertmanager)

Grafana:
  - Dashboards, visualization

Metrics Server:
  - Lightweight, in-cluster resource metrics (CPU/memory)
  - Required for HPA and kubectl top
  - NOT for long-term monitoring (no history)
```

```
Java relevance:
  Micrometer (Spring Boot Actuator) exposes Prometheus metrics:
    /actuator/prometheus

  In pom.xml:
    micrometer-registry-prometheus

  Prometheus scrapes this endpoint automatically if annotations are set:
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "8080"
      prometheus.io/path: "/actuator/prometheus"
[End Code Block]
```

---

11. DEPLOYMENT STRATEGIES — ZERO DOWNTIME

11.1 ROLLING UPDATE (BUILT-IN)

```
[Code Block]
Default strategy. K8s creates new Pods before killing old ones.

Requirements for zero downtime:
  1. maxUnavailable: 0 (don't kill old before new is ready)
  2. Readiness probe (K8s waits for new Pod to be ready)
  3. Graceful shutdown (handle SIGTERM, drain connections)
  4. preStop hook: sleep 5 (handles kube-proxy iptables race)
  5. PodDisruptionBudget (protects during node drains)

  Flow:
    v1-a ✅, v1-b ✅, v1-c ✅
    v2-d created → passes readiness → v1-a terminated
    v2-e created → passes readiness → v1-b terminated
    v2-f created → passes readiness → v1-c terminated
    v2-d ✅, v2-e ✅, v2-f ✅
[End Code Block]
```

11.2 BLUE-GREEN

```
[Code Block]
Not built into K8s natively. Implemented via Service label switching.

  Step 1: "blue" Deployment running (v1), Service points to blue
    Service selector: version=blue

  Step 2: Deploy "green" Deployment (v2), same label app=my-api, version=green
    Green Pods are running but receive NO traffic

  Step 3: Test green independently (internal endpoint, smoke tests)

  Step 4: Switch Service selector: version=green
    All traffic instantly switches to green

  Step 5: If issues → switch back to version=blue (instant rollback)

  Pros: instant cutover, instant rollback
  Cons: double the resources during transition
```

[End Code Block]

11.3 CANARY

[Code Block]
Route small % of traffic to new version, verify, then roll out.

```
With Ingress annotation (NGINX):
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-weight: "10"

  10% of traffic → v2 canary Pods
  90% of traffic → v1 stable Pods

  Monitor error rates, latency for canary.
  If good → increase to 50% → 100%.
  If bad → remove canary annotation (instant rollback).

Advanced: Istio VirtualService for traffic splitting.
  Or Argo Rollouts for automated canary with metric analysis.
```
[End Code Block]

---

12. HELM — PACKAGE MANAGEMENT

[Code Block]
Helm = package manager for Kubernetes (like apt/npm for K8s manifests).

```
Concepts:
  Chart:    a package of K8s manifests (templates + values)
  Release:  an installed instance of a Chart
  Values:   configuration that customizes the Chart
  Repository: a collection of Charts (like Docker Hub for Helm)

Common operations:
  helm repo add bitnami https://charts.bitnami.com/bitnami
  helm install my-postgres bitnami/postgresql --values values.yaml
  helm upgrade my-postgres bitnami/postgresql --values values.yaml
  helm rollback my-postgres 1
  helm uninstall my-postgres

Chart structure:
  mychart/
    Chart.yaml          ← metadata (name, version)
    values.yaml         ← default configuration
    templates/
      deployment.yaml   ← Go template: {{ .Values.replicaCount }}
      service.yaml
      ingress.yaml
      _helpers.tpl      ← template helpers

Why Helm matters for Java devs:
  - Package your microservice as a Helm chart
  - Same chart, different values per environment (dev/staging/prod)
  - CI/CD pipeline: helm upgrade --install in each stage
  - Install dependencies (PostgreSQL, Redis, Kafka) from public charts
```
[End Code Block]

---

13. JAVA ON KUBERNETES — JVM-SPECIFIC CONCERNS

This section is critical. The JVM has behaviors that interact poorly with K8s defaults.

13.1 MEMORY: CONTAINER LIMITS VS JVM HEAP

[Code Block]
```
THE PROBLEM:
  Container limit: 512Mi
  JVM default heap: tries to use 25% of PHYSICAL machine memory
  Machine has 64GB RAM → JVM wants 16GB heap → OOMKilled instantly

THE FIX (Java 10+):
  JVM flag: -XX:+UseContainerSupport (enabled by default since Java 10)
  JVM sees container limits, not host memory.

  With 512Mi container limit:
    JVM sees 512Mi as total memory
    Default MaxRAMPercentage = 25% → ~128Mi heap
    You probably want more → set explicitly:

  JAVA_OPTS: "-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"

  This gives ~384Mi heap out of 512Mi container limit.
  Remaining ~128Mi for: metaspace, thread stacks, native memory, off-heap.

  Rule of thumb for container memory limit:
    container_limit = max_heap + metaspace(~256Mi) + thread_stacks(threads × 1Mi
) + buffer

MEMORY BUDGET EXAMPLE:
  Container limit: 1Gi (1024Mi)
    Max heap:        700Mi  (-XX:MaxRAMPercentage=68.0)
    Metaspace:       ~150Mi (-XX:MaxMetaspaceSize=150m)
    Thread stacks:   ~100Mi (100 threads × 1Mi)
    Native/buffers:  ~74Mi
    Total:           1024Mi ✅
```
[End Code Block]

13.2 CPU: CONTAINER LIMITS VS JVM THREADS

[Code Block]
```
THE PROBLEM:
  CPU limit: 1000m (1 core)
  JVM's Runtime.availableProcessors() might return the HOST's core count
  (fixed in modern Java, but older apps or libs may misbehave)

  ForkJoinPool.commonPool() sizes to availableProcessors()
  If it sees 32 cores but only has 1 → thread oversubscription → throttling

THE FIX:
  Java 11+: -XX:+UseContainerSupport (default) → sees container CPU limit
  Java 8u191+: same flag available

  Verify in container:
    Runtime.getRuntime().availableProcessors()  → should return ~1 for 1000m
```

```
  For 1000m (1 core): JVM sees 1 processor
  For 2000m (2 cores): JVM sees 2 processors
  For 500m (0.5 cores): JVM sees 1 processor (minimum 1)

CPU REQUESTS vs LIMITS:
  - Requests: guaranteed CPU time (scheduler uses this)
  - Limits: maximum — container is THROTTLED (not killed) if exceeded
  - Many teams set NO CPU limit (only requests) to avoid throttling
    → Pod can burst when needed, requests guarantee minimum
```
[End Code Block]

13.3 STARTUP TIME

[Code Block]

```
Java / Spring Boot apps are SLOW to start (10-60 seconds).

Problems without proper config:
  - Liveness probe fires before app is ready → restarts → CrashLoopBackOff
  - Readiness probe fails → traffic never routes → 503 errors
  - HPA scales up but new Pods take 30s → not helpful for spikes

Solutions:

  1. Startup Probe:
    startupProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
      failureThreshold: 30
      periodSeconds: 2
    # Gives up to 60s (30 × 2s) before liveness takes over

  2. Reduce startup time:
    - Use Spring Boot lazy initialization: spring.main.lazy-initialization=true
    - Use CDS (Class Data Sharing) for faster class loading
    - Use GraalVM native images (Quarkus/Micronaut) for <1s startup
    - Pre-compile: -XX:+TieredCompilation -XX:TieredStopAtLevel=1

  3. Graceful shutdown in Spring Boot:
    server.shutdown=graceful
    spring.lifecycle.timeout-per-shutdown-phase=30s

    K8s sends SIGTERM → Spring stops accepting new requests →
    finishes in-flight requests → exits cleanly within 30s
```
[End Code Block]

13.4 DOCKERFILE BEST PRACTICES FOR JAVA

[Code Block (dockerfile)]

```
# Multi-stage build — small final image
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /app
COPY . .
RUN ./mvnw package -DskipTests

FROM eclipse-temurin:21-jre          # JRE only, not JDK
WORKDIR /app
COPY --from=builder /app/target/myapp.jar app.jar

# Non-root user (K8s security context)
```

```
RUN addgroup --system appgroup && adduser --system appuser --ingroup appgroup
USER appuser

EXPOSE 8080

ENTRYPOINT ["java", \
  "-XX:MaxRAMPercentage=75.0", \
  "-XX:+UseContainerSupport", \
  "-jar", "app.jar"]
[End Code Block]

[Code Block]
Key points:
  - Multi-stage: build with JDK, run with JRE (smaller image)
  - eclipse-temurin: trusted OpenJDK distribution
  - Non-root user: matches K8s runAsNonRoot
  - Container-aware JVM flags baked in
  - No :latest tag — pin version
  - .dockerignore: exclude .git, target, node_modules
[End Code Block]

13.5 SPRING BOOT + K8S INTEGRATION
==================================

[Code Block]
Spring Cloud Kubernetes:
  - Auto-discovers config from ConfigMaps and Secrets
  - Service discovery via K8s Services (no Eureka needed in K8s)
  - Config reload: watch ConfigMap changes, refresh beans

Dependencies:
  spring-cloud-starter-kubernetes-client-config
  spring-cloud-starter-kubernetes-client-all

Actuator endpoints for K8s probes:
  management.endpoint.health.probes.enabled=true
  management.health.livenessState.enabled=true
  management.health.readinessState.enabled=true

  → /actuator/health/liveness
  → /actuator/health/readiness

Profile activation:
  In ConfigMap or env:
    SPRING_PROFILES_ACTIVE=kubernetes,production
[End Code Block]

--------------------------------------------------------------------------------

14. NAMESPACES, RESOURCE QUOTAS & LIMIT RANGES
==============================================

14.1 NAMESPACES
===============

[Code Block]
Virtual clusters within a physical cluster. Isolation boundary.

Default namespaces:
  default:      where resources go if you don't specify
  kube-system:  control plane components (CoreDNS, kube-proxy, etc.)
  kube-public:  readable by all (rarely used)
```

```
  kube-node-lease: node heartbeat leases

Use cases:
  Per-environment: dev, staging, production
  Per-team: team-alpha, team-beta
  Per-application: payment-system, user-system

Things that are namespaced: Pods, Services, Deployments, ConfigMaps, Secrets, PV
Cs
Things that are NOT: Nodes, PVs, StorageClasses, ClusterRoles, Namespaces
[End Code Block]
```

14.2 RESOURCE QUOTAS

```
[Code Block]
Limit total resources a namespace can consume.

apiVersion: v1
kind: ResourceQuota
metadata:
  name: staging-quota
  namespace: staging
spec:
  hard:
    requests.cpu: "10"          # total CPU requests across all Pods
    requests.memory: 20Gi       # total memory requests
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"                  # max 50 Pods in this namespace
    services.loadbalancers: "2" # max 2 LoadBalancer Services

If a new Pod would exceed the quota → rejected by API server.
[End Code Block]
```

14.3 LIMIT RANGES

```
[Code Block]
Default and enforce limits for individual Pods/containers in a namespace.

apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: staging
spec:
  limits:
    - type: Container
      default:           # applied if Pod doesn't set limits
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:    # applied if Pod doesn't set requests
        cpu: "100m"
        memory: "128Mi"
      max:               # hard cap per container
        cpu: "2"
        memory: "2Gi"
      min:               # minimum per container
        cpu: "50m"
        memory: "64Mi"
[End Code Block]
```

---

15. CRDS & OPERATORS

[Code Block]
```
CRD (Custom Resource Definition):
  Extend the K8s API with your own resource types.
  After defining a CRD, you can: kubectl get mydatabases

Operator:
  A controller that watches CRDs and takes action.
  Encodes human operational knowledge into code.

  Example: PostgreSQL Operator
    - You create: kind: PostgresCluster, replicas: 3
    - Operator automatically:
        Creates StatefulSet, Services, PVCs
        Configures replication
        Sets up automated backups
        Handles failover
        Manages upgrades

Popular operators:
  Prometheus Operator     → monitoring
  Strimzi                 → Kafka on K8s
  Cert-Manager            → TLS certificate management
  Zalando Postgres        → PostgreSQL HA
  Elastic Cloud on K8s    → Elasticsearch

CRD + Operator = the K8s-native way to manage complex stateful systems.
```
[End Code Block]

---

16. SERVICE MESH (ISTIO BASICS)

[Code Block]
```
A service mesh adds infrastructure-level networking features
without changing application code.

What Istio adds:
  - mTLS between all services (encryption in transit)
  - Traffic management (canary, blue-green, fault injection)
  - Observability (distributed tracing, metrics, access logs)
  - Circuit breaking, retries, timeouts
  - Rate limiting

How it works:
  Istio injects an Envoy sidecar proxy into every Pod (via mutating webhook).

  Pod: [my-java-app] ←→ [envoy-proxy] → network → [envoy-proxy] ←→ [target-app]

  All traffic goes through Envoy. Istio control plane configures all Envoys.

Key objects:
  VirtualService: traffic routing rules (canary weights, header-based routing)
  DestinationRule: circuit breakers, connection pool settings
  Gateway: ingress traffic configuration
  PeerAuthentication: mTLS policies
```

```
When to use:
  - 20+ microservices needing consistent security/observability
  - Zero-trust networking requirements
  - Advanced traffic management (gradual rollouts)

When NOT to use:
  - Small clusters (< 10 services) — overhead not worth it
  - Performance-critical with tight latency budgets (Envoy adds ~1-3ms per hop)
```
[End Code Block]

---

17. CI/CD WITH KUBERNETES

[Code Block]

```
Typical pipeline for a Java microservice:

  1. Developer pushes to Git
  2. CI (Jenkins/GitHub Actions/GitLab CI):
     a. Compile: mvn clean package
     b. Test: mvn test + integration tests
     c. Build image: docker build -t myregistry/my-app:v1.2.3
     d. Push image: docker push myregistry/my-app:v1.2.3
     e. Update K8s manifest or Helm values with new image tag

  3. CD (ArgoCD / Flux / Helm):
     a. Detect new image tag in Git (GitOps)
     b. helm upgrade --install my-app ./chart --set image.tag=v1.2.3
     c. K8s rolling update begins
     d. Monitor rollout: kubectl rollout status deployment/my-app
     e. If fails → automatic rollback (or manual: helm rollback)

GitOps approach (ArgoCD):
  - K8s manifests stored in Git (source of truth)
  - ArgoCD watches Git repo
  - Any change in Git → ArgoCD syncs to cluster
  - Drift detection: if someone manually changes cluster, ArgoCD reverts

Image tag strategy:
  - Never use :latest in production
  - Use Git SHA: myapp:abc123f
  - Or semver: myapp:v1.2.3
  - Enables deterministic rollbacks
```
[End Code Block]

---

18. TROUBLESHOOTING PLAYBOOK

POD STUCK IN PENDING

[Code Block]

```
kubectl describe pod <name>  → check Events

Common causes:
  "Insufficient cpu/memory"          → cluster full, add nodes
  "Didn't match node selector"       → wrong labels on nodes
  "Taint not tolerated"              → add toleration or remove taint
```

```
  "PVC not bound"                    → create PVC, check StorageClass
  "ResourceQuota exceeded"           → quota limit hit in namespace
[End Code Block]
```

POD IN CRASHLOOPBACKOFF

```
[Code Block]
kubectl logs <pod> --previous       → logs from crashed container
kubectl describe pod <pod>          → exit code, events

Common causes:
  Exit 1:    app error (check logs, config, dependencies)
  Exit 137:  OOMKilled (increase memory limit)
  Exit 139:  Segfault
  Liveness probe killing healthy but slow container → tune probe timing
[End Code Block]
```

POD IN IMAGEPULLBACKOFF

```
[Code Block]
kubectl describe pod <pod>  → look for image pull error

Common causes:
  Wrong image name/tag
  Private registry — missing imagePullSecrets
  Rate limiting (Docker Hub anonymous pull limit)
[End Code Block]
```

SERVICE NOT REACHABLE

```
[Code Block]
Step 1: kubectl get endpoints <service>
  → Are there Pod IPs listed? If empty → selector doesn't match Pod labels

Step 2: kubectl exec -it <pod> -- curl <service>:<port>
  → Test from inside the cluster

Step 3: Check readiness probes
  → If Pods aren't ready, they're removed from endpoints

Step 4: Check Network Policies
  → Might be blocking traffic
[End Code Block]
```

NODE NOTREADY

```
[Code Block]
kubectl describe node <node>  → check Conditions and Events

Common causes:
  kubelet not running            → SSH into node, systemctl status kubelet
  Disk pressure                  → node storage full
  Memory pressure                → node OOM
  Network partition              → node can't reach API server
  Certificate expired            → kubelet certs need renewal
[End Code Block]
```

---

19. ESSENTIAL COMMANDS CHEAT SHEET

[Code Block (bash)]

```bash
# ──────── CLUSTER INFO ────────
kubectl cluster-info
kubectl get nodes -o wide
kubectl version --short
kubectl api-resources                       # list all resource types

# ──────── NAMESPACE ────────
kubectl get namespaces
kubectl config set-context --current --namespace=staging  # switch default ns
kubectl get all -n kube-system

# ──────── WORKLOADS ────────
kubectl get pods -o wide                    # see node placement and IPs
kubectl get pods --all-namespaces           # all namespaces
kubectl get deployments,replicasets,pods
kubectl describe pod <pod>                  # detailed info + events
kubectl logs <pod>                          # container logs
kubectl logs <pod> --previous               # previous (crashed) container logs
kubectl logs <pod> -c <container>           # specific container in multi-contain
er Pod
kubectl exec -it <pod> -- /bin/sh           # shell into container
kubectl port-forward <pod> 8080:8080        # access Pod locally

# ──────── DEPLOYMENTS ────────
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=2
kubectl set image deployment/<name> app=image:tag
kubectl scale deployment/<name> --replicas=5

# ──────── DEBUGGING ────────
kubectl get events --sort-by='.lastTimestamp'
kubectl describe nodes | grep -A 10 "Allocated resources"
kubectl top nodes
kubectl top pods
kubectl debug -it <pod> --image=busybox    # ephemeral debug container
kubectl run debug --image=busybox -it --rm -- sh  # temp debug Pod

# ──────── SERVICES & NETWORKING ────────
kubectl get svc
kubectl get endpoints <service>
kubectl get ingress
kubectl get networkpolicies

# ──────── CONFIG & SECRETS ────────
kubectl get configmaps
kubectl get secrets
kubectl create configmap <name> --from-file=app.properties
kubectl create secret generic <name> --from-literal=password=abc123

# ──────── STORAGE ────────
kubectl get pv
kubectl get pvc
kubectl get storageclass

# ──────── RBAC ────────
```

```
kubectl auth can-i create deployments --as=<user>
kubectl auth can-i '*' '*'                 # am I cluster admin?
kubectl get roles,rolebindings -n <ns>
kubectl get clusterroles,clusterrolebindings

# ──────── NODE MANAGEMENT ────────
kubectl cordon <node>                      # mark unschedulable
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>                    # mark schedulable
kubectl taint nodes <node> key=value:NoSchedule
kubectl taint nodes <node> key=value:NoSchedule-  # remove taint

# ──────── SCALING ────────
kubectl autoscale deployment/<name> --min=2 --max=10 --cpu-percent=60
kubectl get hpa

# ──────── HELM ────────
helm repo add <name> <url>
helm search repo <keyword>
helm install <release> <chart> --values values.yaml
helm upgrade <release> <chart> --values values.yaml
helm rollback <release> <revision>
helm list
helm uninstall <release>
```
[End Code Block]

QUICK REFERENCE: OBJECT HIERARCHY
==========

[Code Block]

```
Cluster
  ├── Nodes (worker machines)
  ├── Namespaces
  |     ├── Deployments → ReplicaSets → Pods → Containers
  |     ├── StatefulSets → Pods (ordered, with PVCs)
  |     ├── DaemonSets → Pods (one per node)
  |     ├── Jobs / CronJobs → Pods (run to completion)
  |     ├── Services (ClusterIP / NodePort / LoadBalancer / Headless)
  |     ├── Ingresses (L7 routing)
  |     ├── ConfigMaps, Secrets
  |     ├── PVCs → bound to PVs
  |     ├── ServiceAccounts, Roles, RoleBindings
  |     ├── NetworkPolicies
  |     ├── HPA, VPA
  |     └── PodDisruptionBudgets
  ├── PersistentVolumes (cluster-scoped)
  ├── StorageClasses (cluster-scoped)
  ├── ClusterRoles, ClusterRoleBindings (cluster-scoped)
  └── CRDs (cluster-scoped)
```
[End Code Block]

==========
FILE: K8s-Interview-30.md
==========

KUBERNETES — 30 INTERVIEW QUESTIONS (SENIOR / STAFF ENGINEER LEVEL)
==========

> Calibrated for 7+ YOE. Covers architecture, networking, storage, security, sch

eduling, scaling, debugging, and real-world scenarios. Pure technical — no company-specific fluff.

CATEGORY BREAKDOWN
==========

#	Category	Questions
1–5	Architecture & Core Internals	Control plane, etcd, API server, kubelet
6–10	Workloads, Scheduling & Lifecycle	Pods, Deployments, StatefulSets, DaemonSets, scheduling
11–15	Networking	Services, Ingress, DNS, CNI, Network Policies
16–18	Storage	PV/PVC, StorageClasses, StatefulSet volumes
19–22	Security	RBAC, ServiceAccounts, Pod Security, Secrets
23–25	Scaling & Performance	HPA, VPA, Cluster Autoscaler, resource management
26–28	Observability & Debugging	Logs, probes, debugging CrashLoopBackOff
29–30	Real-World Scenarios	Production troubleshooting, zero-downtime deployments

---

Q1: WALK ME THROUGH WHAT HAPPENS WHEN YOU RUN KUBECTL APPLY -F DEPLOYMENT.YAML — FROM THE MOMENT YOU HIT ENTER TO THE CONTAINER RUNNING.
==========

Answer:

[Code Block]
```
Step 1: kubectl CLIENT
  - kubectl reads deployment.yaml, validates against the OpenAPI schema
  - Authenticates with the API server (using kubeconfig: certificate/token)
  - Sends an HTTP POST/PATCH to the API server:
      POST https://<api-server>:6443/apis/apps/v1/namespaces/default/deployments

Step 2: API SERVER (kube-apiserver)
  - Receives the request
  - Authentication: verifies client identity (certificate, token, OIDC)
  - Authorization: checks RBAC — does this user/SA have permission to create Deployments?
  - Admission Controllers: runs mutating webhooks (e.g., inject sidecar, add labels)
    then validating webhooks (e.g., enforce policies like "no :latest tag")
  - Persists the Deployment object to etcd
  - Returns 201 Created to kubectl

Step 3: CONTROLLER MANAGER (kube-controller-manager)
  - Deployment Controller watches for new/changed Deployments
  - Sees new Deployment → creates a ReplicaSet object
  - ReplicaSet Controller watches for new ReplicaSets
  - Sees new ReplicaSet → creates Pod objects (desired replicas)
  - Pods are created with status: Pending (no node assigned yet)

Step 4: SCHEDULER (kube-scheduler)
  - Watches for Pods with no nodeName (Pending)
  - Runs filtering: eliminates nodes that can't run the Pod
    (insufficient CPU/memory, taints, affinity rules, etc.)
  - Runs scoring: ranks remaining nodes
```

```
    (spread, resource balance, affinity preferences)
  - Binds the Pod to the highest-scoring node:
    writes nodeName to the Pod object in etcd

Step 5: KUBELET (on the chosen node)
  - Watches API server for Pods assigned to its node
  - Sees new Pod → pulls container image (via container runtime: containerd/CRI-
O)
  - Creates the container(s) via CRI (Container Runtime Interface)
  - Sets up networking via CNI plugin (assigns Pod IP, sets up veth pairs)
  - Sets up volumes (mounts PVCs, ConfigMaps, Secrets)
  - Starts the container
  - Reports Pod status back to API server: Running

Step 6: KUBE-PROXY (on every node)
  - If a Service exists for this Deployment, kube-proxy updates
    iptables/IPVS rules so traffic to the Service ClusterIP
    gets forwarded to the new Pod's IP
```
[End Code Block]

What to highlight: The entire system is declarative and watch-based. No componen
t calls another directly. They all watch etcd (via the API server) and react to
state changes. This is the reconciliation loop pattern.

Q2: WHAT IS ETCD'S ROLE IN KUBERNETES, AND WHAT HAPPENS IF ETCD GOES DOWN?

Answer:

Role: etcd is the single source of truth for the entire cluster. Every object (P
ods, Deployments, Services, ConfigMaps, Secrets, RBAC rules) is stored in etcd a
s key-value pairs. The API server is the only component that talks to etcd direc
tly.

What's stored:

[Code Block]
```
/registry/pods/default/my-pod-abc123          → Pod spec + status
/registry/deployments/default/my-app          → Deployment spec
/registry/services/default/my-service         → Service definition
/registry/secrets/default/db-credentials      → Encrypted secret data
/registry/leases/kube-system/kube-scheduler   → Leader election lease
```
[End Code Block]

If etcd goes down:

[Code Block]
```
Phase 1: API SERVER becomes read-only (from its cache) then fails
  - New creates/updates/deletes fail immediately
  - Existing running Pods continue running (kubelet doesn't need etcd)

Phase 2: CONTROLLERS stop reconciling
  - No new ReplicaSets, no scaling, no rescheduling
  - If a node dies during etcd outage, Pods aren't rescheduled

Phase 3: SCHEDULER stops scheduling
  - New Pods stay in Pending forever

Phase 4: EXISTING WORKLOADS keep running
  - Containers already on nodes continue (kubelet runs independently)
```

  - But no new changes, no healing, no scaling

Recovery:
  - Restore etcd from snapshot: etcdctl snapshot restore
  - Or bring etcd quorum back (if multi-node etcd cluster with 3/5 nodes)
[End Code Block]

Production setup: Always run etcd as a 3 or 5 node cluster (Raft consensus needs majority quorum: 2/3 or 3/5). Take automated snapshots. Separate etcd from worker nodes (dedicated hardware or VMs with fast SSDs — etcd is latency-sensitive).

Q3: EXPLAIN THE DIFFERENCE BETWEEN A DEPLOYMENT, STATEFULSET, AND DAEMONSET. WHEN DO YOU PICK EACH?
==========

Answer:

	Deployment	StatefulSet	DaemonSet
Identity	Pods are interchangeable (random names like app-7b4f9c)	Pods have stable, ordered identity (db-0, db-1, db-2)	One Pod per node
Scaling	Scale up/down freely, any order	Scale up sequentially (0→1→2), scale down reverse (2→1→0)	Automatically one per node (or per matching node)
Storage	Shared PVC or no persistent storage	Each Pod gets its own PVC (volumeClaimTemplates) — PVC survives Pod deletion	Usually hostPath or no storage
Network	No stable hostname per Pod	Stable DNS: db-0.db-headless.namespace.svc.cluster.local	N/A
Use case	Stateless apps (API servers, web frontends)	Stateful apps (databases, Kafka, ZooKeeper, Redis cluster)	Node-level agents (log collectors, monitoring, CNI, kube-proxy)

When to pick:
- Deployment: Default choice. Use for anything that doesn't need stable identity or per-Pod storage.
- StatefulSet: When Pods need stable network identity AND/OR their own persistent volume. Think databases, message brokers.
- DaemonSet: When you need exactly one Pod per node (or per qualifying node). Think Fluentd, Datadog agent, kube-proxy.

Q4: WHAT ARE ADMISSION CONTROLLERS? EXPLAIN MUTATING VS VALIDATING WEBHOOKS WITH AN EXAMPLE.
==========

Answer:

Admission controllers intercept requests to the API server after authentication and authorization but before persistence to etcd. They can modify or reject the request.

[Code Block]
Request flow:
  kubectl apply → API Server → AuthN → AuthZ → Mutating Admission → Validating Admission → etcd

Two types:

1. MUTATING ADMISSION WEBHOOK

```
  - Can MODIFY the object before it's saved
  - Runs FIRST
  - Example: Istio sidecar injector
    → You submit a Pod spec with 1 container
    → Mutating webhook adds the Envoy sidecar container automatically
    → Pod saved to etcd has 2 containers

    Before mutation:
      containers:
        - name: my-app
          image: my-app:v1

    After mutation (by Istio webhook):
      containers:
        - name: my-app
          image: my-app:v1
        - name: istio-proxy         ← injected
          image: istio/proxyv2:1.20 ← injected

2. VALIDATING ADMISSION WEBHOOK
  - Can only ACCEPT or REJECT — cannot modify
  - Runs AFTER mutating webhooks
  - Example: Policy enforcement (OPA Gatekeeper / Kyverno)
    → Reject Pods that use :latest tag
    → Reject Pods without resource limits
    → Reject Pods that run as root

    Policy: "All Pods must have CPU/memory limits"
    Pod spec has no limits → webhook returns: DENIED
    "Pod my-app rejected: missing resource limits"
```
[End Code Block]

Order: Mutating runs first (so policies validate the final, mutated object).

Q5: WHAT IS THE DIFFERENCE BETWEEN A REPLICASET AND A DEPLOYMENT? WHY DO WE ALMOST NEVER CREATE REPLICASETS DIRECTLY?

==========

Answer:

A ReplicaSet ensures N identical Pods are running. It's the mechanism that maintains the desired replica count.

A Deployment is a higher-level controller that manages ReplicaSets. It adds:
- Rolling updates — creates a new ReplicaSet, scales it up, scales old one down
- Rollback — kubectl rollout undo switches back to the previous ReplicaSet
- Revision history — keeps old ReplicaSets (configurable: revisionHistoryLimit)
- Update strategy — RollingUpdate (default) or Recreate

[Code Block]
```
Deployment: my-app (replicas: 3)
  └── ReplicaSet: my-app-7b4f9c (current, 3 Pods running)
  └── ReplicaSet: my-app-5a2d8e (previous revision, 0 Pods, kept for rollback)

On image update (v1 → v2):
  └── ReplicaSet: my-app-7b4f9c (v1, scaling DOWN: 3 → 2 → 1 → 0)
  └── ReplicaSet: my-app-c3e7f1 (v2, scaling UP: 0 → 1 → 2 → 3)
```
[End Code Block]

Why not create ReplicaSets directly? You lose rolling updates, rollbacks, and re

vision history. There's no mechanism to transition from one version to another — you'd have to manually create a new ReplicaSet and delete the old one. Deployments automate all of that.

Q6: EXPLAIN THE KUBERNETES SCHEDULING PROCESS. WHAT ARE TAINTS, TOLERATIONS, AND NODE AFFINITY?

Answer:

Scheduling Flow

[Code Block]
```
Pod created (no nodeName) → Scheduler picks it up

Phase 1: FILTERING (eliminate unsuitable nodes)
  - Does the node have enough CPU/memory?
  - Does the Pod tolerate the node's taints?
  - Does the Pod's nodeSelector/nodeAffinity match?
  - Is the node cordoned (unschedulable)?
  - PV affinity: does the PVC's zone match the node?

Phase 2: SCORING (rank remaining nodes)
  - LeastRequestedPriority: prefer nodes with more free resources
  - BalancedResourceAllocation: prefer even CPU/memory ratio
  - InterPodAffinity: prefer nodes that satisfy Pod affinity rules
  - SpreadConstraints: spread across zones/nodes

Phase 3: BIND
  - Pick highest-scoring node
  - Write nodeName to Pod spec
```
[End Code Block]

Taints & Tolerations

Taints are on nodes — they REPEL Pods that don't tolerate them.
Tolerations are on Pods — they ALLOW the Pod to be scheduled on tainted nodes.

[Code Block]
```
Node taint (applied by admin):
  kubectl taint nodes node1 gpu=true:NoSchedule

  Effect: No Pod will be scheduled on node1 unless it tolerates this taint.

Pod toleration:
  tolerations:
    - key: "gpu"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

  This Pod CAN be scheduled on node1.

Common effects:
  NoSchedule     → Don't schedule new Pods (existing stay)
  PreferNoSchedule → Try not to, but allow if no other option
  NoExecute      → Evict existing Pods that don't tolerate
```
[End Code Block]

Use case: GPU nodes, dedicated node pools, cordoning nodes for maintenance.

Node Affinity

Like nodeSelector but more expressive — supports soft (preferred) and hard (required) rules.

[Code Block]
```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:   ← HARD rule
      nodeSelectorTerms:
        - matchExpressions:
            - key: topology.kubernetes.io/zone
              operator: In
              values: ["us-east-1a", "us-east-1b"]

    preferredDuringSchedulingIgnoredDuringExecution:  ← SOFT rule
      - weight: 80
        preference:
          matchExpressions:
            - key: instance-type
              operator: In
              values: ["m5.xlarge"]
```
[End Code Block]

Hard: Pod MUST go to us-east-1a or 1b. Fails if no matching node.
Soft: Prefer m5.xlarge instances, but schedule elsewhere if unavailable.

Q7: WHAT'S THE DIFFERENCE BETWEEN A LIVENESS PROBE, READINESS PROBE, AND STARTUP PROBE?
==========

Answer:

Probe	Purpose	If it FAILS	When to use
Liveness	Is the container alive and healthy?	kubelet kills and restarts the container	Detect deadlocks, hung processes
Readiness	Can the container accept traffic?	Pod removed from Service endpoints (no traffic routed)	Warming up, loading data, temporary overload
Startup	Has the container finished starting?	kubelet kills and restarts (after failureThreshold)	Slow-starting apps (Java, large ML models)

[Code Block]
```
Key behavior:

Liveness:
  Container is running but deadlocked (thread stuck, infinite loop)
  Liveness probe fails → kubelet restarts the container
  Without liveness: container stays "Running" but serves nothing

Readiness:
  Container is starting up, loading a model into memory (takes 30s)
  Readiness probe fails → kube-proxy removes Pod from Service endpoints
  Traffic goes to other ready Pods
  Once ready → Pod added back to endpoints
  Without readiness: traffic hits Pod before it can serve → errors
```

```
Startup:
  Java app takes 120s to start (Spring Boot with large context)
  Without startup probe: liveness probe fires at 10s, container isn't ready
    → liveness fails → kubelet restarts → restart loop
  With startup probe: liveness/readiness probes disabled until startup succeeds
    → gives the app time to boot
    → once startup passes, liveness and readiness take over

Probe types:
  httpGet:    GET /healthz → expect 200
  tcpSocket:  connect to port 8080 → expect connection success
  exec:       run command inside container → expect exit code 0
```
[End Code Block]

Best practice at 7 YOE: ALWAYS set readiness probes. Set liveness probes for long-running processes that can deadlock. Use startup probes for slow-starting apps with aggressive liveness settings.

Q8: EXPLAIN KUBERNETES NETWORKING. HOW DOES POD-TO-POD COMMUNICATION WORK ACROSS NODES?

Answer:

The Three Networking Rules in K8s

[Code Block]
```
Rule 1: Every Pod gets its own unique IP address
Rule 2: Any Pod can communicate with any other Pod using its IP
         (without NAT) — even across nodes
Rule 3: Agents on a node (kubelet, kube-proxy) can communicate
         with all Pods on that node
```
[End Code Block]

How It Works (CNI Plugin Mechanics)

[Code Block]
```
Same Node Communication:
  Pod A (10.244.1.5) → Pod B (10.244.1.8)

  Phase 1: Pod A sends packet to its virtual ethernet (veth) interface
  Phase 2: veth connects to the node's bridge (cbr0 / cni0)
  Phase 3: Bridge sees destination 10.244.1.8 is local
  Phase 4: Forwards to Pod B's veth interface
  Phase 5: Pod B receives the packet

  Path: Pod A → veth → bridge → veth → Pod B

Cross-Node Communication:
  Pod A (10.244.1.5 on Node 1) → Pod C (10.244.2.9 on Node 2)

  Phase 1: Pod A sends packet to its veth → node's bridge
  Phase 2: Bridge doesn't know 10.244.2.0/24 locally
  Phase 3: Packet goes to the node's routing table
  Phase 4: Route says 10.244.2.0/24 → Node 2 (via overlay or direct routing)
  Phase 5: Packet encapsulated (VXLAN/Geneve for overlay CNIs) or routed directly
```

```
  Phase 6: Arrives at Node 2, decapsulated
  Phase 7: Node 2's bridge routes to Pod C's veth
  Phase 8: Pod C receives the packet

Popular CNI plugins:
  Calico:  Uses BGP for direct routing (no overlay) — fastest
  Flannel: VXLAN overlay — simple, moderate performance
  Cilium:  eBPF-based — high performance, advanced features (security, observabi
lity)
  Weave:   Mesh overlay — simple setup
[End Code Block]
```

Q9: WHAT ARE SERVICES IN KUBERNETES? EXPLAIN CLUSTERIP, NODEPORT, LOADBALANCER, AND HEADLESS.

==========

Answer:

```
[Code Block]
ClusterIP (default):
  - Internal-only virtual IP
  - Accessible only from within the cluster
  - kube-proxy creates iptables/IPVS rules to load-balance across Pod IPs
  - Example: backend API service called by frontend Pods

  Service: my-api → ClusterIP 10.96.45.12:80
    → Pod 10.244.1.5:8080
    → Pod 10.244.2.8:8080
    → Pod 10.244.3.3:8080

NodePort:
  - Extends ClusterIP
  - Opens a static port (30000–32767) on EVERY node
  - External traffic can hit <any-node-IP>:<nodePort>
  - kube-proxy forwards to the ClusterIP → Pods

  Service: my-api → NodePort 31080
    External: http://192.168.1.10:31080 → Pod

LoadBalancer:
  - Extends NodePort
  - Provisions a cloud load balancer (AWS ELB/ALB, GCP LB)
  - Cloud LB gets a public IP → routes to NodePort → Pods
  - Used for production external traffic

  Service: my-api → LoadBalancer
    Public: http://a1b2c3.elb.amazonaws.com → NodePort → Pod

Headless (ClusterIP: None):
  - No virtual IP, no load balancing by kube-proxy
  - DNS returns the individual Pod IPs directly
  - Used with StatefulSets for stable per-Pod DNS

  DNS query: db-headless.default.svc.cluster.local
    → returns: 10.244.1.5, 10.244.2.8, 10.244.3.3 (all Pod IPs)

  Each Pod also gets:
    db-0.db-headless.default.svc.cluster.local → 10.244.1.5
    db-1.db-headless.default.svc.cluster.local → 10.244.2.8
[End Code Block]
```

When to use which:
- ClusterIP: Internal service-to-service communication (90% of cases)
- NodePort: Quick external access for testing/dev (not production)
- LoadBalancer: Production external traffic with cloud provider
- Headless: StatefulSets, service discovery where clients need individual Pod IPs

---

Q10: WHAT ARE NETWORK POLICIES AND HOW DO THEY WORK?

Answer:

Network Policies are Kubernetes-native firewall rules at the Pod level. By default, all Pods can talk to all other Pods (open network). Network Policies restrict this.

[Code Block]
```
Default: All Pods can communicate freely (no policies)

When you apply a NetworkPolicy to a Pod:
  - ALL ingress/egress traffic to that Pod is DENIED by default
  - Only traffic explicitly allowed by the policy is permitted

Example: Allow only frontend Pods to reach backend Pods on port 8080

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:                          ← apply to Pods with label app=backend
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:                  ← allow traffic FROM Pods with app=frontend
            matchLabels:
              app: frontend
      ports:
        - port: 8080                    ← only on port 8080

Result:
  frontend Pod → backend Pod:8080    ✅ ALLOWED
  random Pod → backend Pod:8080      ❌ DENIED
  frontend Pod → backend Pod:9090    ❌ DENIED (wrong port)
```
[End Code Block]

Important: Network Policies require a CNI plugin that supports them (Calico, Cilium, Weave). Flannel does NOT enforce Network Policies.

Q11: HOW DOES DNS WORK INSIDE A KUBERNETES CLUSTER?

Answer:

[Code Block]
```
CoreDNS runs as a Deployment in kube-system namespace (typically 2 replicas).
Every Pod gets /etc/resolv.conf pointing to the CoreDNS Service ClusterIP.

DNS resolution patterns:

  Service (same namespace):
    my-service → my-service.default.svc.cluster.local
    Resolves to: ClusterIP (e.g., 10.96.45.12)

  Service (cross-namespace):
    my-service.other-namespace → my-service.other-namespace.svc.cluster.local

  Headless Service Pod:
    pod-0.my-headless.default.svc.cluster.local → Pod IP directly

  External:
    google.com → forwarded to upstream DNS (e.g., 8.8.8.8)

Search domains in Pod's /etc/resolv.conf:
  search default.svc.cluster.local svc.cluster.local cluster.local
  nameserver 10.96.0.10  ← CoreDNS Service IP

This is why you can just write "my-service" in code instead of the FQDN —
the search domains expand it automatically.
```
[End Code Block]

Common issue: DNS resolution latency under load. CoreDNS has limited resources by default. At scale (1000+ Pods doing frequent lookups), you may need to scale CoreDNS replicas or enable NodeLocal DNSCache (DaemonSet that caches DNS on every node).

Q12: WHAT IS AN INGRESS? HOW IS IT DIFFERENT FROM A LOADBALANCER SERVICE?

Answer:

[Code Block]
```
LoadBalancer Service:
  - One cloud load balancer PER service
  - 10 services = 10 load balancers = 10 public IPs = expensive
  - No path-based or host-based routing
  - L4 (TCP/UDP level)

Ingress:
  - One load balancer for ALL services (via Ingress Controller)
  - Routes traffic based on HOST and PATH
  - L7 (HTTP level) — can inspect headers, paths, hosts
  - TLS termination, rewrites, rate limiting (depending on controller)

Example:
  One Ingress Controller (NGINX / Traefik / ALB)
    → api.myapp.com/users   → user-service:8080
    → api.myapp.com/orders  → order-service:8080
    → web.myapp.com/        → frontend-service:3000

  Instead of 3 LoadBalancer Services (3 cloud LBs, 3 public IPs),
  you have 1 Ingress Controller + 1 cloud LB.
```

```
Ingress spec:
  rules:
    - host: api.myapp.com
      http:
        paths:
          - path: /users
            pathType: Prefix
            backend:
              service:
                name: user-service
                port:
                  number: 8080
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 8080
```
[End Code Block]

Key point: Ingress by itself does nothing. You need an Ingress Controller (NGINX Ingress Controller, Traefik, AWS ALB Controller, etc.) installed in the cluster. The Ingress resource is just the routing rule; the controller is the engine that implements it.

Q13: EXPLAIN PERSISTENT VOLUMES (PV), PERSISTENT VOLUME CLAIMS (PVC), AND STORAGECLASSES.
==========

Answer:

[Code Block]
```
PV (Persistent Volume):
  - A piece of storage provisioned in the cluster
  - Can be an EBS volume, NFS share, GCE Persistent Disk, etc.
  - Cluster-level resource (not namespaced)
  - Has a lifecycle independent of any Pod

PVC (Persistent Volume Claim):
  - A REQUEST for storage by a user/Pod
  - Specifies size, access mode, storage class
  - Namespaced resource
  - Kubernetes BINDS a PVC to a matching PV

StorageClass:
  - Defines HOW to dynamically provision storage
  - Specifies the provisioner (aws-ebs, gce-pd, etc.), parameters, reclaim policy
  - Eliminates the need to pre-create PVs manually

Flow:

  STATIC PROVISIONING (admin pre-creates PVs):
    Admin creates PV (10Gi, ReadWriteOnce, EBS vol-123)
    User creates PVC requesting 10Gi, ReadWriteOnce
    Kubernetes binds PVC → PV
    Pod mounts the PVC

  DYNAMIC PROVISIONING (with StorageClass):
```

```
    Admin creates StorageClass (provisioner: aws-ebs, type: gp3)
    User creates PVC requesting 10Gi with storageClassName: gp3
    Kubernetes calls the provisioner → creates EBS volume automatically
    PV is auto-created, bound to PVC
    Pod mounts the PVC

Reclaim policies:
  Retain:  PV kept after PVC deletion (manual cleanup)
  Delete:  PV and underlying storage deleted when PVC is deleted
  Recycle: (deprecated) basic wipe and reuse

Access modes:
  ReadWriteOnce (RWO):  one node can mount read-write
  ReadOnlyMany  (ROX):  many nodes can mount read-only
  ReadWriteMany (RWX):  many nodes can mount read-write (NFS, EFS)
[End Code Block]
```

Q14: HOW DO STATEFULSETS HANDLE STORAGE DIFFERENTLY FROM DEPLOYMENTS?

Answer:

```
[Code Block]
Deployment:
  - All Pods share the same PVC (or have no persistent storage)
  - If Pod dies and reschedules to another node, it gets the same shared PVC
  - No per-Pod storage identity

StatefulSet:
  - Uses volumeClaimTemplates — each Pod gets its OWN PVC
  - PVCs are named deterministically: data-db-0, data-db-1, data-db-2
  - If db-1 Pod dies, a new db-1 Pod is created and re-attached to data-db-1
  - PVCs are NOT deleted when Pods are deleted (data survives)

Example:
  StatefulSet: db (replicas: 3)
    volumeClaimTemplates:
      - metadata:
          name: data
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi

  Creates:
    Pod db-0 → PVC data-db-0 → PV (50Gi EBS)
    Pod db-1 → PVC data-db-1 → PV (50Gi EBS)
    Pod db-2 → PVC data-db-2 → PV (50Gi EBS)

  db-1 crashes → new db-1 Pod → re-mounts data-db-1 (same data)

  Scale down to 2 replicas:
    Pod db-2 deleted
    PVC data-db-2 STILL exists (not deleted)
    Scale back to 3 → new db-2 re-mounts data-db-2

This is why StatefulSets are used for databases — each replica has
its own persistent identity and storage.
[End Code Block]
```

Q15: WHAT IS RBAC IN KUBERNETES? EXPLAIN ROLE, CLUSTERROLE, ROLEBINDING, CLUSTER ROLEBINDING.
==========

Answer:

[Code Block]
```
RBAC = Role-Based Access Control
Controls WHO can do WHAT on WHICH resources.

4 objects:

Role              → defines permissions within a NAMESPACE
ClusterRole       → defines permissions CLUSTER-WIDE
RoleBinding       → grants a Role to a user/SA within a NAMESPACE
ClusterRoleBinding → grants a ClusterRole CLUSTER-WIDE

Example: Allow CI/CD service account to manage Deployments in "staging" namespac
e

Step 1: Create a Role
  apiVersion: rbac.authorization.k8s.io/v1
  kind: Role
  metadata:
    namespace: staging
    name: deployment-manager
  rules:
    - apiGroups: ["apps"]
      resources: ["deployments"]
      verbs: ["get", "list", "create", "update", "patch", "delete"]

Step 2: Create a RoleBinding
  kind: RoleBinding
  metadata:
    namespace: staging
    name: ci-deploy-binding
  subjects:
    - kind: ServiceAccount
      name: ci-bot
      namespace: staging
  roleRef:
    kind: Role
    name: deployment-manager

Result:
  ci-bot SA can CRUD Deployments in staging namespace ONLY.
  ci-bot SA CANNOT touch Deployments in production namespace.
  ci-bot SA CANNOT touch Pods, Services, Secrets (not in the Role).

ClusterRole + ClusterRoleBinding:
  Same idea but applies across ALL namespaces.
  Use for: cluster admins, node management, PV management.
```
[End Code Block]

Q16: HOW DO YOU MANAGE SECRETS IN KUBERNETES? WHAT ARE THE SECURITY CONCERNS?
==========

Answer:

[Code Block]
```
Default Kubernetes Secrets:
  - Stored in etcd as base64 encoded (NOT encrypted by default!)
  - base64 is encoding, NOT encryption — anyone with etcd access can read them
  - Mounted as files in /var/run/secrets/... or exposed as env vars

Security concerns:
  1. etcd stores Secrets in plaintext (unless encryption at rest is configured)
  2. Any Pod in a namespace can mount any Secret in that namespace (unless RBAC
restricts it)
  3. Secrets in env vars appear in process listings and crash dumps
  4. Secrets are cached on nodes in tmpfs — accessible to anyone with node SSH a
ccess

Hardening steps:

  Level 1: Enable encryption at rest for etcd
    apiVersion: apiserver.config.k8s.io/v1
    kind: EncryptionConfiguration
    resources:
      - resources: ["secrets"]
        providers:
          - aescbc:
              keys:
                - name: key1
                  secret: <base64-encoded-key>

  Level 2: Use RBAC to restrict who can read Secrets
    Only specific ServiceAccounts should have "get" on Secrets

  Level 3: Use external secret managers
    - HashiCorp Vault (with Vault Agent Injector)
    - AWS Secrets Manager (with External Secrets Operator)
    - Azure Key Vault
    Secrets never stored in etcd — fetched at runtime from external vault

  Level 4: Mount as files, not env vars
    Files can have restricted permissions (0400)
    Env vars leak in process listings and pod describe output

  Level 5: Enable audit logging for Secret access
    Track who accessed which Secrets and when
```
[End Code Block]

Q17: WHAT IS A POD SECURITY CONTEXT? WHAT IS A PODSECURITYADMISSION / PODSECURIT
YSTANDARD?
==========

Answer:

[Code Block]
```
Security Context: Per-Pod or per-container security settings

  securityContext:
    runAsNonRoot: true          ← container must run as non-root user
    runAsUser: 1000             ← run as UID 1000
    readOnlyRootFilesystem: true ← container filesystem is read-only
    allowPrivilegeEscalation: false
```

```
    capabilities:
      drop: ["ALL"]             ← drop all Linux capabilities

Pod Security Standards (PSS) — 3 levels:

  Privileged:  No restrictions (for system-level Pods like CNI, kube-proxy)
  Baseline:    Prevents obvious escalations (no hostNetwork, no privileged, etc.
)
  Restricted:  Hardened (must run as non-root, drop caps, read-only fs)

Pod Security Admission (PSA) — enforces PSS at namespace level:

  kubectl label namespace production \
    pod-security.kubernetes.io/enforce=restricted \
    pod-security.kubernetes.io/warn=restricted

  Any Pod in "production" namespace that violates "restricted" policy → REJECTED
.

This replaced the deprecated PodSecurityPolicy (PSP) in K8s 1.25+.
[End Code Block]
```

Q18: EXPLAIN HORIZONTAL POD AUTOSCALER (HPA). HOW DOES IT WORK AND WHAT ARE ITS LIMITATIONS?

==========

Answer:

```
[Code Block]
HPA automatically scales Pod replicas based on observed metrics.

How it works:

  Phase 1: METRICS COLLECTION
    - Metrics Server collects CPU/memory usage from kubelet (every 15s default)
    - Custom metrics can come from Prometheus Adapter

  Phase 2: HPA CONTROLLER checks every 15 seconds (configurable)
    - Reads current metric value (e.g., avg CPU = 80%)
    - Compares to target (e.g., target CPU = 50%)
    - Calculates desired replicas:
        desiredReplicas = currentReplicas × (currentMetric / targetMetric)
        desiredReplicas = 3 × (80 / 50) = 4.8 → rounds up to 5

  Phase 3: SCALE
    - HPA updates Deployment replicas: 3 → 5
    - Deployment controller creates 2 new Pods
    - Once new Pods are ready and absorb load, CPU drops

  Phase 4: COOLDOWN
    - Scale-up cooldown: 0s (immediate, default since K8s 1.18)
    - Scale-down cooldown: 5 minutes (default stabilization window)
    - Prevents flapping (rapid scale up/down cycles)

Example:
  apiVersion: autoscaling/v2
  kind: HorizontalPodAutoscaler
  spec:
    scaleTargetRef:
      kind: Deployment
```

```
      name: my-api
    minReplicas: 2
    maxReplicas: 20
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 50

Limitations:
  - Requires resource requests to be set (HPA uses requests as baseline)
  - Metrics Server latency: ~15-30s delay before HPA reacts
  - Doesn't handle burst traffic well (scale-up takes 30-60s for new Pods)
  - Custom metrics (RPS, queue depth) need Prometheus + Adapter setup
  - Can conflict with Cluster Autoscaler (HPA wants more Pods but nodes are full
)
  - VPA and HPA on the same metric (CPU) can conflict — don't overlap
[End Code Block]
```

Q19: WHAT IS THE CLUSTER AUTOSCALER? HOW DOES IT WORK WITH HPA?

Answer:

```
[Code Block]
Cluster Autoscaler scales the NUMBER OF NODES (not Pods).
HPA scales Pods. They work together:

  Load increases → HPA creates more Pods
  → Not enough node capacity for new Pods → Pods stay Pending
  → Cluster Autoscaler detects Pending Pods
  → Provisions new nodes (calls cloud API: EC2, GKE node pool, etc.)
  → New nodes join cluster → Pending Pods get scheduled

  Load decreases → HPA scales down Pods
  → Some nodes become underutilized (<50% of capacity)
  → Cluster Autoscaler waits stabilization period (~10 min)
  → Evicts Pods (respects PDBs) → drains node → terminates node

Scale-up flow:
  Phase 1: DETECT Pending Pods — scheduler can't place them (no resources)
  Phase 2: SIMULATE — which node group/pool can fit these Pods?
  Phase 3: PROVISION — call cloud API to add node(s)
  Phase 4: WAIT — node boots, joins cluster, passes health check (~1-3 min)
  Phase 5: SCHEDULE — pending Pods are assigned to new node

Scale-down flow:
  Phase 1: IDENTIFY underutilized nodes (CPU+memory < threshold, default 50%)
  Phase 2: CHECK — are there Pods that can't be moved?
    (PDBs, local storage, node-selector-only Pods)
  Phase 3: DRAIN — evict Pods (respecting PDBs and graceful termination)
  Phase 4: TERMINATE — deregister and delete the node

Key gotcha:
  Scale-up takes 1-3 minutes (node provisioning). For latency-sensitive
  workloads, over-provision slightly (keep warm spare capacity) to avoid
  waiting for new nodes during traffic spikes.
[End Code Block]
```

Q20: HOW DO YOU DO A ZERO-DOWNTIME DEPLOYMENT IN KUBERNETES?

Answer:

[Code Block]
Required ingredients:

```
1. ROLLING UPDATE STRATEGY (default in Deployments)
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0     ← never kill old Pods before new ones are ready
      maxSurge: 1           ← allow 1 extra Pod during rollout

  Flow:
    Start:   v1-pod-a ✅, v1-pod-b ✅, v1-pod-c ✅  (3 replicas)
    Step 1:  v2-pod-d created (surge), v1 all still running    (4 Pods)
    Step 2:  v2-pod-d passes readiness → v1-pod-a terminated   (3 Pods)
    Step 3:  v2-pod-e created, passes readiness → v1-pod-b terminated
    Step 4:  v2-pod-f created, passes readiness → v1-pod-c terminated
    End:     v2-pod-d ✅, v2-pod-e ✅, v2-pod-f ✅

2. READINESS PROBE (critical!)
  - New Pod must pass readiness BEFORE old Pod is terminated
  - Without readiness: K8s assumes Pod is ready immediately
    → kills old Pod → new Pod isn't actually ready → dropped requests

3. GRACEFUL SHUTDOWN
  In your application:
    - Handle SIGTERM signal
    - Stop accepting new requests
    - Finish in-flight requests
    - Exit cleanly

  In Pod spec:
    terminationGracePeriodSeconds: 30  ← K8s waits 30s before SIGKILL

4. PRE-STOP HOOK (handles the race condition)
  Problem: kube-proxy takes a few seconds to update iptables rules.
    Old Pod gets SIGTERM and starts shutting down, but kube-proxy
    still routes traffic to it for a brief window → errors.

  Fix:
    lifecycle:
      preStop:
        exec:
          command: ["sleep", "5"]  ← wait 5s before shutdown

  Timeline:
    T+0s: Pod marked for termination
    T+0s: kube-proxy starts removing Pod from endpoints (async)
    T+0-5s: preStop hook runs (sleep 5), Pod still accepts traffic
    T+5s: kube-proxy has updated, no new traffic arrives
    T+5s: SIGTERM sent, app gracefully shuts down
    T+35s: SIGKILL if still running

5. POD DISRUPTION BUDGET (PDB)
  Prevents K8s from killing too many Pods at once during voluntary disruptions
```

```
  (rolling updates, node drain, cluster autoscaler scale-down):

  apiVersion: policy/v1
  kind: PodDisruptionBudget
  spec:
    minAvailable: 2          ← always keep at least 2 Pods running
    selector:
      matchLabels:
        app: my-api
[End Code Block]
```

Q21: WHAT IS A CONFIGMAP AND HOW DOES IT DIFFER FROM A SECRET?

Answer:

```
[Code Block]
ConfigMap: stores non-sensitive configuration as key-value pairs
Secret:    stores sensitive data (passwords, tokens, keys)

Key differences:
  - Secrets are base64 encoded (ConfigMaps are plaintext)
  - Secrets can have encryption at rest enabled in etcd
  - Secrets have type field (Opaque, kubernetes.io/tls, etc.)
  - kubelet keeps Secrets in tmpfs (not written to disk on nodes)

Mounting options (same for both):

  As environment variables:
    env:
      - name: DB_HOST
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: database_host

  As volume files:
    volumes:
      - name: config
        configMap:
          name: app-config
    volumeMounts:
      - name: config
        mountPath: /etc/config

  Hot reload: When mounted as files, updates to ConfigMap/Secret
  propagate to the Pod automatically (~30-60s, kubelet sync period).
  When used as env vars: NO hot reload — Pod must restart.
[End Code Block]
```

Q22: WHAT HAPPENS WHEN A NODE BECOMES NOTREADY? WALK THROUGH THE POD EVICTION PROCESS.

Answer:

```
[Code Block]
Phase 1: DETECTION
```

```
  - kubelet sends heartbeats to API server every 10s (node lease)
  - If API server doesn't receive heartbeat for 40s (default):
      node-monitor-grace-period = 40s
  - Node status changes: Ready → NotReady

Phase 2: TAINT APPLIED
  - Node controller adds taint:
      node.kubernetes.io/not-ready:NoExecute
  - Also: node.kubernetes.io/unreachable:NoExecute (if network partition)

Phase 3: TOLERATION TIMEOUT
  - Pods have a default toleration for not-ready with
    tolerationSeconds: 300 (5 minutes)
  - Pod stays on the NotReady node for 5 minutes (hoping node recovers)

Phase 4: EVICTION (after 5 minutes)
  - If node doesn't recover, Pod is evicted
  - Pod object updated: status = Failed, reason = "NodeLost"
  - If Pod is managed by a controller (Deployment, StatefulSet):
      Controller sees fewer Pods than desired → creates replacement Pod
      Scheduler places new Pod on a healthy node

Phase 5: STUCK PODS (StatefulSet edge case)
  - StatefulSet won't create db-1 replacement until old db-1 is confirmed dead
  - If node is partitioned (not truly dead), old db-1 might still be running
  - Force delete may be needed: kubectl delete pod db-1 --force --grace-period=0
  - Risk: two copies of db-1 running (split-brain) — be careful with stateful wo
rkloads

Timeline:
  T+0s:    Last heartbeat received
  T+40s:   Node marked NotReady
  T+40s:   NoExecute taint applied
  T+340s:  (5 min later) Pods evicted
  T+340s+: Replacement Pods scheduled elsewhere
[End Code Block]
```

Q23: HOW DO RESOURCE REQUESTS AND LIMITS WORK? WHAT HAPPENS WHEN A POD EXCEEDS I
TS LIMITS?
==========

Answer:

```
[Code Block]
Requests: GUARANTEED minimum resources — used by scheduler for placement
Limits:   MAXIMUM resources — enforced by kubelet/cgroups

resources:
  requests:
    cpu: "250m"       ← scheduler ensures node has 250 millicores free
    memory: "256Mi"   ← scheduler ensures node has 256Mi free
  limits:
    cpu: "1000m"      ← can burst up to 1 core
    memory: "512Mi"   ← hard cap at 512Mi

CPU behavior:
  - Requests: Pod guaranteed 250m CPU time
  - If pod tries to use more than 1000m (limit): CPU THROTTLED
  - Pod is NOT killed for exceeding CPU limit — just slowed down
  - CPU is compressible — can be taken away and given back
```

```
Memory behavior:
  - Requests: 256Mi guaranteed
  - If Pod tries to allocate more than 512Mi (limit): OOMKilled
  - Memory is incompressible — can't "take back" allocated memory
  - kubelet kills the container, restarts it (CrashLoopBackOff if repeated)

QoS Classes (based on requests/limits):

  Guaranteed: requests == limits for ALL containers
    → Last to be evicted under node pressure
    → Best performance guarantees

  Burstable: requests < limits (or only requests set)
    → Evicted after BestEffort, before Guaranteed

  BestEffort: no requests or limits set
    → First to be evicted under node pressure
    → Avoid in production

Node under memory pressure:
  Phase 1: kubelet detects memory pressure
  Phase 2: Evicts BestEffort Pods first
  Phase 3: Evicts Burstable Pods exceeding requests
  Phase 4: Evicts Guaranteed Pods (last resort)
[End Code Block]
```

Q24: EXPLAIN INIT CONTAINERS. WHEN WOULD YOU USE THEM?

Answer:

```
[Code Block]
Init containers run BEFORE app containers start, one at a time, sequentially.
All init containers must succeed before the app container starts.

Use cases:
  1. Wait for a dependency (DB, API) to be available
  2. Clone a git repo or download config before app starts
  3. Run database migrations
  4. Set file permissions on mounted volumes
  5. Register with a service registry

Example: Wait for database before starting app

  initContainers:
    - name: wait-for-db
      image: busybox
      command: ['sh', '-c',
        'until nc -z postgres-service 5432; do echo waiting; sleep 2; done']

  containers:
    - name: my-app
      image: my-app:v1

Flow:
  Phase 1: Pod scheduled, init container "wait-for-db" starts
  Phase 2: Init container polls postgres:5432 every 2s
  Phase 3: DB becomes available → init container exits successfully
  Phase 4: App container "my-app" starts
```

```
  Phase 5: Pod status: Running

If init container fails:
  → kubelet retarts it (based on Pod restartPolicy)
  → App container NEVER starts until init succeeds
  → Pod status: Init:Error or Init:CrashLoopBackOff
[End Code Block]
```

Q25: WHAT ARE CUSTOM RESOURCE DEFINITIONS (CRDS) AND OPERATORS?

Answer:

```
[Code Block]
CRD (Custom Resource Definition):
  - Extends the Kubernetes API with your own resource types
  - After creating a CRD, you can kubectl apply your custom objects
  - Stored in etcd just like Pods and Deployments

Example: Create a Database CRD
  apiVersion: apiextensions.k8s.io/v1
  kind: CustomResourceDefinition
  metadata:
    name: databases.mycompany.io
  spec:
    group: mycompany.io
    names:
      kind: Database
      plural: databases
    scope: Namespaced
    versions:
      - name: v1
        served: true
        storage: true

Now you can do:
  kubectl get databases
  kubectl apply -f my-postgres.yaml
    apiVersion: mycompany.io/v1
    kind: Database
    metadata:
      name: my-postgres
    spec:
      engine: postgresql
      version: "15"
      replicas: 3
      storage: 100Gi

But the CRD alone does nothing. You need...

OPERATOR:
  - A controller that watches for your custom resources and ACTS on them
  - Encodes operational knowledge (install, upgrade, backup, failover)
  - Runs as a Pod in the cluster

  Operator watches Database CRD:
    Phase 1: Sees "my-postgres" created
    Phase 2: Creates StatefulSet (3 replicas), Service, PVCs, ConfigMaps
    Phase 3: Configures replication (primary + 2 replicas)
    Phase 4: Sets up automated backups (CronJob)
```

```
    Phase 5: Monitors health, handles failover automatically

  Popular operators:
    - Prometheus Operator (monitoring)
    - Strimzi (Kafka on K8s)
    - Zalando Postgres Operator
    - Cert-Manager (TLS certificates)

CRD + Operator = extending Kubernetes to manage ANY stateful system
with the same declarative model (kubectl apply, reconciliation loops).
[End Code Block]
```

Q26: A POD IS IN CRASHLOOPBACKOFF. HOW DO YOU DEBUG IT?

Answer:

```
[Code Block]
CrashLoopBackOff means: container starts, crashes, K8s restarts it,
crashes again, K8s backs off (increasing delays: 10s, 20s, 40s... up to 5min).

Debugging flow:

Step 1: CHECK POD EVENTS
  kubectl describe pod <pod-name>
  → Look at Events section:
    "Back-off restarting failed container"
    "OOMKilled"  ← memory limit exceeded
    "Error: ImagePullBackOff" ← wrong image (different issue)
    "CrashLoopBackOff" with exit code

Step 2: CHECK CONTAINER LOGS
  kubectl logs <pod-name>                   ← current container logs
  kubectl logs <pod-name> --previous        ← logs from PREVIOUS crashed contain
er
  (--previous is critical — current container may have no logs yet)

Step 3: CHECK EXIT CODE
  kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[0].lastStat
e}'

  Exit code 0:    app exited normally (shouldn't happen for a server)
  Exit code 1:    app error (check logs for exception/stack trace)
  Exit code 137:  SIGKILL — likely OOMKilled (check memory limits)
  Exit code 139:  Segfault
  Exit code 143:  SIGTERM — graceful shutdown (but why?)

Step 4: CHECK RESOURCE LIMITS
  kubectl describe pod → look for "OOMKilled" in lastState
  If OOMKilled: increase memory limits

Step 5: RUN A DEBUG SHELL
  kubectl run debug --image=busybox -it --rm -- sh
  → Test connectivity, DNS, environment from inside the cluster

  Or ephemeral debug container (K8s 1.23+):
  kubectl debug -it <pod-name> --image=busybox --target=<container-name>

Common causes:
  - App crashes on startup (missing config, wrong env vars, bad connection strin
```

g)
  - OOMKilled (memory limit too low)
  - Missing dependency (DB not reachable, Secret not mounted)
  - Liveness probe misconfigured (kills healthy container)
  - Read-only filesystem but app tries to write temp files
[End Code Block]

Q27: HOW DOES KUBERNETES HANDLE ROLLING BACK A FAILED DEPLOYMENT?
==========

Answer:

[Code Block]
Kubernetes keeps old ReplicaSets as revision history.

Scenario: v2 deployment is failing (Pods crash on startup)

Automatic detection:
  - Rolling update creates v2 Pods
  - v2 Pods fail readiness probes
  - With maxUnavailable: 0, old v1 Pods are NOT killed
  - Rollout stalls — v2 Pods keep crashing, v1 Pods keep serving
  - progressDeadlineSeconds (default 600s) → after 10 min, Deployment
    condition "Progressing" = False

Manual rollback:
  kubectl rollout undo deployment/my-app
  → Reverts to previous ReplicaSet (v1)
  → v2 ReplicaSet scaled to 0
  → v1 ReplicaSet scaled back up
  → Traffic returns to v1 Pods

Rollback to specific revision:
  kubectl rollout history deployment/my-app
    REVISION CHANGE-CAUSE
    1   initial deploy
    2   image update to v2
    3   image update to v3

  kubectl rollout undo deployment/my-app --to-revision=1

Under the hood:
  - Deployment controller copies the Pod template from the old ReplicaSet
  - Creates a new ReplicaSet with that template
  - Scales it up, scales current (broken) one down
  - It's effectively a new rollout to the old version

Prevention:
  - Use readiness probes (rollout won't proceed if new Pods aren't ready)
  - Set maxUnavailable: 0 (never kill old before new is ready)
  - Set progressDeadlineSeconds to catch stalled rollouts
[End Code Block]

Q28: EXPLAIN THE DIFFERENCE BETWEEN A JOB AND A CRONJOB.
==========

Answer:

[Code Block]
```
Job:
  - Runs one or more Pods to completion (exit code 0)
  - Pod is NOT restarted after successful completion
  - Good for: DB migration, batch processing, one-time tasks

  spec:
    completions: 5        ← run 5 Pods total (all must succeed)
    parallelism: 2        ← run 2 at a time
    backoffLimit: 3       ← retry failed Pods 3 times before marking Job as fail
ed
    activeDeadlineSeconds: 300  ← kill job if not done in 5 minutes

CronJob:
  - Creates Jobs on a schedule (cron syntax)
  - Good for: periodic cleanup, report generation, backups

  spec:
    schedule: "0 2 * * *"              ← every day at 2 AM
    concurrencyPolicy: Forbid          ← don't start new if previous still runni
ng
    successfulJobsHistoryLimit: 3      ← keep 3 completed Jobs
    failedJobsHistoryLimit: 1          ← keep 1 failed Job
    startingDeadlineSeconds: 200       ← if missed by 200s, skip this run

  Concurrency policies:
    Allow:   run concurrent Jobs (default)
    Forbid:  skip new run if previous not finished
    Replace: kill previous, start new
```
[End Code Block]

Q29: YOUR PRODUCTION CLUSTER SUDDENLY HAS PODS STUCK IN PENDING STATE. WALK THRO
UGH YOUR TROUBLESHOOTING.
==========

Answer:

[Code Block]
```
Pending means the scheduler cannot place the Pod on any node.

Debugging flow:

Step 1: DESCRIBE THE POD
  kubectl describe pod <pending-pod>
  → Check Events section for scheduler messages:

  Common messages and causes:

  "Insufficient cpu" / "Insufficient memory"
    → No node has enough free resources
    → Fix: scale down other workloads, add nodes, adjust requests

  "0/5 nodes are available: 5 node(s) had taint
   {node.kubernetes.io/not-ready: }, that the pod didn't tolerate"
    → All nodes are NotReady or have taints the Pod doesn't tolerate
    → Fix: check node health, add tolerations to Pod

  "0/5 nodes are available: 2 node(s) didn't match Pod's
   node affinity/selector"
    → Pod requires specific nodes (label, zone) that don't exist or are full
```

```
    → Fix: check nodeSelector/nodeAffinity, add matching nodes

  "persistentvolumeclaim "data" not found"
    → PVC doesn't exist or isn't bound
    → Fix: create PVC, check StorageClass, check PV availability

  "no persistent volumes available for this claim and
   no storage class is set"
    → Dynamic provisioning isn't configured
    → Fix: create StorageClass or manually create PV

Step 2: CHECK NODE RESOURCES
  kubectl describe nodes | grep -A 5 "Allocated resources"
  → See how much CPU/memory is allocated vs capacity
  → If allocated ≈ capacity: cluster is full

Step 3: CHECK CLUSTER AUTOSCALER LOGS (if using)
  kubectl logs -n kube-system -l app=cluster-autoscaler
  → Is it trying to scale up?
  → Is it hitting max node count?
  → Are there cloud API errors?

Step 4: CHECK FOR PDB BLOCKING
  kubectl get pdb
  → PDB might prevent evictions needed for rescheduling

Step 5: CHECK FOR RESOURCE QUOTAS
  kubectl describe quota -n <namespace>
  → Namespace might have hit its CPU/memory/Pod count quota
[End Code Block]
```

Q30: DESIGN A PRODUCTION-READY KUBERNETES DEPLOYMENT FOR A STATELESS MICROSERVICE. WHAT SPECS WOULD YOU INCLUDE AND WHY?

==========

Answer:

```
[Code Block (yaml)]
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  labels:
    app: order-service
    version: v1
spec:
  replicas: 3                             # ← minimum for HA
  revisionHistoryLimit: 5                  # ← keep 5 old ReplicaSets for rollbac
k
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0                    # ← never kill old before new is ready
      maxSurge: 1                          # ← one extra Pod during rollout
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
```

```yaml
      app: order-service
      version: v1
  spec:
    serviceAccountName: order-service-sa  # ← least privilege SA, not default

    topologySpreadConstraints:            # ← spread across AZs
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: order-service

    containers:
      - name: order-service
        image: myregistry/order-service:v1.2.3  # ← pinned tag, never :latest
        ports:
          - containerPort: 8080

        resources:                        # ← always set both
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "1000m"
            memory: "512Mi"

        readinessProbe:                   # ← critical for zero-downtime
          httpGet:
            path: /healthz
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          failureThreshold: 3

        livenessProbe:                    # ← detect deadlocked processes
          httpGet:
            path: /healthz
            port: 8080
          initialDelaySeconds: 15
          periodSeconds: 20
          failureThreshold: 3

        startupProbe:                     # ← if app takes time to boot
          httpGet:
            path: /healthz
            port: 8080
          failureThreshold: 30
          periodSeconds: 2                # ← gives up to 60s to start

        lifecycle:
          preStop:                        # ← graceful shutdown race condition
fix
            exec:
              command: ["sleep", "5"]

        securityContext:                   # ← hardened
          runAsNonRoot: true
          runAsUser: 1000
          readOnlyRootFilesystem: true
          allowPrivilegeEscalation: false
          capabilities:
```

```
              drop: ["ALL"]

          env:
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: order-config
                  key: db_host
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: order-secrets
                  key: db_password

      terminationGracePeriodSeconds: 30     # ← time for graceful shutdown

---
apiVersion: policy/v1
kind: PodDisruptionBudget                   # ← protect during disruptions
metadata:
  name: order-service-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: order-service

---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler               # ← auto-scale on CPU
metadata:
  name: order-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```
[End Code Block]

Why each piece matters:
- maxUnavailable: 0 + readiness probe = zero-downtime deploys
- preStop sleep = handles kube-proxy iptables race condition
- topologySpreadConstraints = survive AZ failure
- PDB = safe node drains and cluster autoscaler scale-downs
- securityContext = defense in depth, non-root, read-only fs
- Pinned image tag = reproducible, no surprise latest pulls
- HPA = elastic scaling under load
- resources = proper scheduling + QoS class (Burstable)

QUICK REFERENCE: COMMANDS YOU SHOULD KNOW
==========

[Code Block (bash)]
```bash
# Cluster info
kubectl cluster-info
kubectl get nodes -o wide
kubectl top nodes / kubectl top pods

# Debugging
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl exec -it <pod> -- /bin/sh
kubectl debug -it <pod> --image=busybox

# Deployments
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl set image deployment/<name> container=image:tag

# Scaling
kubectl scale deployment/<name> --replicas=5
kubectl autoscale deployment/<name> --min=2 --max=10 --cpu-percent=60

# Resource inspection
kubectl get events --sort-by='.lastTimestamp'
kubectl get pods -o wide --all-namespaces
kubectl describe nodes | grep -A 10 "Allocated resources"

# RBAC
kubectl auth can-i create deployments --as=system:serviceaccount:staging:ci-bot
-n staging
kubectl auth can-i '*' '*' --as=<user>  # check admin access

# Drain / cordon
kubectl cordon <node>      # mark unschedulable
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>    # mark schedulable again
```
[End Code Block]

==========
FILE: K8s-Interview-Master-Guide-Story-Mode.md
==========

KUBERNETES INTERVIEW MASTER GUIDE THROUGH STORY MODE

> This is the combined Kubernetes guide that ties application lifecycle, network
ing, storage, scaling, and security into one end-to-end flow. The baseline is a
React frontend plus a Java Spring Boot backend. The goal is not to memorize rand
om objects. The goal is to understand why each Kubernetes object appears as the
application grows.

TABLE OF CONTENTS

- The App on Your Laptop (#1-the-app-on-your-laptop)
- Why Kubernetes Exists for This App (#2-why-kubernetes-exists-for-this-app)
- The Big Mental Model (#3-the-big-mental-model)
- Stage 1: Packaging the App (#4-stage-1-packaging-the-app)

- Stage 2: Pod, ReplicaSet, Deployment - The Runtime Lifecycle (#5-stage-2-pod-replicaset-deployment---the-runtime-lifecycle)
- Stage 3: Services, DNS, and Ingress - The Traffic Lifecycle (#6-stage-3-services-dns-and-ingress---the-traffic-lifecycle)
- Stage 4: ConfigMaps, Secrets, and ServiceAccounts - The Configuration and Identity Lifecycle (#7-stage-4-configmaps-secrets-and-serviceaccounts---the-configuration-and-identity-lifecycle)
- Stage 5: PV, PVC, StorageClass, and StatefulSet - The Data Lifecycle (#8-stage-5-pv-pvc-storageclass-and-statefulset---the-data-lifecycle)
- Stage 6: Probes, Requests, Limits, HPA, and Cluster Autoscaler - The Scale Lifecycle (#9-stage-6-probes-requests-limits-hpa-and-cluster-autoscaler---the-scale-lifecycle)
- Stage 7: Security - The Protection Lifecycle (#10-stage-7-security---the-protection-lifecycle)
- Stage 8: Zero-Downtime Delivery and Rollback (#11-stage-8-zero-downtime-delivery-and-rollback)
- The Full End-to-End Production Flow (#12-the-full-end-to-end-production-flow)
- How the YAML Objects Relate to Each Other (#13-how-the-yaml-objects-relate-to-each-other)
- What Happens When You Run kubectl apply (#14-what-happens-when-you-run-kubectl-apply)
- Common Interview Traps (#15-common-interview-traps)
- Interview-Ready Answers (#16-interview-ready-answers)
- Quick Revision Sheet (#17-quick-revision-sheet)

---

1. THE APP ON YOUR LAPTOP

Your local setup is usually:

[Code Block (text)]
```
Browser
  ↓
React app on localhost:3000
  ↓
Spring Boot app on localhost:8080
  ↓
PostgreSQL on localhost:5432
```
[End Code Block]

This feels easy because:

- one machine exists
- one network exists
- one disk exists
- restart means restart the process yourself
- scaling is not real yet

On your laptop, there is no meaningful distinction between lifecycle, traffic, storage, scaling, and security. In Kubernetes, they become separate concerns.

---

2. WHY KUBERNETES EXISTS FOR THIS APP

As soon as the app moves to production, five classes of problems appear.

[Code Block (text)]
```
1. Runtime problem:
```

How do I keep the app running if a container or node fails?

2. Traffic problem:
   How do users reach the app, and how do services find each other?

3. Data problem:
   How do I persist database or file data across Pod restarts?

4. Scale problem:
   How do I scale Pods and nodes under real traffic?

5. Security problem:
   How do I separate identity, secrets, permissions, and network access?
[End Code Block]

Kubernetes is valuable because it gives a declarative control plane for all five
.

---

3. THE BIG MENTAL MODEL

This is the simplest master map.

[Code Block (text)]
```
Container      = your packaged process
Pod            = one running instance of that container
ReplicaSet     = keeps Pod count correct
Deployment     = manages ReplicaSets and app rollouts
Service        = stable network identity for Pods
Ingress        = external HTTP entry into Services
ConfigMap      = non-sensitive configuration
Secret         = sensitive configuration
ServiceAccount = Pod identity for calling cluster APIs
PVC            = app request for durable storage
PV             = actual persistent storage resource
StatefulSet    = stateful controller with stable identity and storage
Probe          = health and traffic readiness signals
HPA            = scales Pod count
Cluster Autoscaler = scales node count
RBAC           = permission model
NetworkPolicy  = Pod-level traffic firewall
```
[End Code Block]

If you want the whole app in one picture:

[Code Block (text)]
```
Internet
  ↓
Ingress
  ↓
Service
  ↓
Deployment-managed Pods
  ↓
ConfigMap + Secret + ServiceAccount injected into Pods
  ↓
Pod calls database or storage through stable endpoints
  ↓
PVC/PV or managed database preserve data
  ↓
```

```
HPA scales Pods, Cluster Autoscaler scales nodes
  ↓
RBAC and NetworkPolicy constrain permissions and traffic
```
[End Code Block]

---

4. STAGE 1: PACKAGING THE APP

Before Kubernetes can run anything, the application must be packaged.

For Spring Boot, that usually means a Docker image.

Real-life analogy:

[Code Block (text)]
```
Your source code is the recipe.
The container image is the sealed lunch box.
Kubernetes only runs lunch boxes, not raw ingredients.
```
[End Code Block]

Typical flow:

[Code Block (text)]
```
code → JAR → container image → registry → Kubernetes pulls image
```
[End Code Block]

This matters because every later object points to the image as its executable artifact.

---

5. STAGE 2: POD, REPLICASET, DEPLOYMENT - THE RUNTIME LIFECYCLE

5.1 POD

Pod is the smallest runnable unit.

For most Spring Boot services:

[Code Block (text)]
```
1 Pod = 1 Spring Boot container
```
[End Code Block]

But a Pod alone is not enough for real application management.

5.2 REPLICASET

ReplicaSet says:

[Code Block (text)]
```
"I need exactly N copies of this Pod running."
```
[End Code Block]

If one Pod dies, ReplicaSet replaces it.

5.3 DEPLOYMENT

Deployment manages ReplicaSets and adds:

- rolling updates
- rollback
- revision history
- declarative scaling

Real-life analogy:

[Code Block (text)]
```
Pod         = worker
ReplicaSet  = shift manager
Deployment  = operations manager
```
[End Code Block]

For stateless Spring Boot APIs, Deployment is the normal controller.

5.4 THE KEY INTERVIEW DISTINCTION

[Code Block (text)]
```
Pod runs the app.
ReplicaSet keeps the number of Pods correct.
Deployment manages updates and rollback by controlling ReplicaSets.
```
[End Code Block]

---

6. STAGE 3: SERVICES, DNS, AND INGRESS - THE TRAFFIC LIFECYCLE

6.1 WHY SERVICES EXIST

Pods are ephemeral.
Pod IPs change.
Traffic cannot safely target Pod IPs directly.

So Kubernetes introduces a Service.

Service provides:

- stable virtual IP
- stable DNS name
- load balancing across selected healthy Pods

For example:

[Code Block (text)]
```
frontend Pod → http://backend-service
```
[End Code Block]

not:

[Code Block (text)]
```
frontend Pod → 10.244.2.8
```
[End Code Block]

6.2 DNS

CoreDNS resolves names like:

[Code Block (text)]
```
backend-service
postgres-service
redis-service
```
[End Code Block]

That is how Spring Boot should find internal dependencies.

Example:

[Code Block (properties)]
```
inventory.base-url=http://inventory-service:8080
spring.datasource.url=jdbc:postgresql://postgres-service:5432/appdb
```
[End Code Block]

6.3 INGRESS

Ingress handles external HTTP entry.

Flow:

[Code Block (text)]
```
Browser
  ↓
Ingress Controller
  ↓
Service
  ↓
Pod
```
[End Code Block]

Important distinction:

[Code Block (text)]
```
Ingress routes to Services.
Services route to Pods.
```
[End Code Block]

6.4 EGRESS

When Spring Boot calls Stripe, Twilio, or some external API, that is egress.

[Code Block (text)]
```
Pod → external destination
```
[End Code Block]

Egress is a traffic direction, not usually one single Kubernetes resource like Ingress.

6.5 THE INTERVIEW SUMMARY

[Code Block (text)]
```
Ingress = outside to inside
Service = stable internal access to Pods
DNS = name resolution for Services and Pods
Egress = inside to outside
```
[End Code Block]

---

7. STAGE 4: CONFIGMAPS, SECRETS, AND SERVICEACCOUNTS - THE CONFIGURATION AND IDENTITY LIFECYCLE

7.1 CONFIGMAPS

ConfigMap stores non-sensitive config.

Examples:

- environment-specific URLs
- log level
- feature flags
- Spring profiles

7.2 SECRETS

Secret stores sensitive values.

Examples:

- DB password
- JWT key
- API tokens
- TLS data

Important interview nuance:

[Code Block (text)]
```
Secrets are base64 encoded, not strongly encrypted by default.
You should enable etcd encryption at rest and often use an external secret manager.
```
[End Code Block]

7.3 SERVICEACCOUNT

ServiceAccount is the identity of the Pod when it talks to the Kubernetes API.

Important point:

[Code Block (text)]
```
Do not let production Pods casually use the default ServiceAccount.
Use dedicated ServiceAccounts with least privilege.
```
[End Code Block]

7.4 WHY THESE BELONG TOGETHER

Because when a Pod starts, it needs:

- configuration
- secrets
- identity

These are different concerns and should not be hardcoded into the image.

---

8. STAGE 5: PV, PVC, STORAGECLASS, AND STATEFULSET - THE DATA LIFECYCLE

8.1 WHY DATA CHANGES EVERYTHING

Stateless Pods can die freely.
Stateful workloads cannot.

Container filesystems are not durable storage contracts.

8.2 PV, PVC, AND STORAGECLASS

[Code Block (text)]
```text
PV           = actual durable storage resource
PVC          = application request for that storage
StorageClass = defines how the storage gets provisioned
```
[End Code Block]

Flow:

[Code Block (text)]
```text
Pod mounts PVC
PVC binds PV
PV points to actual cloud or on-prem storage
```
[End Code Block]

8.3 DEPLOYMENT VS STATEFULSET

Use Deployment when replicas are interchangeable.

Use StatefulSet when replicas need:

- stable identity
- stable per-Pod storage
- ordered startup/shutdown
- deterministic peer discovery

8.4 THE SPRING BOOT DISTINCTION

This is one of the most important interview points:

[Code Block (text)]
```text
Spring Boot API with external PostgreSQL is usually still a Deployment.
The database itself may be a StatefulSet or a managed external service.
```
[End Code Block]

Do not convert every app that touches a database into a StatefulSet.

8.5 HEADLESS SERVICE

StatefulSets often pair with a Headless Service so Pods get stable DNS names lik
e:

[Code Block (text)]
```text
postgres-0.postgres-headless
```

```
postgres-1.postgres-headless
postgres-2.postgres-headless
```
[End Code Block]

---

9. STAGE 6: PROBES, REQUESTS, LIMITS, HPA, AND CLUSTER AUTOSCALER - THE SCALE LIFECYCLE

9.1 PROBES

There are three probes.

[Code Block (text)]
```
Startup   = has the app finished booting?
Readiness = should it receive traffic now?
Liveness  = is it alive or should kubelet restart it?
```
[End Code Block]

For Spring Boot, startup and readiness are especially important because JVM apps start slower than many lighter runtimes.

9.2 REQUESTS AND LIMITS

Requests are used by the scheduler.
Limits are enforced at runtime.

Critical distinction:

[Code Block (text)]
```
CPU overuse     → throttling
Memory overuse  → OOMKilled
```
[End Code Block]

9.3 HPA

HPA scales Pod count based on metrics.

Flow:

[Code Block (text)]
```
traffic rises
  ↓
CPU or custom metric rises
  ↓
HPA increases Deployment replicas
  ↓
ReplicaSet creates more Pods
```
[End Code Block]

Important requirement:

[Code Block (text)]
```
HPA needs resource requests to calculate utilization correctly.
```
[End Code Block]

9.4 CLUSTER AUTOSCALER

If HPA creates more Pods but there is no room on nodes:

[Code Block (text)]
```text
Pods stay Pending
  ↓
Cluster Autoscaler adds nodes
  ↓
new nodes join cluster
  ↓
Pending Pods get scheduled
```
[End Code Block]

9.5 THE SCALE STORY IN ONE LINE

[Code Block (text)]
```text
HPA scales Pods. Cluster Autoscaler scales nodes.
```
[End Code Block]

---

10. STAGE 7: SECURITY - THE PROTECTION LIFECYCLE

Security in Kubernetes is layered.

10.1 RBAC

RBAC controls who can do what on which resources.

Objects:

- Role
- ClusterRole
- RoleBinding
- ClusterRoleBinding

This is the permission system for users, service accounts, and controllers.

10.2 POD SECURITY

At the Pod level, you harden containers using securityContext.

Typical production settings:

- run as non-root
- drop Linux capabilities
- read-only root filesystem
- disallow privilege escalation

10.3 NETWORKPOLICY

NetworkPolicy is the Pod-level firewall.

It restricts:

- ingress traffic into Pods
- egress traffic out of Pods

Important distinction:

[Code Block (text)]
Ingress resource handles external HTTP routing.
NetworkPolicy ingress handles Pod-level traffic permission.
[End Code Block]

These are not the same thing.

10.4 SECRETS SECURITY

Secrets should not be treated as magically secure just because they are named Secret.

Hardening means:

- encryption at rest
- least privilege RBAC
- careful mounting method
- external secret managers when possible

---

11. STAGE 8: ZERO-DOWNTIME DELIVERY AND ROLLBACK

A production-ready rollout needs more than changing the image tag.

11.1 ROLLING UPDATE

Key settings:

[Code Block (text)]
maxUnavailable: 0
maxSurge: 1
[End Code Block]

That means new Pods come up before old Pods are removed.

11.2 WHY READINESS MATTERS

Without readiness probes, Kubernetes may route traffic to a Pod too early.

So the true zero-downtime pattern is:

[Code Block (text)]
rolling update + readiness probe + graceful shutdown + preStop + PDB
[End Code Block]

11.3 ROLLBACK

Deployments keep old ReplicaSets.
Rollback means scaling the healthy old revision back up.

[Code Block (bash)]
kubectl rollout undo deployment/order-service
[End Code Block]

---

12. THE FULL END-TO-END PRODUCTION FLOW

This is the combined story.

[Code Block (text)]
```
1. Developer pushes code.
2. CI builds the Spring Boot JAR and container image.
3. CD updates Deployment image tag in Kubernetes.
4. API server stores the new desired state.
5. Deployment creates a new ReplicaSet.
6. ReplicaSet creates Pods.
7. Scheduler places Pods on nodes using requests and constraints.
8. kubelet pulls image, mounts ConfigMaps, Secrets, and PVCs, and starts contain
ers.
9. Startup probe protects boot phase.
10. Readiness probe passes, so Service includes the Pod in endpoints.
11. Ingress sends user traffic to the Service, which routes to healthy Pods.
12. Spring Boot calls internal dependencies via Service DNS names.
13. Spring Boot calls external systems via egress path.
14. HPA scales Pods if metrics rise.
15. Cluster Autoscaler adds nodes if Pods cannot be scheduled.
16. Stateful workloads preserve identity and storage via StatefulSet + PVC/PV.
17. RBAC, ServiceAccounts, Secrets, Pod security, and NetworkPolicies enforce se
curity.
18. If the new version fails readiness, rollout stalls and rollback remains poss
ible.
```
[End Code Block]

That is the entire Kubernetes application story in one flow.

---

13. HOW THE YAML OBJECTS RELATE TO EACH OTHER

This relationship map is the bridge between theory and manifests.

[Code Block (text)]
```
Namespace
  ├─ ServiceAccount
  ├─ ConfigMap
  ├─ Secret
  ├─ Deployment
  |    └─ Pod template
  |         ├─ containers
  |         ├─ env from ConfigMap/Secret
  |         ├─ probes
  |         ├─ resources
  |         └─ serviceAccountName
  ├─ Service
  |    └─ selects Pods by labels from Deployment template
  ├─ Ingress
  |    └─ routes external traffic to Service
  ├─ HPA
  |    └─ targets Deployment
  ├─ NetworkPolicy
  |    └─ governs traffic to/from selected Pods
  ├─ PVC
```

```
  |    └─ binds to PV
  └─ StatefulSet
       ├─ Pod template
       ├─ Headless Service for stable DNS
       └─ volumeClaimTemplates create one PVC per Pod
```
[End Code Block]

If you understand that map, most Kubernetes YAML becomes much easier to reason a
bout.

---

14. WHAT HAPPENS WHEN YOU RUN KUBECTL APPLY

This is the internal control-plane story.

[Code Block (text)]
```
kubectl sends desired state to kube-apiserver
  ↓
API server authenticates and authorizes the request
  ↓
admission controllers validate or mutate it
  ↓
object is stored in etcd
  ↓
controllers observe the change
  ↓
Deployment controller creates/updates ReplicaSet
  ↓
ReplicaSet controller creates Pods
  ↓
Scheduler assigns Pods to nodes
  ↓
kubelet starts containers and mounts storage/config
  ↓
Service endpoints update when readiness passes
  ↓
Ingress and Service routing start sending traffic
```
[End Code Block]

The senior-level principle is:

[Code Block (text)]
```
Kubernetes is declarative and reconciliation-driven.
You declare desired state. Controllers converge actual state toward it.
```
[End Code Block]

---

15. COMMON INTERVIEW TRAPS

15.1 "EVERY APP WITH A DB SHOULD BE A STATEFULSET"

Wrong.

[Code Block (text)]
```
Usually the API is a Deployment.
The database may be a StatefulSet or managed external service.
```
[End Code Block]

15.2 "INGRESS AND NETWORKPOLICY INGRESS ARE THE SAME"

Wrong.

[Code Block (text)]
```
Ingress resource = external HTTP routing
NetworkPolicy ingress = Pod firewall rule
```
[End Code Block]

15.3 "SECRETS ARE ENCRYPTED BY DEFAULT"

Wrong.

They are base64 encoded unless stronger controls are enabled.

15.4 "HPA ALONE SOLVES SCALING"

Wrong.

If nodes are full, HPA can create Pending Pods. Cluster Autoscaler may be needed.

15.5 "SERVICE IS A PROCESS"

Wrong.

Service is an abstraction backed by endpoint lists and kube-proxy rules.

---

16. INTERVIEW-READY ANSWERS

16.1 "WALK ME THROUGH A PRODUCTION SPRING BOOT APP ON KUBERNETES"

[Code Block (text)]
```
I package the app as a container image and run it using a Deployment because the
 API itself is stateless. The Deployment manages ReplicaSets, which maintain the
 desired number of Pods. A Service provides stable internal access to those Pods
, and Ingress exposes the application externally. ConfigMaps and Secrets provide
 configuration and sensitive values, while a dedicated ServiceAccount gives the
Pod identity. Health probes control startup, readiness, and liveness. Resource r
equests and limits define scheduling and runtime boundaries. HPA scales Pods, an
d Cluster Autoscaler scales nodes if needed. For databases or other stateful sys
tems, I use managed services or StatefulSets with PVCs and PVs. RBAC, Pod securi
ty, Secrets handling, and NetworkPolicies enforce security.
```
[End Code Block]

16.2 "HOW DO ALL THE MAIN K8S OBJECTS RELATE TO EACH OTHER?"

[Code Block (text)]
```
Deployment manages ReplicaSets and Pods. Service selects those Pods by labels an
d gives them stable access. Ingress routes external traffic to Services. ConfigM
aps and Secrets inject configuration into Pods. ServiceAccounts give Pods identi
ty. HPA targets a Deployment to adjust replica count. PVCs attach durable storag
```

e to Pods and bind to PVs. StatefulSet is used instead of Deployment when Pods n
eed stable identity and per-Pod storage.
[End Code Block]

16.3 "WHAT ARE THE TOP THINGS THAT MAKE A K8S SETUP PRODUCTION-READY?"

[Code Block (text)]
Health probes, resource requests and limits, rolling update strategy, graceful s
hutdown, PDBs, least-privilege ServiceAccounts, Secrets handling, NetworkPolicie
s, observability, and a clear scaling model with HPA plus node autoscaling where
 needed.
[End Code Block]

---

17. QUICK REVISION SHEET

THE CORE CHAIN

[Code Block (text)]
Image → Pod → ReplicaSet → Deployment
Client → Ingress → Service → Pod
Pod → PVC → PV
HPA → Deployment
NetworkPolicy → selected Pods
RBAC → identities and resource permissions
[End Code Block]

THE DECISION RULES

[Code Block (text)]
Stateless app → Deployment
Stateful clustered system → StatefulSet
Internal stable access → Service
External HTTP entry → Ingress
Durable storage request → PVC
Runtime scale → HPA
Node scale → Cluster Autoscaler
Pod identity to K8s API → ServiceAccount
Permission model → RBAC
Pod traffic firewall → NetworkPolicy
[End Code Block]

GOLD STANDARD SUMMARY

[Code Block (text)]
Kubernetes manages an application through connected control loops. Deployments m
anage stateless application rollout and scaling through ReplicaSets and Pods. Se
rvices and Ingress manage traffic into and within the cluster. ConfigMaps, Secre
ts, and ServiceAccounts provide configuration and identity. PVCs, PVs, StorageCl
asses, and StatefulSets handle durable state when needed. Probes, resource reque
sts, limits, HPA, and Cluster Autoscaler manage health and scale. RBAC, Pod secu
rity, Secrets handling, and NetworkPolicies secure the system. Understanding how
 these pieces connect is what turns Kubernetes from a list of objects into one c
oherent production platform.
[End Code Block]

==========
FILE: K8s-Networking-Traffic-Flow-Story-Mode.md
==========

KUBERNETES NETWORKING AND TRAFFIC FLOW THROUGH STORY MODE

> You have a React frontend and a Spring Boot backend. On your laptop, they talk through localhost and life is simple. In Kubernetes, the app still does the same business work, but the traffic flow becomes far more structured. This guide explains Services, Ingress, Egress, DNS, Network Policies, and how all of them relate to a Spring Boot application in production.

TABLE OF CONTENTS

- How Networking Works on Your Laptop (#1-how-networking-works-on-your-laptop)
- What Changes in Kubernetes (#2-what-changes-in-kubernetes)
- The Big Mental Model: Ingress, East-West, Egress (#3-the-big-mental-model-ingress-east-west-egress)
- The Foundation: Pod Networking (#4-the-foundation-pod-networking)
- Services - Stable Identity for Moving Pods (#5-services---stable-identity-for-moving-pods)
- The Four Service Types and When to Use Them (#6-the-four-service-types-and-when-to-use-them)
- How kube-proxy and Endpoints Make Services Work (#7-how-kube-proxy-and-endpoints-make-services-work)
- DNS and CoreDNS - How Spring Boot Finds Other Services (#8-dns-and-coredns---how-spring-boot-finds-other-services)
- Ingress - How External Traffic Enters the App (#9-ingress---how-external-traffic-enters-the-app)
- Egress - How Your Spring Boot App Calls the Outside World (#10-egress---how-your-spring-boot-app-calls-the-outside-world)
- Network Policies - Who Can Talk to Whom (#11-network-policies---who-can-talk-to-whom)
- The Relationship Between Services, Ingress, and Egress (#12-the-relationship-between-services-ingress-and-egress)
- Spring Boot App Traffic Stories (#13-spring-boot-app-traffic-stories)
- Spring Boot Configuration Examples (#14-spring-boot-configuration-examples)
- Common Mistakes and Troubleshooting (#15-common-mistakes-and-troubleshooting)
- Interview-Ready Answers (#16-interview-ready-answers)
- Quick Revision Sheet (#17-quick-revision-sheet)

---

1. HOW NETWORKING WORKS ON YOUR LAPTOP

On your laptop, your app usually looks like this:

[Code Block (text)]
```
Browser
  ↓
React app on localhost:3000
  ↓
Spring Boot API on localhost:8080
  ↓
PostgreSQL on localhost:5432
```
[End Code Block]

Maybe your React app calls:

[Code Block (text)]
http://localhost:8080/api/orders
[End Code Block]

And your Spring Boot app calls:

[Code Block (text)]
jdbc:postgresql://localhost:5432/appdb
[End Code Block]

That works because:

- everything is on one machine
- IP addressing is trivial
- service discovery does not exist because you already know every host and port
- incoming and outgoing traffic all look the same from the app's point of view

Production Kubernetes is different.

---

2. WHAT CHANGES IN KUBERNETES

In Kubernetes:

- your backend runs in Pods
- Pods can die and come back
- Pod IPs change
- multiple replicas exist
- traffic enters from outside the cluster
- traffic moves between services inside the cluster
- traffic leaves the cluster to call external systems

So networking now has three different questions:

[Code Block (text)]
1. How does external traffic enter the cluster?
2. How do services find each other inside the cluster?
3. How does a Pod call systems outside the cluster?
[End Code Block]

Those map to:

[Code Block (text)]
External entry   → Ingress
Internal routing   → Services + DNS
External outbound → Egress
[End Code Block]

---

3. THE BIG MENTAL MODEL: INGRESS, EAST-WEST, EGRESS

This is the most important networking picture to remember.

[Code Block (text)]
North-South traffic = traffic entering or leaving the cluster boundary

East-West traffic   = traffic between services inside the cluster
[End Code Block]

For your Spring Boot app:

[Code Block (text)]
Internet user → Ingress → Service → Spring Boot Pod   = Ingress path
Spring Boot Pod → user-service / payment-service   = East-West path
Spring Boot Pod → Stripe / Twilio / external DB / SaaS   = Egress path
[End Code Block]

Another way to visualize it:

[Code Block (text)]
```
                INGRESS
Internet  ------------------>  Ingress Controller  --> Service --> Pod

                EAST-WEST
Pod A     ------------------>  Service B          --> Pod B

                EGRESS
Pod A     ------------------>  External API / Internet / Managed DB
```
[End Code Block]

Critical clarification:

[Code Block (text)]
Ingress is a Kubernetes resource.
Service is a Kubernetes resource.
Egress is NOT a single Kubernetes resource type.
Egress is a traffic direction and is usually controlled using Network Policies, cloud networking, NAT, firewalls, or service mesh gateways.
[End Code Block]

That distinction matters in interviews.

---

4. THE FOUNDATION: POD NETWORKING

Before Services and Ingress make sense, you need the base model.

4.1 THE THREE CORE RULES

Kubernetes networking assumes:

[Code Block (text)]
1. Every Pod gets its own IP address.
2. Every Pod can reach every other Pod by IP by default.
3. No NAT is needed for Pod-to-Pod traffic inside the cluster.
[End Code Block]

Real-life analogy:

[Code Block (text)]
Every Pod is like a person with their own direct phone number.
They do not need to share one office phone.
[End Code Block]

4.2 CNI - THE ACTUAL NETWORK ENGINE

==========

Kubernetes itself does not implement Pod networking directly.
That is done by the CNI plugin.

Common CNIs:

- Calico
- Cilium
- Flannel
- Weave

Why this matters:

- CNI gives Pods IPs
- CNI connects Pods across nodes
- some CNIs enforce Network Policies

Important interview detail:

[Code Block (text)]
Network Policies only work if the CNI supports them.
[End Code Block]

---

5. SERVICES - STABLE IDENTITY FOR MOVING PODS

5.1 THE STORY

Your Spring Boot Deployment has 3 Pods.

[Code Block (text)]
order-service-abc123 → 10.244.1.5
order-service-def456 → 10.244.2.8
order-service-ghi789 → 10.244.3.3
[End Code Block]

Then one Pod dies and is recreated:

[Code Block (text)]
order-service-new111 → 10.244.4.2
[End Code Block]

Problem:

[Code Block (text)]
How will the frontend or another service keep track of these changing Pod IPs?
[End Code Block]

Answer: Service.

5.2 REAL-LIFE ANALOGY

[Code Block (text)]
Pods are hotel guests. They can change rooms.
Service is the hotel reception number.

You do not try to memorize every guest's room number.

You call reception and let it route you correctly.
[End Code Block]

5.3 WHAT A SERVICE REALLY GIVES YOU

A Service provides:

- a stable virtual IP
- a stable DNS name
- load balancing across matching Pods

For example:

[Code Block (text)]
```
order-service.default.svc.cluster.local
```
[End Code Block]

or, within the same namespace, just:

[Code Block (text)]
```
http://order-service
```
[End Code Block]

5.4 SPRING BOOT VIEW OF A SERVICE

Your Spring Boot app does not care which Pod it is calling.
It should call the Service name.

Example:

[Code Block (properties)]
```
user.service.base-url=http://user-service:8080
```
[End Code Block]

From Java code:

[Code Block (java)]
```
String url = userServiceBaseUrl + "/api/users/" + userId;
```
[End Code Block]

That is the correct Kubernetes pattern.

---

6. THE FOUR SERVICE TYPES AND WHEN TO USE THEM

6.1 CLUSTERIP

This is the default and the most common.

[Code Block (text)]
```
Used for internal communication only.
Frontend Pod → backend Service
Backend Pod  → database Service
Backend Pod  → Redis Service
```
[End Code Block]

For most Spring Boot microservice calls, this is the normal choice.

Example:

[Code Block (yaml)]
```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```
[End Code Block]

6.2 NODEPORT

[Code Block (text)]
```text
Exposes the Service on a port on every node.
Useful for quick testing.
Usually not preferred for production internet entry.
```
[End Code Block]

Think of it as:

[Code Block (text)]
```text
any-node-ip:nodePort → Service → Pod
```
[End Code Block]

6.3 LOADBALANCER

[Code Block (text)]
```text
Creates a cloud load balancer in EKS/GKE/AKS.
Used when you want external traffic to reach one Service directly.
```
[End Code Block]

This is good for simple cases, but if you have many HTTP services, Ingress is usually more efficient.

6.4 HEADLESS SERVICE

[Code Block (text)]
```text
No ClusterIP.
DNS returns Pod IPs directly.
Mainly used with StatefulSets.
```
[End Code Block]

This matters for databases like Kafka, PostgreSQL clusters, or systems where each Pod has a stable identity.

6.5 THE INTERVIEW SUMMARY

[Code Block (text)]
```text
ClusterIP   = internal service-to-service traffic
NodePort    = simple external access via node port
LoadBalancer= direct cloud LB for one service
```

Headless   = direct Pod discovery, mainly for stateful apps
[End Code Block]

---

7. HOW KUBE-PROXY AND ENDPOINTS MAKE SERVICES WORK

7.1 WHAT MANY PEOPLE MISS

A Service is not a process.
It is an abstraction.

The actual routing happens because Kubernetes maintains endpoint lists and insta
lls network rules on nodes.

7.2 THE FLOW

[Code Block (text)]
Service selector matches Pods
      ↓
Kubernetes builds endpoint list for that Service
      ↓
kube-proxy programs iptables or IPVS rules
      ↓
traffic to the Service virtual IP gets forwarded to one of the Pods
[End Code Block]

So when Spring Boot calls:

[Code Block (text)]
http://payment-service
[End Code Block]

it does not magically call one Pod.
It hits the Service VIP, and kube-proxy forwards it to a healthy backend Pod.

7.3 READINESS AND ENDPOINTS

This is critical:

[Code Block (text)]
If a Pod fails readiness probe, it is removed from Service endpoints.
The Pod may still be running, but it receives no traffic.
[End Code Block]

This is why readiness is fundamentally a networking concept, not just a health-c
heck concept.

---

8. DNS AND COREDNS - HOW SPRING BOOT FINDS OTHER SERVICES

8.1 THE STORY

Your Spring Boot service needs to call:

- user-service
- inventory-service
- redis-service

How does it resolve those names?

Answer: CoreDNS.

8.2 REAL-LIFE ANALOGY

[Code Block (text)]
```text
CoreDNS is the cluster phone directory.
Instead of remembering IP addresses, services ask the directory for names.
```
[End Code Block]

8.3 COMMON DNS PATTERNS

Within same namespace:

[Code Block (text)]
```text
http://order-service
```
[End Code Block]

Cross namespace:

[Code Block (text)]
```text
http://order-service.payments
```
[End Code Block]

Full DNS form:

[Code Block (text)]
```text
order-service.payments.svc.cluster.local
```
[End Code Block]

8.4 WHY THIS MATTERS FOR SPRING BOOT

Your config should point to Kubernetes service names, not Pod IPs.

Examples:

[Code Block (properties)]
```properties
spring.datasource.url=jdbc:postgresql://postgres-service:5432/appdb
redis.host=redis-service
inventory.base-url=http://inventory-service:8080
```
[End Code Block]

That is the clean production-friendly setup.

8.5 COMMON DNS FAILURE PATTERN

If service resolution fails, check:

- Service name correct?
- same namespace or cross-namespace?
- CoreDNS healthy?
- typo in config?

---

9. INGRESS - HOW EXTERNAL TRAFFIC ENTERS THE APP

9.1 THE STORY

You now want users on the internet to reach the app.

You have:

- frontend Service
- order Service
- user Service

You do not want one cloud load balancer per service.

So you introduce Ingress.

9.2 REAL-LIFE ANALOGY

[Code Block (text)]
```text
Ingress is the receptionist at the office entrance.

If a visitor asks for /api/orders, send them to the backend.
If a visitor asks for /, send them to the frontend.
```
[End Code Block]

9.3 WHAT INGRESS ACTUALLY DOES

Ingress is an HTTP routing rule.
It routes based on:

- host
- path
- sometimes TLS termination and other L7 features depending on the controller

Important:

[Code Block (text)]
```text
Ingress does not send traffic directly to Pods.
Ingress routes to Services.
Services route to Pods.
```
[End Code Block]

That relationship is core.

9.4 YOU ALSO NEED AN INGRESS CONTROLLER

Ingress resource alone is just configuration.
You need an engine to implement it.

Common choices:

- NGINX Ingress Controller
- Traefik
- AWS Load Balancer Controller

In cloud environments, the Ingress Controller itself is often exposed through a Service of type LoadBalancer.

That means the practical chain is often:

[Code Block (text)]
```text
Internet
  ↓
Cloud Load Balancer
  ↓
Ingress Controller
  ↓
Service
  ↓
Pod
```
[End Code Block]

9.5 INGRESS EXAMPLE FOR REACT + SPRING BOOT

[Code Block (yaml)]
```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
                port:
                  number: 80
```
[End Code Block]

Here the frontend and backend are both reachable under one hostname.

9.6 WHY INGRESS MATTERS FOR SPRING BOOT

Without Ingress, your browser may call backend separately via another public end point.
With Ingress, you can often keep one domain and route neatly.

That helps with:

- simpler DNS
- simpler TLS
- simpler browser routing
- better production architecture

---

10. EGRESS - HOW YOUR SPRING BOOT APP CALLS THE OUTSIDE WORLD

10.1 THE STORY

Your Spring Boot app needs to call:

- Stripe payment API
- Twilio SMS API
- external OAuth provider
- managed database outside the cluster
- S3 or other cloud APIs

That outbound traffic is Egress.

10.2 THE IMPORTANT CLARIFICATION

Unlike Ingress, Kubernetes does not have a built-in resource called Egress for n
ormal app routing.

Egress means:

[Code Block (text)]
traffic leaving a Pod toward destinations outside its allowed internal scope
[End Code Block]

It is controlled through combinations of:

- Network Policies with egress rules
- cloud NAT gateways or firewalls
- proxies
- service mesh egress gateways
- corporate network controls

10.3 REAL-LIFE ANALOGY

[Code Block (text)]
Ingress is who can enter your office building.
Egress is where employees are allowed to go outside the building.
[End Code Block]

10.4 TYPICAL SPRING BOOT EGRESS EXAMPLES

[Code Block (text)]
backend Pod → api.stripe.com:443
backend Pod → sms-provider.example.com:443
backend Pod → external-postgres.company.net:5432
[End Code Block]

10.5 EGRESS AND SECURITY

If you do nothing, many clusters allow broad outbound access by default.

In mature production systems, teams often restrict it:

- allow only required destinations
- deny unnecessary outbound internet access
- force traffic through proxy or egress gateway

This is where Network Policies matter.

---

11. NETWORK POLICIES - WHO CAN TALK TO WHOM

11.1 THE STORY

Your React frontend should call the backend.
But some random Pod should not call the backend.

Your backend should call Stripe.
But it should not freely talk to every Pod and every external endpoint.

That is the job of NetworkPolicy.

11.2 INGRESS RULES IN NETWORKPOLICY

These control who may send traffic into selected Pods.

Example:

[Code Block (yaml)]
```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-allow-frontend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```
[End Code Block]

Meaning:

[Code Block (text)]
```text
frontend Pods may call backend Pods on 8080
others may not
```
[End Code Block]

11.3 EGRESS RULES IN NETWORKPOLICY

These control where selected Pods may send traffic outward.

Example:

[Code Block (yaml)]
```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-egress-policy
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: data
      ports:
        - protocol: TCP
          port: 5432
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
      ports:
        - protocol: TCP
          port: 443
```
[End Code Block]

Meaning:

[Code Block (text)]
```text
backend may talk to:
  - DB in the data namespace on 5432
  - external HTTPS destinations on 443
```
[End Code Block]

11.4 IMPORTANT INTERVIEW CLARIFICATION

[Code Block (text)]
```text
Ingress resource and NetworkPolicy ingress are not the same thing.

Ingress resource:
  external HTTP routing into Services

NetworkPolicy ingress:
  firewall rule controlling which Pods can send traffic to selected Pods
```
[End Code Block]

That difference is extremely important.

---

12. THE RELATIONSHIP BETWEEN SERVICES, INGRESS, AND EGRESS

This is the exact relationship you asked for.

12.1 THE CLEAN MODEL

[Code Block (text)]
```
Ingress handles incoming HTTP traffic from outside the cluster.
Service handles stable internal routing to Pods.
Egress handles outbound traffic from Pods to the outside world.
```
[End Code Block]

12.2 THEY ARE NOT PEERS IN THE SAME LAYER

They solve different parts of the path:

[Code Block (text)]
```
Internet request path:
  Client → Ingress → Service → Pod

Internal request path:
  Pod A → Service B → Pod B

Outbound request path:
  Pod A → external destination
```
[End Code Block]

12.3 THE MOST IMPORTANT RELATIONSHIP STATEMENT

[Code Block (text)]
```
Ingress usually depends on Services.
Services do not depend on Ingress.
Egress is separate from both and describes outbound traffic behavior from Pods.
```
[End Code Block]

12.4 ANOTHER USEFUL MENTAL MODEL

[Code Block (text)]
```
Ingress = front door
Service = internal reception desk
Egress = exit gate
```
[End Code Block]

---

13. SPRING BOOT APP TRAFFIC STORIES

13.1 STORY 1: BROWSER REACHES THE SPRING BOOT API

You open:

[Code Block (text)]
```
https://app.example.com/api/orders
```
[End Code Block]

Traffic flow:

[Code Block (text)]
```
Browser
  ↓
DNS resolves app.example.com
  ↓
Cloud Load Balancer / Ingress Controller entry
```

```
  ↓
Ingress rule matches /api
  ↓
backend-service
  ↓
one healthy backend Pod
  ↓
Spring Boot handles request
```
[End Code Block]

Why Service is needed here:

- Ingress routes to Service, not directly to Pods
- Service hides Pod churn
- readiness ensures only healthy Pods receive traffic

13.2 STORY 2: SPRING BOOT CALLS ANOTHER INTERNAL SERVICE

Order service needs user profile details.

Traffic flow:

[Code Block (text)]
```
order-service Pod
  ↓
HTTP call to http://user-service:8080/api/users/123
  ↓
CoreDNS resolves user-service
  ↓
user-service ClusterIP
  ↓
kube-proxy forwards to one user-service Pod
  ↓
response comes back
```
[End Code Block]

This is classic east-west traffic.

13.3 STORY 3: SPRING BOOT CALLS STRIPE

Order service processes payment.

Traffic flow:

[Code Block (text)]
```
order-service Pod
  ↓
HTTPS call to api.stripe.com
  ↓
egress path from cluster/network
  ↓
Stripe API
  ↓
response comes back
```
[End Code Block]

This is not handled by a Kubernetes Service.
It is outbound egress.

13.4 STORY 4: FRONTEND TO BACKEND IN ONE DOMAIN

==========

You want:

[Code Block (text)]
```
https://app.example.com/      → frontend
https://app.example.com/api   → backend
```
[End Code Block]

Ingress makes that clean.

This is often the simplest browser-friendly architecture.

---

14. SPRING BOOT CONFIGURATION EXAMPLES

14.1 INTERNAL SERVICE-TO-SERVICE URLS

[Code Block (properties)]
```
user.service.base-url=http://user-service:8080
inventory.service.base-url=http://inventory-service:8080
spring.datasource.url=jdbc:postgresql://postgres-service:5432/appdb
```
[End Code Block]

14.2 CROSS-NAMESPACE EXAMPLE

[Code Block (properties)]
```
payment.service.base-url=http://payment-service.payments:8080
```
[End Code Block]

14.3 EXTERNAL EGRESS EXAMPLE

[Code Block (properties)]
```
stripe.base-url=https://api.stripe.com
sms.base-url=https://api.twilio.com
```
[End Code Block]

14.4 SPRING BOOT CODE EXAMPLE

[Code Block (java)]
```
@Service
public class PaymentClient {

    @Value("${stripe.base-url}")
    private String stripeBaseUrl;

    private final RestTemplate restTemplate;

    public PaymentClient(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    public String charge(Object request) {
        return restTemplate.postForObject(stripeBaseUrl + "/v1/charges", request
, String.class);
    }
```

}
[End Code Block]

14.5 CORS REMINDER FOR BROWSER TRAFFIC

If frontend and backend are served from different origins, browser CORS rules apply.

If both are behind the same Ingress host, you can often reduce CORS complexity significantly.

For example:

[Code Block (text)]
```
app.example.com       → frontend
app.example.com/api   → backend
```
[End Code Block]

This is cleaner than:

[Code Block (text)]
```
frontend.example.com
api.example.com
```
[End Code Block]

though both are valid architectures.

---

15. COMMON MISTAKES AND TROUBLESHOOTING

15.1 "I USED POD IPS IN CONFIG"

[Code Block (text)]
```
Wrong: Pod IPs are ephemeral.
Correct: use Service DNS names.
```
[End Code Block]

15.2 "MY SERVICE EXISTS BUT TRAFFIC IS NOT REACHING PODS"

Check:

[Code Block (text)]
```
1. Does Service selector match Pod labels?
2. Are Pods ready?
3. Does kubectl get endpoints <service> show Pod IPs?
```
[End Code Block]

15.3 "INGRESS EXISTS BUT NOTHING WORKS"

Check:

- Is an Ingress Controller installed?
- Is DNS pointing to the controller or load balancer?
- Are backend Service names and ports correct?
- Is TLS configured correctly if using HTTPS?

15.4 "EGRESS TO EXTERNAL API IS FAILING"

Check:

- DNS resolution working?
- outbound firewall/NAT rules okay?
- NetworkPolicy egress blocking it?
- proxy required in enterprise environment?

15.5 "INTERNAL SERVICE CALL FAILS"

Check:

[Code Block (text)]
```
1. Service name correct?
2. Namespace correct?
3. Target container listening on expected port?
4. Readiness passing?
5. NetworkPolicy blocking traffic?
```
[End Code Block]

15.6 USEFUL COMMANDS

[Code Block (bash)]
```
kubectl get svc
kubectl get endpoints
kubectl get ingress
kubectl get networkpolicy
kubectl describe svc backend-service
kubectl describe ingress app-ingress
kubectl exec -it <pod> -- curl http://user-service:8080/actuator/health
kubectl exec -it <pod> -- nslookup user-service
```
[End Code Block]

---

16. INTERVIEW-READY ANSWERS

16.1 "WHAT IS A SERVICE IN KUBERNETES?"

[Code Block (text)]
```
A Service is a stable network abstraction in front of a dynamic set of Pods. It
provides a fixed virtual IP and DNS name and load balances traffic across health
y Pods selected by labels.
```
[End Code Block]

16.2 "HOW IS INGRESS DIFFERENT FROM SERVICE?"

[Code Block (text)]
```
Service gives internal stable access to Pods. Ingress handles external HTTP entr
y into the cluster and routes requests to Services based on host and path. Ingre
ss typically sits in front of Services, not Pods directly.
```
[End Code Block]

16.3 "WHAT IS EGRESS IN KUBERNETES?"

[Code Block (text)]
Egress is outbound traffic from Pods to destinations outside their current scope
, such as external APIs or the internet. Unlike Ingress, Egress is not usually r
epresented by a single built-in routing resource. It is controlled using Network
 Policies, cloud networking, proxies, or service mesh gateways.
[End Code Block]

16.4 "HOW DOES SPRING BOOT CALL ANOTHER SERVICE IN KUBERNETES?"

[Code Block (text)]
Spring Boot should call the target Service name, not Pod IPs. CoreDNS resolves t
he Service DNS name, kube-proxy forwards traffic to one of the healthy Pods behi
nd that Service, and readiness probes ensure only ready Pods receive traffic.
[End Code Block]

16.5 "WHAT IS THE DIFFERENCE BETWEEN INGRESS RESOURCE AND NETWORKPOLICY INGRESS?"

[Code Block (text)]
Ingress resource is for external HTTP routing into Services. NetworkPolicy ingre
ss is a Pod-level firewall rule that controls which sources may send traffic int
o selected Pods. They solve completely different problems.
[End Code Block]

16.6 "WHY DOES READINESS MATTER FOR NETWORKING?"

[Code Block (text)]
Because readiness determines whether a Pod is included in a Service's endpoint l
ist. If a Pod is not ready, it is removed from traffic routing even if the conta
iner is still running.
[End Code Block]

---

17. QUICK REVISION SHEET

ONE-LINE MAPPING

[Code Block (text)]
```
Pod            = one running app instance with its own IP
Service        = stable name and load balancing for Pods
ClusterIP      = internal-only Service
NodePort       = Service exposed on every node port
LoadBalancer   = cloud LB in front of one Service
Headless       = direct Pod discovery, no ClusterIP
CoreDNS        = cluster DNS resolver
Ingress        = external HTTP routing to Services
Egress         = outbound traffic from Pods to outside destinations
NetworkPolicy  = Pod-level ingress/egress firewall rules
kube-proxy     = programs node rules so Services forward traffic correctly
```
[End Code Block]

THE RELATIONSHIP IN ONE PICTURE

[Code Block (text)]

External user → Ingress → Service → Pod
Pod A → Service B → Pod B
Pod A → external API = Egress
[End Code Block]

GOLD STANDARD ANSWER
==========

[Code Block (text)]
In Kubernetes, I think about traffic in three directions. External client traffi
c enters through Ingress, which routes HTTP requests to Services. Services provi
de stable discovery and load balancing in front of dynamic Pods. Internal servic
e-to-service communication happens through ClusterIP Services and CoreDNS. Outbo
und traffic from Pods to third-party APIs or internet destinations is Egress, wh
ich is controlled separately using Network Policies and infrastructure-level net
working controls. For Spring Boot applications, the correct pattern is to config
ure internal dependencies using Service DNS names and treat readiness as part of
 networking because only ready Pods should receive traffic.
[End Code Block]

==========
FILE: K8s-Storage-StatefulSets-Story-Mode.md
==========

KUBERNETES STORAGE, STATEFULSETS, PV, PVC, AND DATABASES THROUGH STORY MODE
==========

> Your Spring Boot app works fine when everything is stateless. Then you add Pos
tgreSQL, file storage, caching, search, and background jobs that write data. Now
 containers restarting is no longer a harmless event. This guide explains Kubern
etes storage and StatefulSets in story mode, with a Spring Boot application as t
he baseline and a strong focus on interview decisions.

TABLE OF CONTENTS
==========

- How Storage Feels on Your Laptop (#1-how-storage-feels-on-your-laptop)
- What Breaks in Kubernetes (#2-what-breaks-in-kubernetes)
- The Big Mental Model (#3-the-big-mental-model)
- Stage 1: Stateless App with a Deployment (#4-stage-1-stateless-app-with-a-depl
oyment)
- Stage 2: Why Container Filesystems Are Not Real Persistence (#5-stage-2-why-co
ntainer-filesystems-are-not-real-persistence)
- Stage 3: Volumes, PV, PVC, and StorageClass (#6-stage-3-volumes-pv-pvc-and-sto
rageclass)
- Stage 4: Databases Enter the Story (#7-stage-4-databases-enter-the-story)
- Stage 5: Why Deployments Are Wrong for Many Databases (#8-stage-5-why-deployme
nts-are-wrong-for-many-databases)
- Stage 6: StatefulSet - Stable Identity and Stable Storage (#9-stage-6-stateful
set---stable-identity-and-stable-storage)
- When to Use StatefulSet Instead of Deployment (#10-when-to-use-statefulset-ins
tead-of-deployment)
- When Deployment Is Still the Right Choice (#11-when-deployment-is-still-the-ri
ght-choice)
- Database Patterns for Spring Boot Apps (#12-database-patterns-for-spring-boot-
apps)
- StatefulSet Networking and Headless Services (#13-statefulset-networking-and-h
eadless-services)
- Access Modes, Reclaim Policies, and Storage Tradeoffs (#14-access-modes-reclai

m-policies-and-storage-tradeoffs)
- Migrations, Backups, and Operations (#15-migrations-backups-and-operations)
- Common Mistakes and Troubleshooting (#16-common-mistakes-and-troubleshooting)
- Interview-Ready Answers (#17-interview-ready-answers)
- Quick Revision Sheet (#18-quick-revision-sheet)

---

1. HOW STORAGE FEELS ON YOUR LAPTOP

On your laptop, your Spring Boot app usually looks like this:

[Code Block (text)]
```
React frontend        → localhost:3000
Spring Boot backend   → localhost:8080
PostgreSQL            → localhost:5432
uploads folder        → ./uploads or local disk
```
[End Code Block]

Local persistence feels easy because:

- your process and your disk are on the same machine
- localhost never changes
- restarting the app does not wipe your database files if they are on your machine
- you rarely think about volume attachment, node movement, or data identity

That creates a false sense of simplicity.

---

2. WHAT BREAKS IN KUBERNETES

In Kubernetes, containers are ephemeral.

That means:

[Code Block (text)]
```
Pod dies → new Pod may come up on a different node
container restarts → writable layer is reset
Pod name may change → network identity may change
```
[End Code Block]

If your Spring Boot app writes important data into the container filesystem:

[Code Block (text)]
```
/tmp/uploads
/var/lib/postgresql/data
/app/generated-files
```
[End Code Block]

that data is not safely persistent.

So the moment you introduce databases or durable files, the storage model must change.

---

3. THE BIG MENTAL MODEL

If you remember only one storage model, remember this:

[Code Block (text)]
```
Volume        = storage mounted into a Pod
PV            = actual persistent storage resource in the cluster
PVC           = a request for persistent storage
StorageClass  = template/rules for dynamic storage provisioning
Deployment    = best for stateless workloads
StatefulSet   = best for stateful workloads needing stable identity and stable s
torage
Headless Svc  = gives each StatefulSet Pod stable DNS identity
```
[End Code Block]

The core flow is:

[Code Block (text)]
```
Pod needs storage
   ↓
Pod mounts a PVC
   ↓
PVC binds to a PV
   ↓
PV points to actual storage like EBS, EFS, NFS, GCE PD, Azure Disk
```
[End Code Block]

For StatefulSets:

[Code Block (text)]
```
StatefulSet
   ↓
volumeClaimTemplates
   ↓
one PVC per Pod
   ↓
stable Pod identity + stable disk identity
```
[End Code Block]

---

4. STAGE 1: STATELESS APP WITH A DEPLOYMENT

Start with the easy part.

Your Spring Boot API is stateless.
It reads and writes data to an external database, but the app itself does not ow
n durable local state.

That means a Deployment is perfect.

Real-life analogy:

[Code Block (text)]
```
A stateless app server is like a call center agent.
If one agent goes home, another agent can immediately take over.
The real customer record is stored in a central system, not in the agent's noteb
ook.
```
[End Code Block]

That is why most Spring Boot APIs run as Deployments.

Example:

[Code Block (yaml)]
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: myrepo/order-service:v1
          ports:
            - containerPort: 8080
```
[End Code Block]

So the first interview question is always:

[Code Block (text)]
```text
Is this workload actually stateful, or is it only consuming an external stateful
 system?
```
[End Code Block]

If it is only consuming one, Deployment is usually enough.

---

5. STAGE 2: WHY CONTAINER FILESYSTEMS ARE NOT REAL PERSISTENCE

5.1 THE STORY

A developer says:

[Code Block (text)]
```text
"We can just store uploaded files inside the container."
```
[End Code Block]

It works in testing.
Then the Pod restarts.
Files disappear.

That is the first storage lesson in Kubernetes.

5.2 WHY THIS HAPPENS

Every container has a writable filesystem layer, but it is tied to the container
 lifecycle.

If the Pod is recreated:

- that layer is not a durable storage contract

- the new Pod may run on a different node
- your data may be gone

5.3 THE RIGHT MENTAL MODEL

[Code Block (text)]
```
Container filesystem = temporary workspace
Persistent Volume    = real durable storage
```
[End Code Block]

This is why:

- temp files may use emptyDir
- important files should use persistent storage
- databases should not rely on plain container writable layers

---

6. STAGE 3: VOLUMES, PV, PVC, AND STORAGECLASS

6.1 VOLUME

A volume is storage mounted into a Pod.

Important types:

- emptyDir for temporary data shared inside a Pod
- configMap and secret volumes for configuration files
- persistent volumes through PVCs for durable data

6.2 PERSISTENT VOLUME (PV)

Real storage resource in the cluster.

Examples:

- EBS volume in AWS
- NFS share
- EFS filesystem
- Azure Disk
- GCE Persistent Disk

Real-life analogy:

[Code Block (text)]
```
PV is the actual apartment unit.
It exists independently of the tenant.
```
[End Code Block]

6.3 PERSISTENT VOLUME CLAIM (PVC)

PVC is what the application asks for.

Example:

[Code Block (yaml)]
```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
```
[End Code Block]

Real-life analogy:

[Code Block (text)]
```
PVC is the rental application.
The app says: I need 20Gi of durable storage with this access mode.
```
[End Code Block]

6.4 PV AND PVC RELATIONSHIP

[Code Block (text)]
```
App does not usually mount PV directly.
App mounts PVC.
PVC binds to PV.
```
[End Code Block]

That indirection matters because it separates application intent from infrastructure implementation.

6.5 STORAGECLASS

StorageClass defines how storage should be dynamically created.

Example:

[Code Block (text)]
```
gp3 on AWS
standard SSD on GCP
premium managed disk on Azure
```
[End Code Block]

Without StorageClass, admins may create PVs manually.
With StorageClass, PVC creation can trigger automatic disk provisioning.

6.6 THE MOST USEFUL INTERVIEW LINE

[Code Block (text)]
```
PV is the actual storage resource, PVC is the application's request for storage,
 and StorageClass defines how that storage should be provisioned.
```
[End Code Block]

---

7. STAGE 4: DATABASES ENTER THE STORY

Now your Spring Boot app adds PostgreSQL.

At first, you think:

[Code Block (text)]
"Let us run PostgreSQL in a Pod and be done with it."
[End Code Block]

But databases are different from stateless apps because they care about:

- durable storage
- consistent identity
- startup order
- replication roles
- controlled failover

Your application server only needs a stable database endpoint.
The database itself may need stable node identity.

That is exactly where StatefulSet appears.

---

8. STAGE 5: WHY DEPLOYMENTS ARE WRONG FOR MANY DATABASES

8.1 WHAT DEPLOYMENT ASSUMES

Deployment assumes Pods are interchangeable.

That is perfect for stateless workloads.
It is often wrong for clustered databases and brokers.

8.2 WHAT GOES WRONG WITH A DEPLOYMENT FOR A DATABASE

If you run a replicated database with a Deployment:

- Pod names are not stable
- Pod identity is not stable
- storage attachment is not naturally one-volume-per-replica
- startup and shutdown order are not guaranteed the way a clustered data system often needs

Real-life analogy:

[Code Block (text)]
Deployment treats all workers like interchangeable support staff.
Databases often need named roles like primary, replica-1, replica-2.
Those are not interchangeable identities.
[End Code Block]

8.3 IMPORTANT NUANCE

A single-instance database Pod with a PVC can technically be run via Deployment.
But for serious stateful systems, StatefulSet is usually the better model.

That nuance makes your interview answer stronger.

---

9. STAGE 6: STATEFULSET - STABLE IDENTITY AND STABLE STORAGE

9.1 THE STORY

You now need a PostgreSQL cluster or a Kafka cluster.
Each node has its own identity.
Each node must get its own disk.

That is where StatefulSet fits.

Real-life analogy:

[Code Block (text)]
```text
StatefulSet is like assigning fixed offices to named executives.

CEO office = room 1
CFO office = room 2
CTO office = room 3

If the CTO leaves and comes back, they return to the same office.
You do not randomly reassign everything every time.
```
[End Code Block]

9.2 WHAT STATEFULSET GUARANTEES

StatefulSet provides:

- stable Pod names like postgres-0, postgres-1, postgres-2
- stable network identity for each Pod
- stable storage per Pod through volumeClaimTemplates
- ordered startup and shutdown

9.3 STATEFULSET EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
```

```
        name: data
      spec:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 50Gi
```
[End Code Block]

9.4 WHAT THIS CREATES

[Code Block (text)]
```
postgres-0 → PVC data-postgres-0 → PV disk A
postgres-1 → PVC data-postgres-1 → PV disk B
postgres-2 → PVC data-postgres-2 → PV disk C
```
[End Code Block]

If postgres-1 dies:

[Code Block (text)]
```
new postgres-1 Pod comes back
it reattaches to data-postgres-1
same identity, same storage
```
[End Code Block]

That is the central StatefulSet promise.

---

10. WHEN TO USE STATEFULSET INSTEAD OF DEPLOYMENT

Use StatefulSet when the workload needs one or more of these:

10.1 STABLE POD IDENTITY

Examples:

- PostgreSQL replica sets
- MongoDB replica members
- Kafka brokers
- ZooKeeper nodes
- Elasticsearch nodes

If a node must be known as node-0 or broker-2, StatefulSet is the right fit.

10.2 STABLE PER-POD STORAGE

If each replica must keep its own data volume:

- StatefulSet

If all replicas are stateless and can be replaced freely:

- Deployment

10.3 ORDERED STARTUP AND SHUTDOWN

Some systems care about bringing nodes up in sequence.

Examples:

- clustered databases
- consensus systems
- brokers with leader/follower coordination

10.4 STATEFUL PEER DISCOVERY

If each replica must discover the others by deterministic DNS names:

[Code Block (text)]
```
db-0.db-headless
db-1.db-headless
db-2.db-headless
```
[End Code Block]

use StatefulSet.

10.5 STRONG INTERVIEW SENTENCE

[Code Block (text)]
```
I choose StatefulSet when Pods are not interchangeable and each replica needs st
able identity, stable storage, or ordered lifecycle semantics.
```
[End Code Block]

---

11. WHEN DEPLOYMENT IS STILL THE RIGHT CHOICE

This is just as important.

Do not overuse StatefulSet.

Use Deployment when:

- app instances are interchangeable
- no per-Pod durable identity is needed
- storage is external to the app
- scaling can happen in arbitrary order
- rolling updates should treat replicas as generic stateless workers

For most Spring Boot business APIs:

[Code Block (text)]
```
Deployment for the API
Managed database outside the cluster or dedicated stateful data tier
```
[End Code Block]

Examples where Deployment remains right:

- Spring Boot REST API
- React frontend
- auth gateway
- worker that consumes queue messages but stores state elsewhere
- cache client app that itself is stateless

Important nuance:

[Code Block (text)]
```
Your Spring Boot app using PostgreSQL does NOT mean your app should become a Sta
tefulSet.
Usually only the database would need StatefulSet, not the API.
```
[End Code Block]

That distinction matters a lot.

---

12. DATABASE PATTERNS FOR SPRING BOOT APPS

12.1 THE MOST COMMON PRODUCTION PATTERN

[Code Block (text)]
```
Spring Boot app        → Deployment
PostgreSQL/MySQL       → managed cloud DB or StatefulSet
Redis                  → managed service or StatefulSet depending on architectur
e
File uploads           → object storage, not local Pod disk
```
[End Code Block]

This is usually the cleanest design.

12.2 PATTERN A: MANAGED DATABASE OUTSIDE KUBERNETES

This is often the best answer in interviews when the company is on cloud.

Why?

- operationally simpler
- backups, failover, patching often handled by provider
- application remains fully stateless in Kubernetes

Spring Boot config example:

[Code Block (properties)]
```
spring.datasource.url=jdbc:postgresql://my-rds-endpoint:5432/appdb
spring.datasource.username=appuser
spring.datasource.password=${DB_PASSWORD}
```
[End Code Block]

12.3 PATTERN B: DATABASE INSIDE KUBERNETES VIA STATEFULSET

Use this when:

- self-managed data platform is required
- on-prem or hybrid environment
- you need in-cluster data services
- operators exist to manage replication/failover/backups

Examples:

- PostgreSQL operator
- MongoDB operator
- Strimzi for Kafka

12.4 PATTERN C: SINGLE INSTANCE DEV/TEST DATABASE

For dev or test, a simple database with one PVC may be enough.

But be careful not to treat that as production architecture.

---

13. STATEFULSET NETWORKING AND HEADLESS SERVICES

StatefulSets almost always pair with a Headless Service.

13.1 WHY

Normal Service gives one virtual IP.
That is great when you want generic load balancing.

But stateful clustered systems often need to know each peer individually.

That is why Headless Service exists.

13.2 EXAMPLE

[Code Block (yaml)]
```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```
[End Code Block]

This gives DNS entries like:

[Code Block (text)]
```text
postgres-0.postgres-headless.default.svc.cluster.local
postgres-1.postgres-headless.default.svc.cluster.local
postgres-2.postgres-headless.default.svc.cluster.local
```
[End Code Block]

That is how replicas discover each other.

13.3 INTERVIEW SUMMARY

[Code Block (text)]
```text
Deployment usually wants generic Service load balancing.
StatefulSet often wants Headless Service for stable per-Pod DNS identities.
```
[End Code Block]

---

14. ACCESS MODES, RECLAIM POLICIES, AND STORAGE TRADEOFFS

==========

14.1 ACCESS MODES

ReadWriteOnce (RWO)

[Code Block (text)]
```
One node mounts read-write.
Very common for block storage like EBS.
Common for single database instance or one Pod per volume.
```
[End Code Block]

ReadOnlyMany (ROX)

[Code Block (text)]
```
Many nodes mount read-only.
Less common for application write workloads.
```
[End Code Block]

ReadWriteMany (RWX)

[Code Block (text)]
```
Many nodes mount read-write.
Used with shared filesystems like EFS or NFS.
Useful when multiple Pods need the same shared files.
```
[End Code Block]

14.2 RECLAIM POLICIES

Delete

[Code Block (text)]
```
When PVC is deleted, underlying storage is deleted too.
Convenient, but risky if used carelessly for important data.
```
[End Code Block]

Retain

[Code Block (text)]
```
Storage stays even after PVC deletion.
Safer for critical data because accidental PVC deletion does not immediately des
troy the disk.
```
[End Code Block]

14.3 INTERVIEW TRADEOFF EXAMPLES

[Code Block (text)]
```
Database data volume   → often RWO, often Retain
Shared uploads folder  → often RWX
Temporary scratch data → emptyDir, not PV/PVC
```
[End Code Block]

---

15. MIGRATIONS, BACKUPS, AND OPERATIONS

Storage is not only about mounting disks.
The lifecycle also includes operations.

15.1 SCHEMA MIGRATIONS

For Spring Boot, schema migrations usually use:

- Flyway
- Liquibase

Best practice:

[Code Block (text)]
```
Do not make every app replica fight to run migrations.
Use a controlled migration step, init container, Job, or deployment pipeline sta
ge.
```
[End Code Block]

15.2 BACKUPS

A PVC is not a backup.

That is a critical interview sentence.

[Code Block (text)]
```
Persistent storage preserves data across Pod restarts.
Backup protects against corruption, accidental deletion, and disaster.
These are different concerns.
```
[End Code Block]

For databases, you still need:

- snapshots
- logical backups
- point-in-time recovery strategy
- tested restore process

15.3 SCALING STATE

Scaling a Deployment from 3 to 10 is straightforward.
Scaling a StatefulSet is more operationally sensitive because data systems have
replication and consistency concerns.

That is why stateful systems often need operators or specialized tooling.

15.4 OPERATORS MATTER

For serious stateful platforms, operators are often the right production answer.

Examples:

- PostgreSQL operator
- MongoDB operator
- Kafka operator

They automate:

- backups
- failover
- replication setup
- rolling maintenance
- cluster-aware upgrades

---

16. COMMON MISTAKES AND TROUBLESHOOTING

16.1 "WE STORED IMPORTANT DATA INSIDE THE CONTAINER FILESYSTEM"

[Code Block (text)]
Problem: data disappears after restart or reschedule.
Fix: move durable data to PV/PVC or external managed storage.
[End Code Block]

16.2 "WE USED DEPLOYMENT FOR A CLUSTERED STATEFUL SYSTEM"

[Code Block (text)]
Problem: no stable identity, poor storage mapping, awkward peer discovery.
Fix: use StatefulSet and usually a Headless Service.
[End Code Block]

16.3 "PVC IS PENDING"

Check:

- matching StorageClass exists
- requested size/access mode can be satisfied
- dynamic provisioner is installed and working
- cluster/cloud permissions allow disk provisioning

16.4 "POD CANNOT MOUNT VOLUME"

Check:

- event messages in kubectl describe pod
- storage class and CSI driver status
- node zone compatibility for attached disk
- access mode mismatch

16.5 "DATABASE CAME BACK BUT WITH WRONG DATA BEHAVIOR"

Check:

- is the same PVC actually reattached?
- was volume accidentally recreated?
- was reclaim policy Delete and data destroyed?
- is replication correctly configured?

16.6 USEFUL COMMANDS

[Code Block (bash)]
```
kubectl get pv
kubectl get pvc
kubectl describe pvc postgres-data
kubectl describe pod postgres-0
kubectl get storageclass
kubectl get statefulset
kubectl get pods -o wide
```
[End Code Block]

---

17. INTERVIEW-READY ANSWERS

17.1 "WHAT IS THE DIFFERENCE BETWEEN PV AND PVC?"

[Code Block (text)]
```
PV is the actual persistent storage resource available to the cluster. PVC is th
e application's request for storage. The Pod mounts the PVC, and Kubernetes bind
s that PVC to a suitable PV.
```
[End Code Block]

17.2 "WHAT IS A STATEFULSET AND HOW IS IT DIFFERENT FROM DEPLOYMENT?"

[Code Block (text)]
```
Deployment is for stateless, interchangeable Pods. StatefulSet is for Pods that
need stable identity, stable per-Pod storage, and ordered lifecycle behavior. St
atefulSet is commonly used for databases, brokers, and distributed systems.
```
[End Code Block]

17.3 "WHEN WOULD YOU USE STATEFULSET INSTEAD OF DEPLOYMENT?"

[Code Block (text)]
```
I use StatefulSet when replicas are not interchangeable and each Pod needs stabl
e identity or dedicated storage. Examples include PostgreSQL clusters, MongoDB r
eplica sets, Kafka brokers, ZooKeeper, and Elasticsearch nodes. If the workload
is stateless and only consumes external state, I use Deployment.
```
[End Code Block]

17.4 "SHOULD A SPRING BOOT APP WITH A DATABASE RUN AS A STATEFULSET?"

[Code Block (text)]
```
Usually no. The Spring Boot API itself is typically stateless and should run as
a Deployment. The database may run outside the cluster as a managed service, or
if self-hosted in Kubernetes, it may run as a StatefulSet.
```
[End Code Block]

17.5 "WHY DOES STATEFULSET USE A HEADLESS SERVICE?"

[Code Block (text)]
```
Because stateful replicas often need stable per-Pod DNS names rather than generi
c load balancing. A Headless Service gives direct DNS records like db-0.db-headl
ess, db-1.db-headless, and db-2.db-headless so replicas can identify each other
predictably.
```
[End Code Block]

17.6 "IS A PVC ENOUGH FOR DATABASE SAFETY?"

[Code Block (text)]
```
No. A PVC gives persistence across Pod restarts, but it is not a backup strategy
. You still need backups, snapshots, replication where appropriate, and tested r
estore procedures.
```
[End Code Block]

---

18. QUICK REVISION SHEET

ONE-LINE MAPPING

[Code Block (text)]
```
Deployment    = stateless app controller
StatefulSet   = stateful app controller with stable identity/storage
Volume        = storage mounted into a Pod
PV            = actual persistent storage resource
PVC           = request for persistent storage
StorageClass  = rules for dynamic provisioning
Headless Svc  = stable per-Pod DNS for stateful workloads
RWO           = one node read-write
RWX           = many nodes read-write
Retain        = keep storage after PVC deletion
Delete        = delete storage after PVC deletion
```
[End Code Block]

THE DECISION RULE

[Code Block (text)]
```
If Pods are interchangeable → Deployment
If Pods need stable identity or per-Pod durable storage → StatefulSet
```
[End Code Block]

THE STORAGE FLOW

[Code Block (text)]
```
App needs disk
  ↓
Pod mounts PVC
  ↓
PVC binds PV
  ↓
PV points to actual storage
```
[End Code Block]

GOLD STANDARD ANSWER

[Code Block (text)]
```
In Kubernetes, I use Deployments for stateless Spring Boot services because repl
icas are interchangeable and state lives outside the Pod. For durable storage, P
ods mount PVCs, which bind to PVs, often provisioned dynamically through a Stora
geClass. I use StatefulSets when the workload itself is stateful and each replic
a needs stable identity, stable network naming, or dedicated persistent storage,
 such as PostgreSQL, MongoDB, Kafka, or ZooKeeper. A PVC provides persistence, b
```

ut it does not replace backups or proper database operations.
[End Code Block]

==========
FILE: YAML-Examples/01-foundation.yaml
==========

```yaml
# This file defines the basic non-runtime building blocks for an application nam
espace.
# These objects do not serve traffic themselves, but the runtime objects depend
on them.

---
apiVersion: v1                       # Core API group for foundational objects.
kind: Namespace                      # Logical isolation boundary for this app/en
vironment.
metadata:
  name: interview-demo               # All namespaced resources below live inside
 this namespace.

---
apiVersion: v1
kind: ServiceAccount                 # Identity that Pods use when talking to the
 K8s API.
metadata:
  name: order-service-sa             # Deployment will reference this via spec.se
rviceAccountName.
  namespace: interview-demo
automountServiceAccountToken: false # Disable token mount if app does not need A
PI access.

---
apiVersion: v1
kind: ConfigMap                      # Stores non-sensitive configuration.
metadata:
  name: order-config
  namespace: interview-demo
data:
  SPRING_PROFILES_ACTIVE: kubernetes,prod   # App runtime profile selection.
  DB_HOST: postgres-service                  # Service DNS name, not Pod IP.
  DB_PORT: "5432"                           # String because ConfigMap data is t
ext.
  LOG_LEVEL: INFO                            # Example application config.
  INVENTORY_BASE_URL: http://inventory-service:8080  # Internal service call via
 Service DNS.

---
apiVersion: v1
kind: Secret                         # Stores sensitive configuration.
metadata:
  name: order-secrets
  namespace: interview-demo
type: Opaque                         # Generic key-value secret type.
stringData:                          # Plain strings here; API converts to base64
 in storage.
  DB_USERNAME: appuser
  DB_PASSWORD: change-me
  JWT_SECRET: replace-me

---
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role                          # Namespace-scoped permission set.
metadata:
  name: app-read-config
  namespace: interview-demo
rules:
  - apiGroups: [""]                # Empty string means the core API group.
    resources: ["configmaps"]      # Resource type this role can access.
    verbs: ["get", "list"]         # Allowed actions.

---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding                   # Connects a Role to a subject.
metadata:
  name: order-service-config-binding
  namespace: interview-demo
subjects:
  - kind: ServiceAccount            # Binding is granting permissions to a Servi
ceAccount.
    name: order-service-sa
    namespace: interview-demo
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: app-read-config

# Relationship summary:
# Namespace groups everything.
# ServiceAccount gives Pod identity.
# ConfigMap and Secret feed environment/config into Pods.
# Role and RoleBinding decide what that ServiceAccount may do.


================================================================================
FILE: YAML-Examples/02-stateless-app-traffic.yaml
================================================================================

# This file shows the main stateless app flow:
# Deployment creates Pods, Service discovers them, Ingress exposes them external
ly.

---
apiVersion: apps/v1
kind: Deployment                    # Best controller for stateless Spring Boot
apps.
metadata:
  name: order-service
  namespace: interview-demo
  labels:
    app: order-service              # Shared identity label.
spec:
  replicas: 3                       # Desired Pod count.
  revisionHistoryLimit: 5           # Old ReplicaSets kept for rollback.
  selector:
    matchLabels:
      app: order-service            # Must match template labels exactly.
  strategy:
    type: RollingUpdate             # Safer than Recreate for APIs.
    rollingUpdate:
      maxUnavailable: 0             # Never take old Pods down before new are re
ady.
      maxSurge: 1                   # Create one extra Pod during rollout.
  template:
```

```yaml
    metadata:
      labels:
        app: order-service          # Service selector uses this label.
        version: v1                 # Useful for rollout visibility or canary st
rategies.
      annotations:
        prometheus.io/scrape: "true"           # Tell Prometheus to scrape metri
cs.
        prometheus.io/path: "/actuator/prometheus"
        prometheus.io/port: "8080"
    spec:
      serviceAccountName: order-service-sa     # References ServiceAccount from
01-foundation.yaml.
      terminationGracePeriodSeconds: 30        # Time given for graceful shutdow
n.
      containers:
        - name: order-service
          image: myregistry/order-service:v1.2.3   # Pinned image for reproducib
ility.
          imagePullPolicy: IfNotPresent            # Avoid unnecessary pulls whe
n image already exists.
          ports:
            - name: http
              containerPort: 8080                  # App listens here inside the
 Pod.
          env:
            - name: SPRING_PROFILES_ACTIVE
              valueFrom:
                configMapKeyRef:
                  name: order-config               # Pulls from ConfigMap.
                  key: SPRING_PROFILES_ACTIVE
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: order-config
                  key: DB_HOST
            - name: DB_PORT
              valueFrom:
                configMapKeyRef:
                  name: order-config
                  key: DB_PORT
            - name: DB_USERNAME
              valueFrom:
                secretKeyRef:
                  name: order-secrets              # Pulls sensitive values from
 Secret.
                  key: DB_USERNAME
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: order-secrets
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: "250m"                          # Scheduler reserves this much
 CPU.
              memory: "256Mi"                     # Scheduler reserves this much
memory.
            limits:
              cpu: "1000m"                        # CPU above this is throttled.
              memory: "512Mi"                     # Memory above this can cause O
OMKilled.
```

```
          startupProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            failureThreshold: 30                  # 30 * 2s = 60s startup allowa
nce.
            periodSeconds: 2
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10                     # Controls whether Pod becomes
 a Service endpoint.
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            initialDelaySeconds: 20
            periodSeconds: 20                     # Restarts deadlocked or stuck
 containers.
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]         # Gives kube-proxy time to stop
routing before shutdown.
          securityContext:
            runAsNonRoot: true
            runAsUser: 1000
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]

---
apiVersion: v1
kind: Service                       # Stable internal address in front of Pods.
metadata:
  name: order-service
  namespace: interview-demo
spec:
  type: ClusterIP                   # Internal-only service. Normal default for
microservice traffic.
  selector:
    app: order-service              # Finds Pods created by the Deployment.
  ports:
    - name: http
      port: 80                      # Service port clients call.
      targetPort: http              # Forwards to container port named http.

---
apiVersion: networking.k8s.io/v1
kind: Ingress                       # External HTTP routing into Services.
metadata:
  name: app-ingress
  namespace: interview-demo
spec:
  ingressClassName: nginx           # Tells cluster which Ingress controller sho
uld implement this.
  rules:
    - host: app.example.com
      http:
```

```
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: order-service  # Ingress routes to Service, not directly t
o Pods.
                port:
                  number: 80

# Relationship summary:
# Deployment creates Pods.
# Service selects Pods by labels from the Deployment template.
# Readiness probe controls whether Pods appear behind the Service.
# Ingress routes external HTTP traffic to the Service.


================================================================================
FILE: YAML-Examples/03-scaling-resilience-security.yaml
================================================================================

# This file adds the production controls around the stateless app:
# autoscaling, disruption safety, and Pod-level traffic restrictions.

---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler       # Scales the Deployment replica count.
metadata:
  name: order-service-hpa
  namespace: interview-demo
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service             # HPA points to the Deployment, not the Pods
 directly.
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60    # HPA compares observed CPU to this target.

---
apiVersion: policy/v1
kind: PodDisruptionBudget           # Protects availability during voluntary dis
ruptions.
metadata:
  name: order-service-pdb
  namespace: interview-demo
spec:
  minAvailable: 2                   # Always keep at least 2 matching Pods runni
ng.
  selector:
    matchLabels:
      app: order-service            # Applies to Pods from the Deployment.

---
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy                 # Pod-level firewall.
metadata:
  name: order-service-policy
  namespace: interview-demo
spec:
  podSelector:
    matchLabels:
      app: order-service            # Policy applies to these Pods.
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: ingress-nginx    # Example: only traffic coming from ingress
controller Pods.
      ports:
        - protocol: TCP
          port: 8080
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: interview-demo
      ports:
        - protocol: TCP
          port: 5432                # Example: allow DB traffic inside namespace
.
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
      ports:
        - protocol: TCP
          port: 443                 # Example: allow outbound HTTPS to external
APIs.

# Relationship summary:
# HPA reads metrics and changes Deployment replicas.
# PDB protects those Pods during node drains or rolling operations.
# NetworkPolicy controls who may talk to the Pods and where the Pods may connect
.


================================================================================
FILE: YAML-Examples/04-stateful-database.yaml
================================================================================

# This file shows the stateful side of the system.
# The API usually stays a Deployment, but the database may need StatefulSet sema
ntics.

---
apiVersion: v1
kind: Service                       # Headless Service for stable per-Pod DNS.
metadata:
  name: postgres-headless
  namespace: interview-demo
spec:
  clusterIP: None                   # Headless means no virtual IP; DNS returns
Pod identities directly.
  selector:
```

```yaml
    app: postgres
  ports:
    - name: postgres
      port: 5432
      targetPort: 5432

---
apiVersion: apps/v1
kind: StatefulSet                   # Controller for workloads needing stable id
entity and storage.
metadata:
  name: postgres
  namespace: interview-demo
spec:
  serviceName: postgres-headless    # Must point to the Headless Service above.
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: order-secrets
                  key: DB_USERNAME
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: order-secrets
                  key: DB_PASSWORD
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data  # Database files live on pers
istent volume here.
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes:
          - ReadWriteOnce           # Typical block storage mode for one Pod per
 volume.
        storageClassName: gp3       # Dynamic provisioning template.
        resources:
          requests:
            storage: 50Gi           # Each replica gets its own 50Gi PVC.

# Relationship summary:
# StatefulSet creates Pods named postgres-0, postgres-1, postgres-2.
# Each Pod gets its own PVC from volumeClaimTemplates.
# Headless Service gives each Pod stable DNS like postgres-0.postgres-headless.
# This is why StatefulSet is used for stateful clustered systems instead of Depl
oyment.
```